



**COMSATS University Islamabad, Pakistan**

## **SheRides**

**A project presented to  
COMSATS Institute of Information Technology, Islamabad**

**In partial fulfillment  
of the requirement for the degree of**

***Bachelor of Science in Software Engineering (2022-2026)***

**By**

**Warda Ashfaq CIIT/FA22-BSE-048/SWL**

**Amina Riaz CIIT/FA22-BSE-068/SWL**

**Uswah Abid CIIT/FA22-BSE-107/SWL**

# DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Warda Ashfaq

Amina Riaz

Uswah Abid

-----

-----

-----

# CERTIFICATE OF APPROVAL

It is to certify that the final year project "**SheRides: Women-Only Ride-Sharing Platform**" was developed by **Warda Ashfaq (CIIT/FA22-BSE-048/SWL)**, **Amina Riaz (CIIT/FA22-BSE-068/SWL)**, and **Uswah Abid (CIIT/FA22-BSE-107/SWL)** under the supervision of **Dr. Javed Ferzund** and co-supervisor **Hafsa Saleem** and that in their opinion, it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Science.

-----  
**Supervisor**

**Dr. Javed Ferzund**

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

-----  
**External Examiner**

-----  
**Head of Department**

**Dr. Zafar Iqbal Roy**

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

# Executive Summary

SheRides is a women-only ride-sharing platform developed to provide safe, reliable, and comfortable transportation for female passengers in Pakistan. The project addresses the growing concerns of harassment, insecurity, and lack of inclusivity in existing ride-hailing services such as InDrive and Yango, as well as unsafe public transport. Its primary goal is to create a trusted mobility ecosystem where women can travel independently and securely. The system consists of a Flutter-based mobile application, a Node.js and Express backend integrated with Firebase Authentication and MongoDB, and a MERN-based admin panel. Key features include real-time GPS tracking, one-tap SOS alerts, trusted contacts for live ride monitoring, recurring ride booking, AI-assisted mechanic support, secure payments, and driver verification managed by administrators. The project followed the Incremental Software Process Model with Object-Oriented Programming principles, allowing each module to be developed, tested, and integrated in stages for improved stability and quality.

The completed system successfully delivers all planned functionalities. Passengers can book instant or recurring rides, share live locations, trigger SOS alerts within five seconds, and request mechanic assistance. Drivers can register, upload verification documents, accept rides, manage availability, and track earnings, while administrators can verify drivers, monitor rides, manage complaints, and handle SOS incidents through the web dashboard. The application was tested using unit, integration, functional, and automated testing with Flutter Test, Postman, and Firebase Test Lab, achieving all major functional requirements, approximately 99% system uptime, and support for up to 500 concurrent ride requests. The project was developed using Flutter, Node.js, Express, MongoDB, Firebase Authentication, React.js, Google Maps API, Firebase Cloud Messaging, Figma, and Draw.io. Team responsibilities were divided among three members: Warda Ashfaq developed the mobile application, SOS module, and real-time tracking incident management, and driver verification; Amina Riaz just implemented the admin dashboard, and Uswah Abid just designed the UI/UX. Overall, SheRides provides a practical solution for improving women's transportation safety in Pakistan while strengthening the team's expertise in full-stack development, software engineering, testing, and collaborative project management. Future enhancements include AI-based driver safety scoring, intercity ride support, voice-activated SOS, and offline functionality.

# Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Dr Javed Ferzund” and our Co- Supervisor “Ms. Hafsa Saleem”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Warda Ashfaq

Amina Riaz

Uswah Abid

-----

-----

-----

# Abbreviations

|             |                                     |
|-------------|-------------------------------------|
| <b>API</b>  | Application Programming Interface   |
| <b>DFD</b>  | Data Flow Diagram                   |
| <b>FR</b>   | Functional Requirements             |
| <b>GPS</b>  | Global Positioning System           |
| <b>MERN</b> | MongoDB, Express, React, Node.js    |
| <b>NOR</b>  | Non-Functional Requirement          |
| <b>OOP</b>  | Object-Oriented Programming         |
| <b>RTM</b>  | Requirements Traceability Matrix    |
| <b>SDD</b>  | Software Design Description         |
| <b>SOS</b>  | Save Our Souls (Emergency Signal)   |
| <b>SRS</b>  | Software Requirements Specification |
| <b>SSL</b>  | Secure Sockets Layer                |
| <b>UC</b>   | Use Case                            |
| <b>UI</b>   | User Interface                      |
| <b>UML</b>  | Unified Modeling Language           |
| <b>UE</b>   | User Experience                     |

## Table of Contents

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 1. Introduction.....                                                       | 2  |
| 1.1 Brief .....                                                            | 3  |
| 1.1.1 Purpose of the Project.....                                          | 3  |
| 1.1.2 Scope .....                                                          | 3  |
| 1.1.3 Overview of Project.....                                             | 4  |
| 1.1.4 Outcome .....                                                        | 4  |
| 1.2 Relevance to Course Modules.....                                       | 5  |
| 1.2.1 Software Requirements Engineering (SRE).....                         | 6  |
| 1.2.2 Software Design and Architecture.....                                | 6  |
| 1.2.3 Database Systems .....                                               | 7  |
| 1.2.4 Human Computer Interaction .....                                     | 8  |
| 1.2.5 Mobile and Backend Development with Object Oriented Programming..... | 8  |
| 1.2.6 Software Quality Assurance and Testing .....                         | 10 |
| 1.3 Project Background.....                                                | 11 |
| 1.4 Literature Review.....                                                 | 14 |
| 1.5 Analysis from Literature Review (in the context of this project).....  | 17 |
| 1.6 Tools and Technologies .....                                           | 20 |
| 1.7 Methodology and Software Lifecycle for This Project.....               | 21 |
| 1.7.1 Methodology .....                                                    | 21 |
| 1.7.2 Understanding the Incremental Model in Practice .....                | 22 |
| 1.7.3 Planning and Building Initial Increments.....                        | 23 |
| 1.7.4 Feedback and Iterative Improvement .....                             | 23 |
| 1.7.5 Balancing Design Flexibility and Technical Constraints .....         | 24 |
| 1.7.6 Integration with Core Systems .....                                  | 24 |
| 1.7.7 Why Not Waterfall or Agile? .....                                    | 25 |
| 1.7.8 Supporting Usability and User-Centric Design.....                    | 25 |
| 1.7.9 Managing Testing Within the Incremental Model .....                  | 25 |
| 1.7.10 Handling Feedback and Iteration Cycles.....                         | 26 |
| 1.7.11 Coordination and Team Collaboration .....                           | 27 |
| 1.7.12 Tools and Technologies Supporting the Lifecycle .....               | 28 |
| 1.8 Rationale behind Selected Methodology .....                            | 28 |
| 1.8.1 Object-Oriented Programming (OOP) Approach.....                      | 28 |
| 1.8.2 Incremental Methodology (Iterative Development).....                 | 29 |
| 1.8.3 Structured Design (Modular Architecture).....                        | 29 |
| 1.8.4 Choice of SDLC (Incremental + Iterative).....                        | 29 |
| 2. Problem Definition.....                                                 | 31 |
| 2.1 Problem Statement .....                                                | 31 |

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 2.2 Deliverables and Development Requirements .....             | 32 |
| 2.2.1 Deliverables.....                                         | 32 |
| 2.2.2 Development Requirements .....                            | 33 |
| 2.3 Current System.....                                         | 34 |
| 2.3.1 Unsafe Public Transport Options .....                     | 34 |
| 2.3.2 Safety Issues in Existing Ride-Hailing Applications ..... | 35 |
| 2.3.3 Limited Emergency Response and Monitoring.....            | 36 |
| 2.3.4 Driver Verification Gaps.....                             | 36 |
| 2.3.5 Lack of Passive Monitoring Features .....                 | 37 |
| 2.3.6 No Women-Only Driver Option.....                          | 37 |
| 2.3.7 Summary of Limitations.....                               | 38 |
| 2.4 Summary .....                                               | 38 |
| 3. Requirement Analysis.....                                    | 41 |
| 3.1 Introduction to Requirement Analysis .....                  | 41 |
| 3.2 System Overview .....                                       | 41 |
| 3.3 Actors in the System .....                                  | 42 |
| 3.3.1 Passenger.....                                            | 42 |
| 3.3.2 Driver .....                                              | 42 |
| 3.3.3 Administrator .....                                       | 43 |
| 3.4 Use Case Diagrams .....                                     | 43 |
| 3.5 Use Case Descriptions .....                                 | 45 |
| 3.5.1 Book Ride Use Case.....                                   | 45 |
| 3.5.2 Send SOS Alert Use Case .....                             | 47 |
| 3.5.3 Request Mechanic Assistance Use Case .....                | 50 |
| 3.5.4 Verify Driver and Handle Complaints Use Case .....        | 53 |
| 3.6 Functional Requirements .....                               | 56 |
| 3.6.1 Women Safety and SOS Alerts (FR-1) .....                  | 56 |
| 3.6.2 Recurring Rides (FR-2).....                               | 57 |
| 3.6.3 Mobile Mechanic Assistance (FR-3).....                    | 57 |
| 3.6.4 Trusted Contact Circle (FR-4).....                        | 58 |
| 3.6.5 Payment Method (FR-5).....                                | 59 |
| 3.6.6 Driver Registration and Verification (FR-6) .....         | 59 |
| 3.6.7 Admin Panel (FR-7) .....                                  | 60 |
| 3.6.8 Customer Care (FR-8).....                                 | 60 |
| 3.7 Non-Functional Requirements .....                           | 61 |
| 3.7.1 Performance .....                                         | 61 |
| 3.7.2 Security.....                                             | 61 |
| 3.7.3 Reliability and Availability .....                        | 62 |
| 3.7.4 Usability .....                                           | 62 |
| 3.7.5 Maintainability .....                                     | 62 |



|                                                |     |
|------------------------------------------------|-----|
| 3.7.6 Performance Monitoring and Logging ..... | 63  |
| 4. Design and Architecture .....               | 65  |
| 4.1 System Architecture .....                  | 65  |
| 4.1.1 Architecture Style.....                  | 65  |
| 4.1.2 Components.....                          | 65  |
| 4.1.3 Interactions .....                       | 66  |
| 4.1.4 System Architecture Diagram .....        | 67  |
| 4.2 Data Representation .....                  | 68  |
| 4.2.1 Database Design.....                     | 68  |
| 4.2.2 Data Entities .....                      | 69  |
| 4.2.3 Data Dictionary .....                    | 70  |
| 4.2.4 Entity–Relationship Representation ..... | 71  |
| 4.2.4 Data Flow .....                          | 73  |
| 4.3 Process Flow / Representation .....        | 77  |
| 4.3.1 Login and Authentication Process.....    | 77  |
| 4.3.2 Ride Booking Process .....               | 77  |
| 4.3.3 SOS Alert Process .....                  | 77  |
| 4.3.4 Driver Verification Process .....        | 77  |
| 4.3.5 Payment Processing Workflow .....        | 78  |
| 4.4 Design Models .....                        | 78  |
| 4.4.1 Class Model Description .....            | 78  |
| 4.4.2 Sequence Model Description.....          | 80  |
| 4.4.3 State Transition Model Description.....  | 80  |
| 4.4.4 Component Model Description .....        | 81  |
| 5. Implementation .....                        | 84  |
| 5.1 Algorithms .....                           | 84  |
| 5.1.1 User Authentication Algorithm .....      | 84  |
| 5.1.2 Ride Booking Algorithm .....             | 85  |
| 5.1.3 SOS Alert Algorithm.....                 | 87  |
| 5.1.4 Driver Verification Algorithm.....       | 89  |
| 5.1.5 Payment Processing Algorithm .....       | 90  |
| 5.2 External APIs .....                        | 93  |
| 5.3 User Interface.....                        | 94  |
| 5.3.1 Admin Panel.....                         | 95  |
| 5.3.2 Signup Screen.....                       | 95  |
| 5.3.3 Home Screen (Passenger).....             | 96  |
| 5.3.4 Ride History Screen (Passenger) .....    | 97  |
| 5.3.5 Trip Details Screen (Driver).....        | 98  |
| 5.3.6 Document Upload Screen (Driver).....     | 99  |
| 5.3.7 Earnings Screen (Driver).....            | 100 |

|                                                          |     |
|----------------------------------------------------------|-----|
| 5.4 Chapter Summary .....                                | 101 |
| 6. Testing and Evaluation .....                          | 103 |
| 6.1 Manual Testing .....                                 | 103 |
| 6.1.1 System Testing .....                               | 103 |
| 6.1.2 Unit Testing.....                                  | 104 |
| 6.1.3 Functional Testing.....                            | 105 |
| 6.1.4 Integration Testing .....                          | 107 |
| 6.2 Automated Testing.....                               | 108 |
| 6.3 Testing Summary .....                                | 109 |
| 7. Conclusion .....                                      | 111 |
| 7.2 Future Work .....                                    | 112 |
| 7.2.1 AI-Based Safety Scoring.....                       | 112 |
| 7.2.2 Integration with Third-Party Payment Gateways..... | 112 |
| 7.2.3 Intercity Rides .....                              | 112 |
| 7.2.4 Voice-Activated SOS .....                          | 113 |
| 7.2.5 Predictive Analytics for Demand Forecasting.....   | 113 |
| 7.2.6 Offline Mode Functionality.....                    | 113 |
| 7.2.7 Driver Training and Certification Module .....     | 113 |
| 7.2.8 Emergency Services Integration.....                | 113 |
| 7.2.9 Multi-Language Support .....                       | 113 |
| 7.2.10 Ride Sharing Feature .....                        | 113 |
| 8. References.....                                       | 114 |

## List of Tables

|                                                                                        |     |
|----------------------------------------------------------------------------------------|-----|
| Table 3.1: Structured description of Ride Booking.....                                 | 45  |
| Table 3.2: Structured description of the Send SOS Alert use case.....                  | 47  |
| Table 3.3: Structured description of the Request Mechanic Assistance use case.....     | 50  |
| Table 3.4: Structured description of Verify Driver and Handle Complaints use case..... | 53  |
| Table 3.5: FR-1 (SOS Alert).....                                                       | 56  |
| Table 3.6: FR-2 Recurring Ride.....                                                    | 57  |
| Table 3.7: FR-3 Mechanic Assistant.....                                                | 58  |
| Table 3.8: FR-4 Share Location.....                                                    | 58  |
| Table 3.9: FR-5 Secure Payment and Authentication.....                                 | 59  |
| Table 3.10: FR-6 Driver Profile.....                                                   | 59  |
| Table 3.11: FR-7 Admin Monitoring.....                                                 | 60  |
| Table 3.12: FR-8 24/7 Support and Complaint Handling.....                              | 60  |
| Table 4.1: Entities stored in MongoDB.....                                             | 69  |
| Table 4.2: Attributes and Method Associate.....                                        | 70  |
| Table 5.1: Details of APIs Used in the Project.....                                    | 93  |
| Table 6.1: Unit Testing.....                                                           | 105 |
| Table 6.2: Functional Testing.....                                                     | 106 |
| Table 6.3: Integrated Testing.....                                                     | 107 |
| Table 6.4: Automated Testing.....                                                      | 108 |
| Table 7.1: Key Achievements of SheRides.....                                           | 111 |

## List of Figures

|                                              |     |
|----------------------------------------------|-----|
| Figure 3.1: Use Case Diagram.....            | 44  |
| Figure 4.1: Architecture Diagram.....        | 68  |
| Figure 4.2: Entity-Relationship Diagram..... | 72  |
| Figure 4.3: Data Flow Diagram.....           | 73  |
| Figure 4.4: Sequence Diagram.....            | 75  |
| Figure 4.5: Activity Diagram.....            | 76  |
| Figure 4.6: Class Diagram.....               | 79  |
| Figure 4.7: State Transition Diagram.....    | 81  |
| Figure 5.1: Admin Panel.....                 | 95  |
| Figure 5.2: User SignUp.....                 | 96  |
| Figure 5.3: Home Screen.....                 | 97  |
| Figure 5.4: History Page.....                | 98  |
| Figure 5.5: Driver Detail Page.....          | 99  |
| Figure 5.6: Document Page.....               | 100 |
| Figure 5.7: Earning Page.....                | 101 |

# **Chapter 1**

## **INTRODUCTION**

# 1. Introduction

SheRides is a women-only ride-sharing platform developed to provide safe, reliable, and comfortable transportation exclusively for female passengers in Pakistan. The core idea behind the platform is to blend traditional ride-hailing mechanics with structured safety features and strategic decision-making for women's security. Unlike many existing ride-hailing applications that operate on mixed-gender models, SheRides introduces a more secure approach where each ride requires verification and safety monitoring. The system presents users with a mobile interface divided into passenger and driver modules, each accessible after role-based authentication. Passengers begin by reviewing available drivers and estimated fares from the home screen, which acts as the central hub for managing ride requests.

Safety features such as SOS alerts and trusted contacts play a critical role, as they are needed to ensure passenger security during rides. Passengers must activate these safety tools before or during a trip. Once a ride is booked, the real-time tracking system provides live location updates, helping trusted contacts monitor the journey. The emergency response system is alert-based, where passengers use a dedicated SOS button representing immediate help requests. Each ride consists of the passenger requesting a ride, the driver accepting, followed by real-time tracking, and finally ride completion with fare calculation.

The operational loop focuses on safety, real-time monitoring, and secure execution rather than random outcomes. Rides are not visualized with complex animations but are presented through structured interactions and status updates, keeping development within manageable scope while still providing meaningful feedback. After each ride, the trip summary is displayed, and the ride history is updated based on whether the ride was completed successfully or canceled. As passengers use the platform more frequently, safety features become more refined, encouraging continued trust and engagement.

SheRides does not include male users, offline ride booking, or third-party payment gateways in this phase, which helps maintain the project's focus and feasibility. Instead, it emphasizes security, structure, and simplicity, while still offering enough depth to ensure passenger confidence. The project aims to deliver a satisfying and safe ride-sharing experience through well-defined safety mechanics, logical user and driver management, and progressive system monitoring. This introduction outlines the core features and mechanics that define SheRides, setting the stage for further sections of this report. The project aims to deliver a satisfying and safe ride-sharing experience through well-defined safety mechanics, logical user and driver management, and progressive system monitoring.

## **1.1 Brief**

SheRides is a specialized, women-exclusive ride-sharing application engineered to directly confront the pervasive challenge of safe mobility for women in Pakistan. The system consists of a mobile application for passengers and drivers, and a web-based admin portal for system monitoring[1]. The mobile app allows female passengers to book rides either instantly or on a scheduled basis, with core features including live GPS tracking, SOS emergency alerts, trusted contact circle, and mobile mechanic assistance. Drivers can register through the app, undergo document verification, and accept ride requests once approved by the admin.

The backend is developed using Node.js + Express with Firebase Authentication and MongoDB, ensuring scalability, fast communication, and secure data handling. The admin panel built with MERN stack allows administrators to verify drivers, monitor live rides, manage complaints, and oversee system operations. The project was developed using the Incremental Software Process Model, following Object-Oriented Programming principles.

### **1.1.1 Purpose of the Project**

The purpose of SheRides is to dismantle the significant barrier of unsafe transportation that restricts the freedom, opportunity, and daily lives of women in Pakistan. The platform aims to achieve this by creating a dedicated, women-exclusive ride-sharing ecosystem that prioritizes uncompromising safety above all else. SheRides goes beyond basic ride-hailing by building a verified community of female passengers and drivers, thereby establishing an inherent layer of trust. This foundation is strengthened through integrated, proactive security features — including real-time ride tracking, one-tap SOS alerts to trusted contacts and authorities, and discreet emergency protocols — that provide active protection throughout every journey. The system is designed to provide safe, reliable, and comfortable travel options for women by connecting verified female drivers with female passengers.

### **1.1.2 Scope**

SheRides is a mobile and web-based ride-sharing system developed exclusively for women. The main purpose of this software is to ensure safe and secure travel by introducing features that protect users during rides and simplify daily commuting. The system allows passengers to book rides, share real-time locations, use SOS alerts, and connect with trusted contacts in emergencies. Drivers can register, verify their profiles, and accept ride requests, while administrators monitor trips, verify drivers, and handle complaints through a web portal. Additional modules such as Mobile Mechanic Assistance, Recurring Rides, and AI-based guidance further improve reliability and user experience. Overall, SheRides aims to empower

women by making mobility safe, accessible, and independent.

The system will not allow male passengers or male drivers. It will not provide offline ride booking, as all bookings must be done online. The system will not integrate third-party payment gateways in this phase, and only basic or wallet-based payments will be supported. It will not provide AI-based predictive features beyond simple rule-based alerts in this version. It will not cover long-distance intercity rides, as the focus remains on urban and semi-urban mobility.

### **1.1.3 Overview of Project**

SheRides is a specialized, women-exclusive ride-sharing application engineered to directly confront the pervasive challenge of safe mobility for women in Pakistan. Recognizing that transportation insecurity is a significant barrier to women's education, employment, and social participation, SheRides is more than just an application — it is a safety-focused mobility ecosystem. It creates a protected digital and physical space where female passengers can connect exclusively with verified female drivers, ensuring peace of mind from the moment a ride is booked to its completion.

The platform ensures safety through integrated features including secure user verification, real-time ride tracking, in-app SOS with emergency alerts, and a trusted contact circle for passive monitoring. To serve daily commuters, SheRides also supports scheduling for recurring rides. Key features include real-time ride tracking, SOS and emergency alerts, recurring rides for daily commuters, a trusted contact circle for passive protection, and a clean Flutter interface for both passengers and drivers. The backend is developed using Node.js + Express, Firebase Authentication, and MongoDB, ensuring scalability, fast communication, and secure data handling.

SheRides covers two main user roles: passengers, who can book rides, schedule recurring rides, share location with trusted contacts, and use emergency safety tools; and drivers, who can receive ride requests, manage profiles, and complete rides. The admin panel allows administrators to verify drivers, monitor live rides, manage complaints, and oversee system operations.

### **1.1.4 Outcome**

- The passenger successfully books and completes a ride by applying safety features and selecting appropriate ride options
- Each ride updates the ride history, visually reflecting completed trips and the passenger's overall travel record



- The use of a SOS alert system enhances passenger security by encouraging immediate emergency response when needed
- As passengers use the platform more frequently, they encounter increasingly verified drivers, making each ride more secure
- The passenger becomes more skilled at managing trusted contacts and safety tools, which directly impact security during rides
- The trusted contact system allows passengers to share live location with family members, improving pre-ride and during-ride decision-making and reinforcing thoughtful safety practices
- Resource and ride management teach passengers how to balance preparation and risk before each trip
- A gradual security learning curve allows passengers to understand safety features step-by-step, creating a smooth and structured experience
- The platform offers a rewarding sense of security after each ride, motivating continued use
- Once all ride safety features including SOS alerts and trusted contacts are utilized, the passenger achieves complete security coverage during the journey
- The project delivers an interactive, safety-driven ride-sharing platform that aligns with course learning objectives and practical implementation goals.

## **1.2 Relevance to Course Modules**

The SheRides project demonstrates the integration of concepts and practical skills acquired from various courses throughout the academic journey. It serves as a bridge between theoretical knowledge and real-world application in the domain of software and ride-sharing system development. From the Object-Oriented Programming course, concepts such as class design, inheritance, encapsulation, and modular coding were applied to define structured entities like passengers, drivers, rides, SOS alerts, and trusted contacts. The Software Engineering course played a central role in guiding the development process, including requirements gathering, system design, use case creation, UML modeling, and the adoption of appropriate development methodologies specifically the incremental model and iterative feedback cycles.

In addition, Database Management Systems (DBMS) contributed to the understanding of data

structuring, allowing implementation of storage solutions using MongoDB for tracking user profiles, ride history, SOS logs, and payment records. The Human-Computer Interaction (HCI) course was crucial in developing a clean and intuitive user interface, where usability, user testing, and visual feedback became priorities. Furthermore, concepts from Software Project Management helped plan tasks, divide responsibilities, and track progress. Altogether, this project represents a comprehensive application of course learning in a functional and interactive ride-sharing platform.

### **1.2.1 Software Requirements Engineering (SRE)**

The Software Requirements Specification (SRS) document for the SheRides project was developed by applying techniques and methodologies studied in the Software Requirements Engineering (SRE) course. It includes detailed functional requirements such as user authentication, ride booking, SOS alert execution, trusted contact sharing, driver verification, and admin monitoring and non-functional requirements like performance, usability, security, and system reliability. The process began with stakeholder analysis, identifying the needs and expectations of both passengers and development team members. From there, requirements were prioritized by balancing engaging safety features with the technical and resource constraints of the team. Techniques like MoSCoW (Must-have, Should-have, Could-have, Won't-have) were used to classify features based on their importance and feasibility.

Additionally, the creation of use cases, user stories, and flow diagrams was guided by structured requirement modeling practices, ensuring that all user interactions were clearly documented and aligned with safety objectives. This formal requirement process helped maintain clarity throughout the development cycle, minimized scope creep, and ensured that a well-planned and achievable version of the application was delivered. By following the principles of SRE, the application was ensured to be not only functional but also feasible and aligned with user expectations.

### **1.2.2 Software Design and Architecture**

The development of the SheRides project involved the design of a well-structured and modular system architecture, following layered design principles to ensure clarity, maintainability, and scalability. The architecture was broken down into key independent modules such as the Ride Booking System, SOS Alert Module, Trusted Contact System, Driver Verification Module, and Admin Dashboard. Each module was designed to operate autonomously while communicating with others through defined interfaces, which allowed the development team to work on components concurrently and reduced dependencies. This modularity also supports easier

debugging and future expansion of the application.

To promote efficient and reusable design, well-known software design patterns were incorporated. The Singleton Pattern was applied for system-wide managers, such as the Notification Manager and Session Manager, ensuring that only one instance of these controllers exists during application runtime. The Observer Pattern was used within the SOS Alert System to handle real-time notifications to admin and trusted contacts dynamically, allowing flexibility in implementing and triggering various alert mechanisms.

Comprehensive UML diagrams were also created, including class diagrams to represent the static structure of the system entities (e.g., User, Driver, Ride, SOSAlert, TrustedContact), and sequence diagrams to model real-time interactions such as ride booking, SOS triggering, and driver verification. These diagrams, documented in the Software Design Document (SDD), provided a clear blueprint for implementation and played a critical role in aligning development with design specifications. Overall, the use of layered architecture, design patterns, and visual modeling tools contributed significantly to the organization, clarity, and robustness of the project.

### **1.2.3 Database Systems**

In the SheRides project, MongoDB was chosen as the primary solution for structured data storage due to its lightweight footprint, fast performance, and seamless integration with Node.js, which made it highly suitable for a ride-sharing application. MongoDB provides a document-based database system that does not require complex table structures, making it ideal for managing persistent, local application data across sessions. The database plays a central role in tracking and storing a wide variety of application data, ensuring that user progress is preserved and accurately reflected throughout application usage.

MongoDB was used to structure essential components such as user profiles, driver documents, ride history, SOS logs, trusted contacts, and payment records. For example, the Users collection records attributes like userId, name, email, phone, gender, and authentication token. The Rides collection contains data about each booked ride, including pickup location, destination, fare, driverId, passengerId, status, and timestamps. Another important collection is SOSAlerts, which logs every emergency event, including the userId, location, timestamp, and status along with notification delivery records.

To ensure database consistency and ease of future maintenance, a well-structured data dictionary was created as part of the Software Design Document (SDD). This dictionary

formally defines all entities, their attributes, data types, and inter-entity relationships. Key entities include User, Driver, Ride, SOSAlert, TrustedContact, Payment, and Admin. Each entry in the dictionary aids developers in understanding how data flows and is stored within the application. Overall, the use of MongoDB not only helped organize data efficiently but also enabled smooth session management, robust application state management, and a solid foundation for scalability in future updates or expansions.

## **1.2.4 Human Computer Interaction**

In the development of SheRides, UI/UX principles were carefully applied to design a user interface that is not only visually appealing but also functional, accessible, and easy to navigate. Tools such as Figma and Adobe Illustrator were used to create mockups and wireframes of various application screens, including the login screen, passenger home screen, driver dashboard, ride booking interface, SOS alert panel, and admin portal. These tools allowed quick iteration, experimentation with layout options, and early feedback before implementation. The goal was to ensure a smooth and intuitive experience for users from the moment they launch the application.

Key design principles such as consistency, clarity, visual hierarchy, and feedback were prioritized. Buttons, icons, and navigation elements were placed logically to reduce cognitive load and enhance accessibility. Typography and color schemes were chosen to ensure good readability across different devices and resolutions. Clear visual indicators and animations were also introduced to guide users through actions like ride booking, SOS activation, and payment confirmation.

Accessibility and usability were further enhanced through iterative prototyping and user testing. Early feedback helped identify and resolve usability issues such as overcrowded menus or unclear icons. Interface elements were revised to simplify user flows and minimize the number of clicks needed for key actions. Additionally, a tutorial system was integrated that guides new users step-by-step, using on-screen tips and prompts.

Through careful planning, design, and refinement, the UI/UX process ensured that the application remained user-friendly, engaging, and aligned with the overall ride-sharing experience.

## **1.2.5 Mobile and Backend Development with Object Oriented Programming**

### **1.2.5.1 Encapsulation**

1. All core entities such as User, Driver, Ride, SOSAlert, and TrustedContact were

implemented as individual classes with private fields to protect internal data.

2. Attributes like `userId`, `rideStatus`, `sosTriggerTime`, `driverAvailability`, and `paymentAmount` were marked as private, and accessed only through controlled public methods.
3. For instance, the `Ride` class contains private `String rideStatus`; and uses methods like `updateStatus(String status)` or `getRideStatus()` to safely interact with this value.
4. Encapsulation allowed the application to maintain clean boundaries between systems, ensuring that ride data cannot be altered by any system other than the intended logic (for example ride booking or ride completion).
5. This helped prevent unintended side effects during application usage and supported better debugging and testing since data flow was restricted and predictable.

#### **1.2.5.2 Inheritance**

1. A general `User` base class was created to hold common attributes like name, email, phone, and `userId`.
2. Two specialized child classes `Passenger` and `Driver` inherited from `User`. These classes added unique behavior:
  - `Passenger` had ride history, trusted contacts list, and SOS preferences
  - `Driver` had vehicle details, document verification status, and earnings history
3. Similarly, a `Notification` base class could be introduced for common attributes (e.g., `notificationId`, `message`, `timestamp`), with derived classes like `SOSNotification`, `RideConfirmationNotification`, and `PaymentReceiptNotification` overriding behavior.
4. Inheritance simplified the structure, allowing generalized code to be reused while still supporting specialized application logic.

#### **1.2.5.3 Polymorphism**

1. The notification system relied on interface-based polymorphism to handle dynamic alert delivery during application usage.
2. All notification types implemented a shared interface `Notifiable` with a method like `sendAlert(User user, String message)`.
3. This allowed the application to treat all notifications uniformly while still triggering unique behaviors like sending SOS alerts to admin, sending ride confirmations to

passengers, or sending payment receipts to drivers.

4. At runtime, when a notification is triggered, the system simply calls `notifiable.sendAlert()` without needing to check which type of notification it is.
5. Polymorphism also allowed easy expansion: new notification types could be added later (e.g., `Promotional Notification`, `DriverBonusNotification`) without modifying existing logic.
6. This made the application's notification logic flexible and open for future updates or enhancements without reworking core mechanics.

#### **1.2.5.4 Abstraction**

1. Though not mentioned earlier, abstraction was also applied in designing system managers like `AuthManager`, `RideManager`, `SOSManager`, and `NotificationManager`.
2. Each manager exposed only the necessary public methods (e.g., `loginUser()`, `bookRide()`, `triggerSOS()`, `sendNotification()`) while hiding complex internal implementations.
3. This kept the codebase clean and focused, ensuring that developers interacted with simple, meaningful functions without needing to understand the internal workings of each system.

#### **1.2.5.5 Benefits and Real-World Application**

1. Using OOP helped divide complex application systems into smaller, logical components that were easier to build, test, and maintain.
2. The combination of encapsulation, inheritance, and polymorphism ensured that the code was not only reusable but also adaptable to changes throughout the project.
3. For example, changing how SOS alerts work only required updates in the `SOSAlert` class without affecting unrelated parts of the application.
4. This modularity allowed team members to work on different systems (UI, ride logic, safety features) in parallel without code conflicts.

#### **1.2.6 Software Quality Assurance and Testing**

Unit testing (for example, ride booking logic) and integration testing (SOS system + notification delivery) were conducted. Non-functional requirements including performance, reliability, and security were addressed in the SRS.

## 1.3 Project Background

In recent years, ride-sharing applications have grown significantly in popularity among users who value convenience, affordability, and accessibility. Titles like Uber, InDrive, and Yango have captured audiences by delivering simple, fast-paced, and engaging experiences rooted in on-demand transportation and real-time tracking. These applications utilize a minimalist interface and focus heavily on continuous, rapid actions such as requesting a ride, tracking the driver, and completing payments. While this model has proven effective in reaching casual users, it presents several limitations that restrict deeper user trust and long-term retention, particularly for female passengers in conservative societies.

After extensive observation and usage sessions with these popular applications, a recurring theme was identified: a lack of safety-focused design for women. While functional for general audiences, these applications often become problematic for female passengers due to harassment [13] incidents [3], lack of gender filters, and inadequate emergency response mechanisms. There is little room for women-specific safety features, real-time emergency alerts, or passive monitoring by family members. There are no substantial features that encourage passengers to plan safe rides, manage trusted contacts, or choose from a set of diverse safety options. The outcome of a ride largely depends on driver behavior and general platform policies rather than proactive safety measures.

These observations laid the foundation for this project: SheRides. The goal was to create an application that keeps the addictive on-demand ride-hailing model while introducing a much deeper layer of safety-focused features, suited for women passengers who prioritize security over speed. SheRides retains the visual and structural appeal of a ride-hailing application but completely reimagines the safety and monitoring mechanisms through the addition of SOS alerts, trusted contact circles, driver verification, and admin monitoring. This allows passengers to control not only the outcome of individual rides but also their broader safety, making each decision meaningful and impactful.

Unlike the applications that inspired it, SheRides does not reward just speed it rewards safety, preparation, foresight, and adaptability. Before requesting a ride, passengers must first verify that trusted contacts are added and SOS features are active. These safety tools are finite and must be configured wisely. Passengers can choose to share live location with multiple contacts, enable automatic SOS triggers, or manually activate alerts but not all settings may suit every

trip. This introduces a natural consideration in safety configuration, forcing passengers to think critically about which features to enable and when to take precautions.

The SOS system is where SheRides truly diverges from traditional ride-hailing applications. Rather than relying on generic emergency buttons or third-party helplines, battles are conducted via a dedicated SOS module, each with a unique function immediate alert, location sharing, admin notification, and trusted contact alert. The passenger activates SOS and must confirm during an emergency, balancing immediate response with accurate reporting. This mechanic turns every emergency into a structured response that challenges the passenger to act quickly, anticipate admin behavior, and respond appropriately. Unlike random outcomes or delayed responses, the effectiveness of SOS alerts is heavily influenced by passenger choices and system configuration, removing uncertainty and emphasizing preparedness.

Moreover, SheRides introduces a trusted contact system a feature largely absent in most ride-hailing applications. Before or during a ride, passengers can share live location and trip details with family members. These contact shares have a variable reliability rate based on the passenger's configuration and offer real-time updates about driver location, route deviation, or estimated arrival time. This mechanic simulates real-world family monitoring and adds another layer of safety: should a passenger share location with all contacts, or only with a primary contact? The answer depends on the passenger's preferences and family availability.

Another key feature is the progressive verification model. In applications like Uber or InDrive, driver verification is generally uniform and based on basic document submission. In SheRides, as drivers complete more rides, their verification level increases. Basic verification gives way to enhanced verification, gold verification levels that require superior documentation and training. This gradual increase in verification standards ensures that passengers are consistently matched with trusted drivers and must adapt their expectations to meet new safety standards. It prevents the platform from becoming unreliable and gives passengers a tangible sense of security and trust.

The motivation for building this application also came from a desire to merge technology with social impact and women's empowerment. By requiring passengers to make safety-conscious choices, manage a constrained set of safety tools, and anticipate driver behavior, SheRides becomes more than just a transportation app it becomes a platform for developing awareness, planning, and decision-making skills. Each aspect of the application encourages passengers to evaluate trade-offs, prioritize objectives, and learn from past ride experiences. This kind of cognitive engagement is often missing in standard ride-hailing applications, which prioritize



convenience over awareness.

From a design and development standpoint, the application also offered a unique opportunity to integrate a wide range of academic concepts. The use of object-oriented programming principles helped structure the application's systems in a modular, scalable manner. Application entities like User, Passenger, Driver, Ride, SOSAlert, and TrustedContact were all implemented using encapsulation, inheritance, and polymorphism. This not only made the codebase more manageable but also allowed quick iteration, independent feature testing, and high software quality maintenance.

The Incremental Development Model further supported the project vision. Unlike the rigid Waterfall Model, incremental development allowed the building of playable versions of the application early on. Ride booking mechanics could be tested, SOS logic adjusted, or driver verification balanced in real-time, leading to faster feedback and better design decisions. User feedback, both formal and informal, played a major role in refining application features. For example, early testers were confused by how SOS alerts were triggered; the team responded by improving the UI and adding visual cues that clarified SOS interactions. These improvements not only enhanced the application but demonstrated the importance of user-centric development, especially in safety-critical applications where experience and clarity are crucial.

Lastly, the design of SheRides also serves as a case study in developing applications that are both technically feasible and socially impactful. The goal was never to compete with global giants or create a feature-heavy product; instead, focus remained on clean design, thoughtful safety features, and meaningful interactivity. By deliberately keeping the application women-only and single-platform, unnecessary complexity was avoided and focus remained on delivering a polished, secure core experience. The application is lightweight, playable on mid-range Android devices, and uses efficient data storage techniques such as MongoDB for local application state management. This accessibility ensures that more users can enjoy the application without needing advanced hardware.

In conclusion, the creation of SheRides stemmed from a clear gap in the current landscape of ride-hailing applications. While popular applications emphasize speed and convenience, a project was envisioned that embraced safety depth, deliberate planning, and immersive security features. SheRides takes inspiration from its predecessors but transforms the formula into something safer, smarter, and more rewarding. By combining SOS alerts, trusted contact monitoring, driver verification, and admin oversight into a cohesive operational loop, an application was created that not only serves but protects women. This project is not only a

product of technical and creative efforts it reflects the academic journey, understanding of user psychology, and belief in the power of interactive safety systems.

## **1.4 Literature Review**

In digital ride-sharing applications, especially within mobile markets, titles like Uber, InDrive, and Yango have carved a significant niche by offering fast, intuitive ride-hailing experiences that focus heavily on on-demand transportation and minimal input mechanics. These applications attract a large user base due to their simplicity, accessibility, and convenience. However, while these applications are successful in delivering immediate satisfaction and accessibility, they tend to lack long-term trust and safety due to limited women-specific security features and an overemphasis on speed-based operations.

**Yango:** Yango [12] is an international ride-hailing application that recently entered the Pakistani market. The application offers affordable rides but has received complaints about harassment, overcharging, and rude driver behavior. Many women passengers reported discomfort due to the absence of gender filters. In response to criticism, Yango partnered with the Punjab Safe City Authority to connect its application with emergency helplines. However, incidents still highlight weaknesses in its safety ecosystem. The application prioritizes affordability and speed over proactive safety measures for women passengers.

**InDrive:** InDrive [11] is one of the most popular ride-hailing applications in Pakistan. The application allows passengers to bid for rides and negotiate fares in real-time. However, the application has faced criticism due to safety and trust issues. Cases have been reported where drivers misbehaved with female passengers, including harassment and inappropriate conduct during rides. There have also been scams where drivers accepted deliveries and then disappeared, leaving passengers at a loss. These incidents show that while InDrive is convenient, it lacks strong monitoring and safety measures. The application does not provide women-only ride options or integrated emergency response systems.

**Uber:** Uber introduced safety features such as Share My Trip and emergency assistance. These features remain add-ons rather than core design principles. The mixed-gender model continues to concern female passengers in conservative societies. Uber's driver verification process, while present, has not eliminated incidents of harassment. The application lacks dedicated features for women passengers including SOS alerts integrated with local authorities, trusted contact circles for passive monitoring, and women-only driver options.

These design choices align with what transportation and HCI researchers call "efficiency-first design." According to research in human-computer interaction and safety-critical system design, efficiency-first applications focus on lowering the entry barrier by minimizing the complexity of input and maximizing speed of service delivery. While this approach is suitable for short-term user engagement, it does not foster deep trust, emotional security, or sustained user loyalty, especially among female passengers in conservative societies like Pakistan.

A major shortcoming of applications like Uber, InDrive, and Yango is the absence of women-specific safety features. Female passengers are often dropped into rides with no verification of driver background beyond basic document checks. There is no guided safety progression or layered introduction to emergency features. This lack of safety-focused design can cause anxiety for female users and limits user retention among women. In SheRide, a deliberate incorporation of safety features has been implemented, supported by SOS alerts, trusted contact circles, and admin monitoring. This aligns with modern safety system design principles that emphasize "proactive protection" as a key usability factor, especially in women-centric applications.

Another safety component absent in reference applications is passive monitoring. In Uber, InDrive, and Yango, ride tracking is available but must be manually shared. There is no automatic location sharing with trusted contacts. SheRides addresses this flaw through the trusted contact circle system, where live location and trip details are automatically shared with pre-added contacts during every ride. This feature adds a layer of passive security rarely seen in the ride-hailing genre.

The integration of an SOS alert system in SheRides is also based on observations from safety-critical applications such as emergency response systems and personal safety applications like Be Safe and Life360. While Uber and InDrive allow passengers to contact emergency services, they provide no integrated SOS mechanism that simultaneously alerts admin, trusted contacts, and local authorities. SheRides SOS system introduces a multi-channel alert mechanism based on passenger action that mimics real-world emergency response operations. This not only increases passenger confidence but also fosters proactive safety behavior. Passengers must choose when to trigger SOS based on perceived threat levels.

From an academic standpoint, the feature taxonomy of SheRides addresses several layers of passenger safety absent in its predecessors. The inclusion of macro-safety features (admin monitoring, driver verification) and micro-safety layers (SOS alerts, trusted contact sharing) aligns with principles in safety-critical system design and risk management. The passenger is

not merely reacting to unsafe situations; they are building a long-term safety plan, adjusting to new information, and making meaningful choices that directly influence their security during rides.

Furthermore, many ride-hailing applications lack personalization of safety features, resulting in one-size-fits-all protection. SheRides includes a trusted contact system that allows passengers to store emergency contacts and configure notification settings. This not only supports passenger expression but also introduces a personalized safety layer where passengers refine their safety network over time — an element notably missing in Uber or InDrive, where no such memory or customization exists.

In terms of user interface and accessibility, applications like Uber rely heavily on visual minimalism to appeal to a broader audience. However, this often leads to lack of clarity in emergency scenarios, such as where to find the emergency button or how to share location. SheRides takes a different approach by employing a clean UI design with feedback-rich interactions. Elements such as SOS button placement, trusted contact status indicators, ride tracking visualization, and alert confirmation dialogs help guide the user and reduce cognitive load during emergencies, a principle rooted in Human-Computer Interaction (HCI).

Lastly, many ride-hailing applications implement surge pricing or time-based fare increases that can discourage usage during peak hours or emergencies. This approach frustrates users, especially women who may need rides during unsafe hours. SheRides was built with a transparent, safety-first pricing model in mind, where fare is calculated based on distance and time without surge multipliers during emergencies. This decision aligns with the goal of building an accessible, fair, and trustworthy platform, particularly suited for academic demonstration and real-world application of software engineering principles for social impact.

In summary, the literature and market review clearly identify a void between standard ride-hailing applications and women-centric safety-focused transportation platforms. SheRides is designed to bridge this gap by taking successful elements from both sides — such as operational simplicity and fast accessibility from Uber, and proactive safety features and emergency response from personal safety applications — and blending them into a cohesive, trustworthy experience. By introducing systems for SOS alerts, trusted contact circles, driver verification, admin monitoring, and recurring rides, SheRides offers not just another ride-hailing application, but a safety-focused mobility ecosystem that encourages long-term trust, passenger empowerment. The platform improves upon the shortcomings of existing applications and brings much-needed safety depth to a genre that often prioritizes speed over substance.

## **1.5 Analysis from Literature Review (in the context of this project)**

The analysis of applications like Uber, InDrive, and Yango offers crucial insights into both the success and limitations of fast-paced, convenience-focused ride-hailing platforms. These applications have been praised for their simplicity and quick service delivery loops, appealing to a wide casual user audience. However, a deeper study reveals that such applications often operate on narrow safety frameworks that emphasize service speed over passenger security, ultimately limiting their long-term trust and safety for women passengers. This project, SheRides, directly responds to these design gaps by offering a ride-sharing experience that is secure, proactive, and passenger-centric, aligning more closely with modern female passenger expectations for safety, personalization, and emergency readiness.

### **Safety vs. Speed: The Security Gap**

Uber, InDrive, and Yango use real-time ride assignment systems without incorporating layers of safety verification or proactive emergency planning. Once a passenger requests a ride, the system does not require or even allow intervention beyond basic ride cancellation. This type of operation, though easy to use, leaves little room for safety decision-making beyond trusting the assigned driver. As a result, passenger choices become limited and reactive, relying more on luck and platform reputation than on analysis, prediction, or proactive safety measures.

SheRides, in contrast, introduces a multi-layered safety model governed by SOS alerts, trusted contact circles, and admin monitoring. Each ride requires evaluation of current safety settings, available emergency contacts, and potential risk factors. Passengers must not only plan their immediate ride but also anticipate emergency possibilities, such as the need to trigger SOS or share location with trusted contacts. This model promotes proactive safety behavior, a concept widely regarded as essential for meaningful security in women-centric transportation platforms. By integrating features like SOS alerts, trusted contact sharing, and driver verification, passengers engage with the platform not just through ride booking but through safety awareness and foresight — elements missing in the reference applications.

### **Trust and Long-Term User Retention**

One of the most notable weaknesses of standard ride-hailing applications is their limited trust-building for women passengers. With only basic driver verification and no women-specific safety features, female users eventually experience diminishing trust in the platform after repeated exposure to safety incidents or harassment reports. Uber, for example, provides no mechanism for women-only ride options, trusted contact integration, or SOS escalation beyond

generic emergency buttons. The passenger may complete many rides, but the safety experience remains the same.

SheRides addresses this issue through several innovative systems:

**Progressive Driver Verification:** As drivers complete more rides, their verification level increases from basic to enhanced to gold, requiring superior documentation and training

**SOS Alert Evolution:** Passengers unlock more comprehensive emergency response features over time, encouraging familiarization with safety tools

**Trusted Contact Management:** Custom safety networks can be saved and reused, enabling passengers to develop and refine long-term safety approaches

These systems ensure that no two rides feel the same from a safety perspective and that the passenger continues to engage with the platform's safety features in new and meaningful ways.

### **User Engagement and Cognitive Load During Emergencies**

From a cognitive psychology standpoint, safety-critical systems that involve emergency preparedness, threat recognition, and response planning tend to result in deeper user confidence and higher platform loyalty. According to safety system design literature, the presence of "proactive safety choices" is directly correlated with user trust and satisfaction. The minimalistic safety design of Uber and InDrive lacks such meaningful safety branching paths. Once a ride starts, the safety options are often limited and rarely change based on in-ride events.

SheRides intentionally introduces emergency preparedness features to increase passenger confidence:

- The SOS System introduces multi-channel alerting, challenging passengers to make informed decisions during emergencies
- Trusted Contact Constraints limit the number of contacts available, requiring prioritization of who should receive ride update
- Real-time Tracking introduces continuous location sharing, which increases the number of passive safety measures and keeps the passenger's family informed

### **Planning and Decision-Making: Core Design Philosophies**

Speed-focused ride-hailing applications appeal primarily to convenience-oriented users — those who value speed, low cost, and simplicity. However, they do not cater to safety-conscious users — women who value security, monitoring, and emergency preparedness. According to

user classification in safety-critical system design, such users fall under "security-seekers" and "risk-aware users," both of whom demand systems that reward safety-conscious behavior and forethought over service speed.

SheRides was designed with these users in mind. The platform introduces multiple layers of safety decision-making:

- **Pre-Ride Preparation:** Passengers choose trusted contacts, verify SOS settings, and review driver details before ride start
- **During-Ride Monitoring:** Passengers monitor live location, have SOS button accessible, and receive automatic updates to trusted contacts
- **Post-Ride Review:** After ride completion, passengers review safety performance, provide feedback on driver behavior, and update trusted contact lists

This cyclical flow of safety planning, monitoring, and review mirrors more complex safety-critical systems and offers a more satisfying experience for security-conscious passengers.

- **Balancing Convenience and Safety**

It is important to note that SheRides does not aim to completely replicate the complexity of enterprise safety systems or emergency response platforms. Instead, it seeks a middle ground — a space that blends the approachability of standard ride-hailing applications with the safety depth of dedicated personal security applications. This required adoption of a modular system design, allowing passengers to interact with simplified UIs while operating within a deeper safety rule set.

Examples include:

- One-tap SOS, but with confirmation and multi-channel alert delivery
- Simplified trusted contact addition, but with automatic location sharing during all rides
- Straightforward driver verification display, but with multiple verification levels affecting driver privileges

This kind of design aligns with modern user-centered safety design principles, ensuring that safety complexity is layered and only revealed when needed, thus preventing user overwhelm during normal operations while maintaining full functionality during emergencies.

### **Feedback, Monitoring, and Safety Evolution**

Applications like Uber and InDrive do not significantly respond to passenger safety behavior. There is no mechanism for the platform to learn or adapt based on the user's safety preferences or incident history. In contrast, SheRides integrates systems that change dynamically as passengers use the platform:

- SOS response times are tracked and improved based on historical data
- Driver verification levels increase based on completed ride history and passenger feedback
- Admin monitoring provides real-time oversight of all active rides, enabling immediate intervention when needed

This evolutionary approach ensures that passengers are protected at every level of platform usage and that their safety strategies can evolve alongside the platform.

### **Alignment with Modern Female Passenger Expectations**

Today's female passengers, especially in countries like Pakistan, are more aware of safety risks and seek transportation experiences that offer more than just convenience. According to research on women's mobility in developing countries, female passengers rank safety, driver verification, and emergency response among their top expectations in ride-hailing platforms. SheRides reflects these preferences by offering

- Meaningful passenger control over multiple safety features
- Real-time emergency response through SOS alert
- Passive monitoring through trusted contact sharing
- Transparent driver verification with multiple levels

Admin oversight of all rides

This results in a platform that not only provides transportation but also respects the safety concerns and preferences of its female passengers.

## **1.6 Tools and Technologies**

- Development: Flutter [6] (mobile app), Node.js + Express (backend), Firebase Authentication, MongoDB (database)
- Design: Figma, Adobe Illustrator (UI/UX)
- Documentation: Draw.io (diagrams), MS Word, PowerPoint



## **1.7 Methodology and Software Lifecycle for This Project**

The development of SheRides followed the Incremental Model, a highly iterative and feedback-driven software development methodology. This model was selected to support the evolving nature of safety-critical features, changing requirements based on user feedback, and the need for constant testing and feature refinement. In contrast to linear approaches like the Waterfall Model, the Incremental Model allowed early development start, critical feature testing at a foundational level, and dynamic response to issues, suggestions, and design evolution throughout the project

### **Choosing the Right Methodology for Safety-Critical Ride-Sharing Development**

Ride-sharing application development, especially for women-centric safety platforms, is inherently different from conventional application development. It involves more abstract goals, such as creating "trust," achieving safety compliance, and delivering intuitive emergency response experiences. These goals are not always fully clear at the start of the project and often change based on user interaction and safety incident analysis. For these reasons, a model was needed that encouraged flexibility, experimentation, and rapid feedback. The Incremental Model allowed meeting these criteria by focusing on building functional versions of the system early, demonstrating features in action, and refining them over multiple cycles.

### **1.7.1 Methodology**

The development of SheRides followed the Incremental Model, which is particularly well-suited for ride-sharing application development where user experience, interface flow, and safety features require frequent refinement. This model enabled the team to quickly create simplified working versions of core features such as user authentication, ride booking system, SOS alert mechanism, and admin dashboard. These early-stage increments allowed the team and stakeholders to interact with tangible versions of the application rather than relying on static designs or lengthy documentation. By visualizing mechanics like ride tracking and SOS execution early on, design flaws, usability issues, and technical constraints were identified.

Each incremental cycle focused on implementing a specific feature or improving an existing one, such as adjusting SOS response times, enhancing the trusted contact system's functionality, or refining how rides are calculated and tracked. As the project progressed, feedback gathered from these increments informed necessary adjustments to system balance, user interface layout, and logical flow. Because the Incremental Model encourages

experimentation, it gave the team the flexibility to explore multiple approaches before committing to the final design. This was especially valuable for safety-critical systems like SOS logic, where alert effects had to be tested and adjusted multiple times. Ultimately, this model reduced development risks, improved the quality of core features, and ensured a more secure user experience by the time the final build was reached.

Alongside incremental development, an Iterative Feedback Methodology was integrated to ensure continuous improvement throughout the project life-cycle. After completing each increment or module whether it was the ride booking system, SOS interface, or driver verification mechanism structured review sessions were conducted. These sessions included peer testing within the team as well as informal feedback from target users. Testers were asked to evaluate usability, understandability, system logic, and how effectively each feature supported passenger safety and decision-making.

Feedback collected from each cycle was carefully documented, prioritized, and incorporated into the next development sprint. For example, initial testing revealed that users were unclear about SOS alert confirmation and its impact during emergencies. Based on this, the SOS design was improved and better visual cues and explanations were added. Similarly, the early driver verification system was deemed too simplistic, leading to the introduction of document upload status indicators that offered a clearer sense of verification progress. This cycle of testing, reviewing, and refining continued throughout development.

The iterative approach not only helped adapt to user expectations but also made the application more polished with each pass. It allowed the team to stay agile, addressing bugs, usability issues, and system imbalances promptly without waiting for final deployment. Additionally, this method promoted better collaboration among team members, as everyone could see and respond to progress in real time. In the context of a safety-critical application like **SheRides**, where user security is paramount, the Iterative Feedback Methodology ensured the final product was not only functional but also reliable and intuitive.

### **1.7.2 Understanding the Incremental Model in Practice**

The Incremental Model starts with preliminary requirement gathering and leads quickly into building a simple version of the product. This version called an increment demonstrates key features and interactions without requiring full implementation. After the increment is evaluated by stakeholders or users, the team incorporates feedback and repeats the cycle. With each iteration, the system becomes more complete, more polished, and more aligned with expectations.

For SheRides, this meant that instead of completing all design and documentation before building the application, early functional modules were developed including user authentication, ride booking interface, SOS alert system, trusted contact management, and driver verification. These modules were tested immediately by both developers and peer users to gather insights into usability, safety logic, and user confidence. Feedback collected during each incremental cycle guided decisions on interface changes, safety feature balancing, and system enhancements.

### **1.7.3 Planning and Building Initial Increments**

The first stages of development focused on capturing the core concept of the application. These included elements like user registration, ride booking, driver assignment, and basic ride tracking. The goal was not to build a complete version but to validate whether these features worked well together and whether users understood how to interact with them safely.

Early increments included:

- A basic authentication system using Firebase for secure login
- A simple ride booking UI where passengers could enter pickup and drop-off locations
- A real-time tracking system showing driver location on map

These systems were minimalistic and lacked advanced features like SOS alerts, trusted contacts, and admin monitoring — but they allowed testers to experience the structure of the application and provide useful suggestions. This rapid validation approach ensured that the core loop of ride booking was strong before investing time in more detailed safety features.

### **1.7.4 Feedback and Iterative Improvement**

User feedback was an integral part of the process. After each increment build, informal testing sessions were conducted within the team and with external users. The goal of these sessions was to identify areas where the application felt confusing, unsafe, or unintuitive. Based on this feedback, significant improvements were made across all modules.

For example, testers initially struggled to understand how SOS alerts worked and whether alerts were actually sent. To address this, the SOS button was redesigned to include confirmation dialogs, vibration feedback, and visual indicators that alerts had been received by admin and trusted contacts. The trusted contact system also underwent several changes after users found it difficult to understand how contacts were added or how location sharing worked. The visual

feedback was refined and status indicators (pending, approved, rejected) were added to make this feature more transparent and reliable.

Through this ongoing process of testing, reviewing, and refining, the system continually moved closer to the final product. Each cycle of development added layers of polish and sophistication while preserving the underlying structure validated in earlier iterations.

### **1.7.5 Balancing Design Flexibility and Technical Constraints**

Another major advantage of the Incremental Model was the ability to introduce new ideas mid-development without destabilizing the entire project. For example, the concept of "Recurring Rides" as a feature for daily commuters was introduced after the initial system for instant ride booking was already built. Instead of reworking the entire application, the existing ride class was extended and new attributes for schedule, frequency, and automated booking were added. This change was made possible by the modular and testable nature of incremental development.

Similarly, the idea of "Mobile Mechanic Assistance" was proposed halfway through development. Since earlier increments had already clarified how rides were tracked and how SOS alerts worked, it was relatively easy to build on that foundation and allow users to request roadside mechanic help during breakdowns.

These additions were tested in mini-increments and then integrated into the full system once validated. This modular flexibility allowed innovation without creating technical debt or interrupting other areas of the project.

### **1.7.6 Integration with Core Systems**

Each system module including ride booking, SOS alerts, trusted contacts, driver verification, admin monitoring, and payment processing was treated as a self-contained module. During each incremental cycle, focus remained on improving one or two systems at a time. This focus allowed isolation of bugs, tracking of improvements, and avoidance of conflicts in code or design. For example, when working on SOS logic, a temporary test environment was developed where SOS triggers could be tested outside of full rides. This sandbox approach simplified testing and made debugging faster and more accurate.

In later stages of the project, once core systems had been individually tested, integration into a full operational loop began. The transition from increment to final product was made easier because each part functioned correctly on its own. Once combined, the systems worked together smoothly, with only minor adjustments needed to ensure consistent flow and balanced interactions.

### **1.7.7 Why Not Waterfall or Agile?**

It is worth noting why other methodologies were not suitable for this project. The Waterfall Model, while thorough, is highly sequential and does not permit changes once a phase is completed. In a safety-critical application like SheRides, where features had to be tested and reworked frequently based on user feedback and safety requirements, this rigidity would have led to excessive rework and wasted time.

On the other hand, Agile methodologies, while flexible, require structured sprints, dedicated product owners, and formal backlogs. Given the academic setting, small team size of three members, and variable workload from other courses, full Agile practices could not be implemented. However, Agile principles like iterative cycles and continuous feedback were incorporated into the incremental model, allowing maintenance of progress and responsiveness throughout development.

### **1.7.8 Supporting Usability and User-Centric Design**

The Incremental Model also supported commitment to user-centric design, an important principle in both safety-critical application development and software engineering. By building testable versions early, changes could be made not just based on theory or design intentions, but on actual user experience. This approach helped ensure that the UI was intuitive, the ride booking flow was clear, and that users could understand the application's safety features without needing extensive tutorials.

This process also revealed areas where users struggled. For example, the SOS button initially lacked clear confirmation that an alert had been sent. Adding a confirmation message and vibration feedback made this feature far more reliable and easier to understand. The trusted contact system initially had no status indicators for whether contacts had received location updates. Adding status indicators (shared, pending, failed) made this system far more transparent and easier to trust. These types of refinements would not have been identified as easily without putting real prototypes in front of users.

### **1.7.9 Managing Testing Within the Incremental Model**

One of the most important strengths of the Incremental Model was its built-in alignment with testing. Unlike linear models where testing occurs only at the end, incremental development allowed testing from the very first functional build. As soon as a feature like user registration or ride booking was implemented, unit-level manual tests were conducted to verify expected behavior

For example, when implementing the SOS system, early tests checked for:

- Whether the passenger could trigger SOS during an active ride
- Whether SOS alerts reached admin and trusted contacts within 5 seconds
- Whether live location was included in the alert
- Whether the system retried sending if internet was unavailable.
- Whether false SOS could be cancelled within 5 seconds.

Manual exploratory testing was also applied during each iteration. This meant using the application in its current state and trying to uncover unexpected behaviors or bugs. In one case, it was found that certain sequences allowed SOS to be triggered even when no ride was active. This prompted revision of the SOS validation logic to check for active ride status before sending alerts

As features became more complex, integration testing became necessary. For instance, once the ride booking system, SOS module, and trusted contact system were integrated, testing interactions across modules began, such as booking a ride, triggering SOS, and verifying that trusted contacts received the alert with live location. Testing this loop required both functional tests and careful observation of notification delivery, location updates, and UI behavior.

In later phases, automated tests were written using Flutter Test Framework to validate basic operations like checking for correct screen navigation after login, ensuring SOS button appears only during active rides, and verifying that ride history updates after ride completion. These lightweight automated tests helped maintain consistency as new features were added.

#### **1.7.10 Handling Feedback and Iteration Cycles**

Throughout the development process, feedback played a central role, guiding nearly every design decision. With each increment, feedback was collected in several ways:

- Internal reviews: After completing a feature, team members tested and reviewed each other's modules
- Peer testing: Fellow students were invited to use early builds and describe their experience
- Supervisor input: Guidance was received during formal review sessions and milestone demos

This feedback was compiled into iteration plans, where the most pressing changes were prioritized for the next cycle. Often, what was believed to be complete had to be adjusted based on user expectations.

An illustrative example is the trusted contact system. The first version simply stored contact numbers but did not automatically share ride location. Users found this feature less useful than expected. In response, automatic location sharing during active rides was added, along with status indicators showing whether each contact had received the update.

Another case was the driver verification system. Initially, document upload had no status indicator. Users were confused about whether documents had been submitted successfully or were still being reviewed. Based on that input, status indicators (pending, approved, rejected) and notification messages were implemented, along with email alerts when verification status changed.

Each of these refinements occurred because the Incremental Model encouraged listening, iterating, and rebuilding as necessary without waiting for final phases or long development cycles.

### **1.7.11 Coordination and Team Collaboration**

Using the Incremental Model also helped the team work in parallel with minimal dependency conflicts. Because development and testing of features was done independently before integration, each member could work on a specific module without interfering with others. For example:

One member handled the mobile app, SOS module, and real-time tracking

Another built and refined the admin dashboard, incident management, and driver verification tools

The third focused on UI/UX design, mechanic assistance module, and mobile app contribution

To coordinate work, Git was used as version control. Each member committed changes to separate branches and merged only when features were tested and stable. This allowed safe experimentation and rollback if something went wrong.

Trello and Google Docs were used for task management and progress tracking. Each incremental cycle had a clear set of goals, such as "Implement SOS Alerts," "Test Driver Verification," "Add Recurring Rides UI," or "Complete Admin Dashboard." Tasks were assigned at the beginning of each week, and progress was discussed in regular team sync-ups

This structured, modular approach made the team highly productive and allowed consistent progress even during academic workload peaks such as midterms and quizzes

### **1.7.12 Tools and Technologies Supporting the Lifecycle**

To execute this methodology efficiently, tools were selected that aligned with fast prototyping, iterative development, and lightweight testing

Flutter: Served as the mobile application development platform, with a rapid hot-reload feature and support for component-based development

Dart Language: Allowed clean, object-oriented design using inheritance, interfaces, and encapsulation, well suited for modular architecture

Figma: Used during early design stages to sketch UI layouts and get visual feedback before implementing in Flutter

Firebase: Integrated for authentication, real-time database, and push notifications

MongoDB: Used for storing persistent data like user profiles, ride history, SOS logs, and payment records

Draw.io: Used for creating class diagrams, sequence diagrams, and flowcharts to plan logic before building increments

Postman: Used for API testing during backend development.

Flutter Test Framework: Used to run automated widget tests and unit tests for basic validation of UI components and business logic

By using these tools, a fast feedback cycle was supported, manual errors were reduced, and traceability across builds and iterations was maintained, which is essential when following an incremental development model for a safety-critical ride-sharing application.

## **1.8 Rationale behind Selected Methodology**

### **1.8.1 Object-Oriented Programming (OOP) Approach**

Modularity and Reusability: System components such as Passenger, Driver, Ride, SOS Alert, and Trusted Contact were modeled as objects, enabling code reuse across different modules

Scalability: OOP made it easier to add new features such as mechanic assistance and recurring rides without rewriting core logic

Encapsulation: Critical for managing system state including ride status, user authentication,



and SOS alerts securely, ensuring that data cannot be altered by unintended systems

### **1.8.2 Incremental Methodology (Iterative Development)**

**Early Feedback:** Rapid increments allowed testing of core mechanics such as ride booking and SOS alerts with stakeholders before full implementation, ensuring that safety features met user expectations

**Risk Mitigation:** Identified design flaws early such as missing SOS confirmation and lack of trusted contact status indicators, and iteratively refined them before they became major issues

**Agile Alignment:** Short increments matched academic timelines, ensuring steady progress from authentication to ride booking to SOS to trusted contacts to admin panel

### **1.8.3 Structured Design (Modular Architecture)**

**Separation of Concerns:** The system was divided into independent modules including Ride Booking, SOS Alerts, Trusted Contacts, Driver Verification, Admin Monitoring, and Payment Processing for easier debugging and updates

**Team Collaboration:** Clear modules allowed parallel development where one member worked on mobile UI while another implemented backend APIs and a third focused on admin dashboard

### **1.8.4 Choice of SDLC (Incremental + Iterative)**

**Why Not Waterfall?** Ride-sharing application development, especially for women-centric safety platforms, required flexibility because requirements evolved during testing, such as adjusting SOS response times, adding trusted contact features, and enhancing driver verification processes

**Why Not Pure Agile?** Academic constraints including fixed deadlines, small team size of three members, and variable workload from other courses necessitated a hybrid approach with planned milestones from SRS to SDD to final implementation

## **Chapter 2**

### **Problem Statement**

## **2. Problem Definition**

This chapter discusses the precise problem to be solved by the SheRides project. The main issue is the lack of a safe, women-only ride-sharing platform in Pakistan. Currently, women face serious difficulties when using existing ride-hailing applications due to safety concerns, harassment incidents, and lack of gender-specific features. Traditional transport options including public buses, rickshaws, and taxis remain overcrowded and unsafe. This fragmented and unsafe transportation system creates barriers to women's education, employment, and social participation, compromises their safety, and limits their independence.

Women in Pakistan face numerous challenges that affect both their mobility and quality of life. The absence of a women-centric transportation platform results in limited travel options, fear of harassment, inefficient trip planning, and gaps in emergency response. Many women continue to rely on male family members for transportation or use unsafe public transport, which often leads to uncomfortable experiences, safety incidents, and difficulty in maintaining independence. These challenges are particularly pronounced in urban areas where women travel for work and education, and in semi-urban regions where transport infrastructure is limited.

In addition, the growing female workforce and increasing number of women pursuing higher education have intensified the need for a safe transportation system that can handle female passengers' security requirements effectively. Women frequently have to depend on male relatives for rides, limiting their autonomy and increasing their dependence on others. Existing ride-hailing applications fail to provide verified female drivers, real-time emergency alerts, or passive monitoring features, which negatively impacts women's trust in these platforms and leads to reduced usage.

Furthermore, the lack of dedicated safety features in existing applications contributes to harassment incidents, misbehavior by drivers, and difficulties in reporting complaints. There is also a lack of secure mechanisms for sharing live location with trusted contacts during rides, which compromises passenger safety and peace of mind. The absence of a women-only transportation system prevents female passengers from traveling confidently, monitoring ride safety effectively, or accessing emergency support when needed.

### **2.1 Problem Statement**

The transportation system in Pakistan faces significant challenges for women due to the absence of women-only ride-sharing platforms, lack of verified female drivers, inadequate

safety features, and inconsistent emergency response mechanisms. Female passengers often struggle to find safe and reliable transportation, leading to limited mobility, fear of harassment, and reduced participation in work and education. Existing ride-hailing applications lack dedicated safety tools including SOS alerts, trusted contact sharing, and real-time monitoring. Drivers face difficulties in being verified as trustworthy female drivers. Administrators lack a unified platform to verify drivers, monitor rides, and handle complaints securely.

The SheRides project addresses these issues by providing a women-only [14], safety-focused ride-sharing platform. It enables secure ride booking with verified female drivers, SOS alerts with multi-channel notification, trusted contact circle for passive monitoring during rides, recurring rides for daily commuters, and mobile mechanic assistance for roadside emergencies. By integrating passengers, drivers, and administrators into a single safety-centric system, SheRides aims to improve women's mobility [15], reduce safety concerns, enhance passenger confidence, and deliver a reliable, secure transportation experience across Pakistan.

## **2.2 Deliverables and Development Requirements**

The SheRides project is designed to create a comprehensive women-only ride-sharing platform. This section outlines the main deliverables of the project and the technical and functional requirements necessary for its successful development and deployment. Deliverables define the expected outputs of the project, while development requirements specify the tools, technologies, and system capabilities needed to achieve these outcomes.

### **2.2.1 Deliverables**

The primary deliverables of the SheRides project include:

#### **2.2.1.1 Mobile Application for Passengers**

A Flutter-based mobile application that enables female passengers to book instant rides, schedule recurring rides, track drivers in real-time, view fares before confirming, add trusted contacts, trigger SOS alerts during emergencies, share live location with family members, and manage their profiles. The mobile app provides a clean, intuitive interface to simplify ride booking and improve passenger safety.

#### **2.2.1.2 Mobile Application for Drivers**

A Flutter-based mobile application designed for verified female drivers to accept or reject ride requests, start and end rides, view earnings, manage availability status, upload verification documents, and update profile information. The driver app ensures that only approved female drivers can provide transportation services on the platform.

### **2.2.1.3 Admin Web Panel**

A MERN-based administrative panel that oversees the entire system. The admin panel includes functionalities such as driver verification, document review, live ride monitoring, SOS alert handling, complaint management, user management, and system analytics.

### **2.2.1.4 Centralized Cloud Database**

A secure MongoDB database to store user profiles, driver documents, ride records, SOS logs, trusted contacts, payment history, and other ride-sharing related information. The database supports scalability, secure access, and efficient retrieval of data.

### **2.2.1.5 Backend REST API**

A Node.js with Express based backend API that handles all business logic including ride assignment, driver matching, fare calculation, notification delivery, and integration with Firebase Authentication and Google Maps API.

## **2.2.2 Development Requirements**

The development requirements define the technical needs for building the SheRides system.

### **2.2.2.1 Technical Requirements**

The technical requirements define the tools and technologies needed for SheRides. Flutter 3.x is used for cross-platform mobile development for both Android and iOS with a single codebase. Node.js with Express handles server operations and RESTful API development. Firebase Authentication manages secure login and identity verification. MongoDB stores user and ride data securely. Google Maps API provides location tracking and routing. Firebase Cloud Messaging delivers push notifications for SOS alerts and ride updates

- **Mobile Frontend Development:** Flutter 3.x with Dart for cross-platform mobile application development
- **Backend Development:** Node.js with Express for server-side operations, RESTful API development, and handling business logic
- **Authentication:** Firebase Authentication for secure user login and registration
- **Database:** MongoDB for secure storage of user profiles, ride records, SOS logs, and payment information
- **Maps and Tracking:** Google Maps API for real-time location tracking, route calculation, and geocoding

- Notifications: Firebase Cloud Messaging for push notifications and SOS alerts
- Admin Panel: React.js [9] 18.x with Node.js for web-based administration interface
- Cloud Hosting: Deployment on cloud infrastructure to enable accessibility, scalability, and remote data access

## 2.3 Current System

The transportation system in Pakistan for women is still largely dependent on traditional, unsafe, and fragmented options that limit mobility, security, and independence. Although a few ride-hailing applications such as Uber, InDrive, and Yango have become popular, these platforms operate on mixed-gender models and lack dedicated safety features for women. There is currently no women-only ride-sharing platform in Pakistan that combines verified female drivers, real-time SOS alerts, trusted contact sharing, and admin monitoring into a single integrated system.

Due to the absence of such a unified platform, female passengers face challenges in finding safe transportation, accessing emergency support during rides, and ensuring their security while traveling. Many women continue to rely on public buses, rickshaws, taxis, or male family members for transportation. Existing ride-hailing applications that do exist lack women-specific safety features and do not provide verified female driver options, resulting in safety concerns across the transportation ecosystem.

These gaps become even more evident in urban areas where women travel daily for work and education, and in semi-urban regions where transportation options are limited. Female passengers often have to compromise their safety while traveling, and there is no reliable way of ensuring that drivers are verified and trustworthy. Overall, Pakistan's current transportation system lacks the women-centric focus, safety integration, and digital accessibility required to support modern female mobility needs, highlighting the need for a comprehensive solution like SheRides.

### 2.3.1 Unsafe Public Transport Options

A significant portion of transportation options available to women in Pakistan including public buses, minibuses, rickshaws, and taxis remain overcrowded, unregulated, and unsafe. Public transport vehicles operate without gender segregation, and women often face harassment, inappropriate behavior, and safety concerns during their journeys. There are no dedicated women-only buses or transport services in most cities, and even where such services exist, they

are limited in coverage and availability.

This unsafe public transport system creates several mobility and personal security challenges for women. Women face verbal harassment, physical discomfort, and lack of security during their commutes. In many cases, women choose to avoid public transport altogether due to fear of harassment, forcing them to depend on male family members for rides or stay at home. This lack of safe transport options limits their ability to pursue education, employment, and social activities.

Additionally, the absence of regulated women-only transport imposes a significant psychological burden. Women spend considerable time worrying about their safety during travel, reducing their productivity and peace of mind. Over time, many women develop anxiety about using public transport. These inefficiencies highlight the urgent need for a secure, women-only ride-sharing platform that ensures safety, comfort, and independence for female passengers.

### **2.3.2 Safety Issues in Existing Ride-Hailing Applications**

Ride-hailing applications currently operating in Pakistan including InDrive, Uber, and Yango remain inconsistent in their safety features for women, decentralized in their emergency response mechanisms, and largely lacking women-only options. Passengers typically book rides through these applications, but driver verification is often inadequate, and there are no gender filters to ensure that only female drivers are assigned to female passengers. In some cases, basic in-app emergency buttons may be available, but these systems rarely integrate with trusted contacts or provide real-time admin monitoring.

This fragmented approach to women's safety creates several security difficulties. Female passengers often face challenges in trusting their assigned drivers, especially when drivers have poor ratings or negative feedback. Since there is no women-only filter in these applications, women cannot request female drivers specifically, resulting in discomfort and safety concerns. Female passengers also struggle to access emergency support when needed, as most applications lack dedicated SOS systems that notify both admin and family members simultaneously.

Common issues such as driver misbehavior, route deviation, harassment during rides, and lack of response from customer support persist due to poor safety monitoring. Ride-hailing applications often experience safety incidents during peak hours, where female passengers are more vulnerable. Harassment complaints are also frequent, as passengers do not receive timely

follow-up or resolution from application support teams.

Ultimately, the absence of a women-only ride-sharing platform with integrated safety features not only reduces women's mobility but also negatively impacts their trust in ride-hailing services. Female passengers may experience frustration due to safety concerns, uncertainty about driver verification, and difficulty accessing timely emergency support. A women-only, safety-focused ride-sharing platform would streamline ride booking processes, improve passenger security, and significantly enhance the travel experience for women.

### **2.3.3 Limited Emergency Response and Monitoring**

In the existing ride-hailing system, the accessibility of emergency response features is highly restricted due to the absence of dedicated SOS mechanisms integrated with admin monitoring and trusted contact notification. Passenger safety data is typically limited to basic ride tracking within a single application, making it nearly impossible for family members or authorities to retrieve or review real-time ride information when an emergency occurs. This becomes particularly problematic when female passengers face harassment, route deviation, or other safety incidents during rides. As a result, passenger security is compromised, and family members often have to rely on incomplete or delayed information provided by ride-hailing platforms.

Communication channels between passengers and emergency contacts are equally limited and inefficient. Most interactions occur through manual phone calls or text messages managed by passengers themselves. These communication methods often lead to delayed emergency response, missed notifications regarding ride status changes, or unavailability of timely updates about passenger location. Important safety features such as automatic location sharing with trusted contacts, real-time admin monitoring of rides, and immediate SOS alert delivery are seldom automated, leading to higher chances of delayed emergency intervention.

### **2.3.4 Driver Verification Gaps**

The current ride-hailing infrastructure does not adequately verify driver backgrounds to ensure passenger safety. Most ride-hailing platforms rely on basic document submission such as CNIC and driving license, but verification processes are often superficial and lack thorough background checks. These systems are vulnerable to a range of issues, including fake documents, expired licenses being accepted, or drivers with previous complaints continuing to operate on the platform. Due to the absence of standardized verification protocols, unverified or untrustworthy drivers may be allowed to accept rides, putting female passengers at risk.



Privacy and safety concerns are particularly significant because ride-hailing involves sharing personal location and travel details with drivers. Female passengers may be reluctant to use ride-hailing services due to fear of harassment or mishandling of their personal information, which can lead to limited mobility. Weak verification practices not only violate passenger trust but also reduce the credibility of ride-hailing platforms. These gaps are more pronounced in smaller cities where oversight mechanisms, compliance standards, and digital literacy levels are often low. Ensuring robust driver verification is therefore essential for protecting passenger safety and enhancing the trustworthiness of ride-hailing providers.

### **2.3.5 Lack of Passive Monitoring Features**

Although some ride-hailing applications have introduced basic ride tracking features for passengers, these tools are typically manual and designed to operate within a single user session. There is no integrated trusted contact system or automatic location sharing infrastructure to interconnect family members with ongoing rides, resulting in fragmented safety monitoring. Family members of female passengers cannot access shared ride records or real-time location updates, and location data generated by one user session is not automatically shared with pre-added trusted contacts.

The lack of passive monitoring prevents the seamless flow of safety information from passengers to their family members. For example, a passenger's live ride location cannot be automatically shared with a trusted contact without manual intervention by the passenger, forcing family members to rely on periodic check-ins.

Overall, the inadequate integration of passive monitoring features significantly limits the potential of technology to enhance passenger safety. A centralized, automatic location sharing system is essential for ensuring that passenger location data flows seamlessly to trusted contacts, enabling more effective emergency response, improved family peace of mind, and better long-term passenger safety outcomes.

### **2.3.6 No Women-Only Driver Option**

The current ride-hailing market does not offer women-only driver options, creating a noticeable gap in service for female passengers who prefer to travel with female drivers for cultural or safety reasons. While major ride-hailing platforms may have basic safety features, none provide the ability to request female drivers exclusively. Many female passengers feel uncomfortable traveling with male drivers due to past negative experiences, cultural norms, or personal preferences. As a result, these women either avoid using ride-hailing services altogether or travel with constant anxiety and discomfort.

Female passengers in conservative societies also face significant social barriers when traveling with male drivers. Many families discourage or prohibit female members from using ride-hailing services that do not offer female drivers. This not only limits women's mobility but also causes them to miss out on education and employment opportunities. Moreover, when female passengers are forced to use male-driven vehicles due to lack of options, they may experience harassment or uncomfortable situations.

These gaps create unequal mobility experiences, contributing to poorer quality of life outcomes for women in conservative societies. The lack of a women-only driver option therefore widens the gender mobility gap and highlights the pressing need for a women-centric ride-sharing platform that exclusively connects female passengers with verified female drivers.

### **2.3.7 Summary of Limitations**

Pakistan's current transportation system for women is hindered by multiple structural and operational limitations that collectively compromise safety, accessibility, and passenger confidence. The heavy dependence on mixed-gender ride-hailing platforms leads to safety concerns, harassment incidents, and inconsistent emergency response mechanisms. Female passengers often lack real-time safety tools including SOS alerts and trusted contact sharing, which affects their confidence and trust in ride-hailing services, especially during night-time or long-distance travel. Ride monitoring remains equally inefficient, as most applications lack admin oversight and cannot synchronize ride data with family members. This results in delayed emergency response, lack of accountability, and poor coordination between passengers and safety monitors.

These issues are amplified in smaller cities and semi-urban regions, where limited technological resources and lower awareness create further barriers to safe transportation for women. Collectively, these limitations demonstrate the urgent need for a secure, women-only, safety-focused ride-sharing platform. A system like SheRides would address these challenges by providing women-only ride options, real-time SOS alerts with multi-channel notification, automatic trusted contact sharing for passive monitoring, verified female drivers through document verification, admin oversight of all active rides, and mobile mechanic assistance for roadside emergencies. By digitizing and integrating safety features into a women-centric platform, SheRides has the potential to significantly elevate the overall safety, accessibility, and reliability of transportation services for women in Pakistan.

## **2.4 Summary**

The second chapter of this report presents a detailed exploration of the challenges within Pakistan's current transportation system for women and establishes the necessity for a unified, safety-focused, women-only solution such as SheRides. The chapter begins by defining the core problem: Pakistan lacks a women-only ride-sharing platform capable of integrating verified female drivers, real-time SOS alerts, trusted contact sharing, ride tracking, admin monitoring, and secure payment workflows. This fragmented transportation environment creates systemic safety concerns that negatively impact female passengers, female drivers, and administrators alike.

The chapter highlights how the absence of women-centric transportation features results in safety incidents, harassment, limited mobility, and poor emergency response. Many women continue to rely on public transport or mixed-gender ride-hailing applications, which operate without dedicated safety features. As a result, passenger safety becomes difficult to ensure, personal security is easy to compromise, and emergency support remains almost impossible to access across different ride-hailing platforms.

These limitations are further intensified by Pakistan's growing female workforce, increased pressure on women to pursue education and employment, and the rising need for safe, reliable, and accessible transportation options.

The chapter concludes that these challenges — unsafe public transport, safety issues in existing ride-hailing applications, limited emergency response, driver verification gaps, lack of passive monitoring features, and absence of women-only driver options — collectively demonstrate the urgent need for a centralized, women-only, safety-focused ride-sharing platform. SheRides is presented as a comprehensive solution designed to provide safe, reliable, and secure transportation for women across Pakistan. By offering women-only rides with verified female drivers, real-time SOS alerts, trusted contact circle for passive monitoring, recurring rides for daily commuters, admin monitoring of all active rides, and mobile mechanic assistance, SheRides aims to enhance women's mobility, improve passenger safety, strengthen administrative oversight, and ensure travel confidence. Through this safety-centric ecosystem, the project strives to transform Pakistan's transportation landscape for women from a fragmented, unsafe system into a unified, secure, and accessible mobility framework.

## **Chapter 3**

# **REQUIREMENT ANALYSIS**

## **3. Requirement Analysis**

The Requirement Analysis chapter provides a comprehensive and structured understanding of what the proposed system must accomplish in order to function effectively within the intended women-only ride-sharing environment. It serves as a foundation for translating user expectations into clear technical requirements by identifying system behavior, user interactions, and the specific tasks the system must perform. This chapter outlines how the SheRides system is expected to operate, detailing both functional and non-functional requirements while defining the roles and responsibilities of each actor involved, such as passengers, drivers, and administrators. Using the provided use case diagram as a reference, this chapter organizes the requirements into well-defined sections that guide the design, development, and validation processes. Through this analysis, the development team gains a precise understanding of the system's objectives, constraints, and performance expectations, ensuring that the final implementation aligns with the needs of all stakeholders and effectively addresses the limitations of the current transportation system for women in Pakistan.

### **3.1 Introduction to Requirement Analysis**

Requirement Analysis is a critical phase in the software development lifecycle, serving as the foundation upon which the entire system is designed, implemented, and evaluated. For the SheRides system, this phase ensures that all functional expectations, performance needs, and operational constraints are clearly understood before development begins. The purpose of conducting requirement analysis is to bridge the gap between user needs and technical implementation, ensuring the final system effectively resolves existing transportation challenges for women such as unsafe travel options, lack of verified female drivers, absence of emergency response features, and inefficient safety monitoring across ride-hailing platforms. This chapter outlines the detailed requirements of the SheRides platform, describing how each system component interacts with users, how the data flows, and what outcomes are expected from every interaction [1].

### **3.2 System Overview**

The SheRides system is designed as an integrated women-only ride-sharing platform that connects female passengers, verified female drivers, and administrators within a unified safety-focused environment. The system enables passengers to book rides, share real-time locations, use SOS alerts, schedule recurring rides, and connect with trusted contacts during emergencies. Drivers, on the other hand, can receive ride requests, manage profiles, accept or reject rides, start and end trips, view earnings, and maintain updated availability status. Meanwhile, the

administrator oversees the entire system by verifying driver documents, monitoring live rides, handling complaints, reviewing SOS incidents, managing user accounts, and ensuring that safety standards and database integrity are maintained.

The use case diagram illustrates the relationships between these actors and the system, showing how functionalities such as booking rides, triggering SOS alerts, requesting mechanic assistance, verifying drivers, and handling complaints are interconnected. This overview reflects the system's role as a centralized safety-centric [1] hub that replaces unsafe manual procedures with a streamlined, women-only, cloud-based solution accessible from mobile and web platforms. By consolidating all these processes, the SheRides platform addresses key issues of women's safety concerns, lack of verified female drivers, absence of emergency response features, and lack of uniform safety monitoring in existing ride-hailing applications.

### **3.3 Actors in the System**

The SheRides system interacts with multiple categories of users, each performing distinct roles that contribute to the overall functionality of the platform. These users, also known as actors, engage with the system through specific use cases that reflect their responsibilities, permissions, and operational needs. Understanding these actors and their interactions is crucial for defining accurate system requirements and ensuring that the platform delivers a seamless, safe, and secure experience for all users. Describe each actor in detail, outlining their characteristics, goals, and the actions they perform within the SheRides ecosystem.

#### **3.3.1 Passenger**

The passenger is the primary beneficiary of the SheRides system. Passengers interact with the platform to register themselves, book instant or recurring rides, and maintain access to their ride history and safety settings. Passengers can also trigger SOS alerts during emergencies, share live location with trusted contacts, receive ride confirmations and driver details, and provide feedback after ride completion. The system ensures that passengers can conveniently manage their travel needs without relying on unsafe public transport or male family members for rides. Female passengers are the only users allowed to book rides on the platform, as the system is exclusively designed for women.

#### **3.3.2 Driver**

Drivers are essential users who utilize the platform to receive ride requests, accept or reject rides, navigate to passenger locations, complete trips, and maintain their earnings and availability status. The system enables drivers to access real-time ride requests, ensuring efficient matching with nearby passengers. The digital interface helps female drivers reduce

dependency on external job sources and enhances their economic empowerment by providing organized, easily accessible ride requests and payment tracking. Only verified female drivers are allowed to register and accept rides on the platform.

### **3.3.3 Administrator**

The administrator oversees the entire SheRides platform. Their responsibilities include driver verification, document review, live ride monitoring, SOS alert handling, complaint management, user account management, and ensuring system security and data integrity. Administrators handle the approval of driver applications, review uploaded documents (CNIC, driving license, vehicle registration), monitor active rides in real-time, respond to SOS incidents, manage user complaints, and ensure that the database remains consistent and secure. Their control panel provides tools for monitoring system health, generating reports for operational improvements, and maintaining safety standards across the platform.

## **3.4 Use Case Diagrams**

Use case diagrams provide a clear and structured representation of how different users interact with the SheRides system. They help define the system's functional boundaries and illustrate the various operations performed by each actor. Figure 3.1 shows the primary use case diagram for the SheRides platform, based on the requirements identified in the SRS document.

In Figure 3.1, three main actors — Passenger, Driver, and Admin — interact with the system through a set of defined use cases. The Passenger uses the system to register an account, log in, book a ride, send SOS alerts, view ride history, add trusted contacts, request mechanic assistance, and make payments. The Driver interacts with the system by registering, logging in, accepting or rejecting rides, starting and ending rides, viewing earnings, and updating availability status. The Admin actor oversees system-level functionalities such as verifying drivers, monitoring live rides, handling complaints, viewing SOS alerts, and managing users.

Use case highlights how these actors connect with key system functionalities, such as ride booking, SOS alert triggering, trusted contact sharing, driver verification, and admin monitoring. Each use case represents a specific function that contributes to the overall ride-sharing safety workflow. By visualizing these interactions, the use case diagram ensures that all user needs, functional requirements, and system behaviors are correctly captured.

Overall, Figure 3.1 serves as a foundational model for understanding the operational structure of the SheRides system. It guides developers and stakeholders by clearly depicting what the system is expected to perform and how different users will interact with it. This helps eliminate

ambiguity, supports accurate requirement validation, and ensures that the system design aligns with the intended objectives of the project.

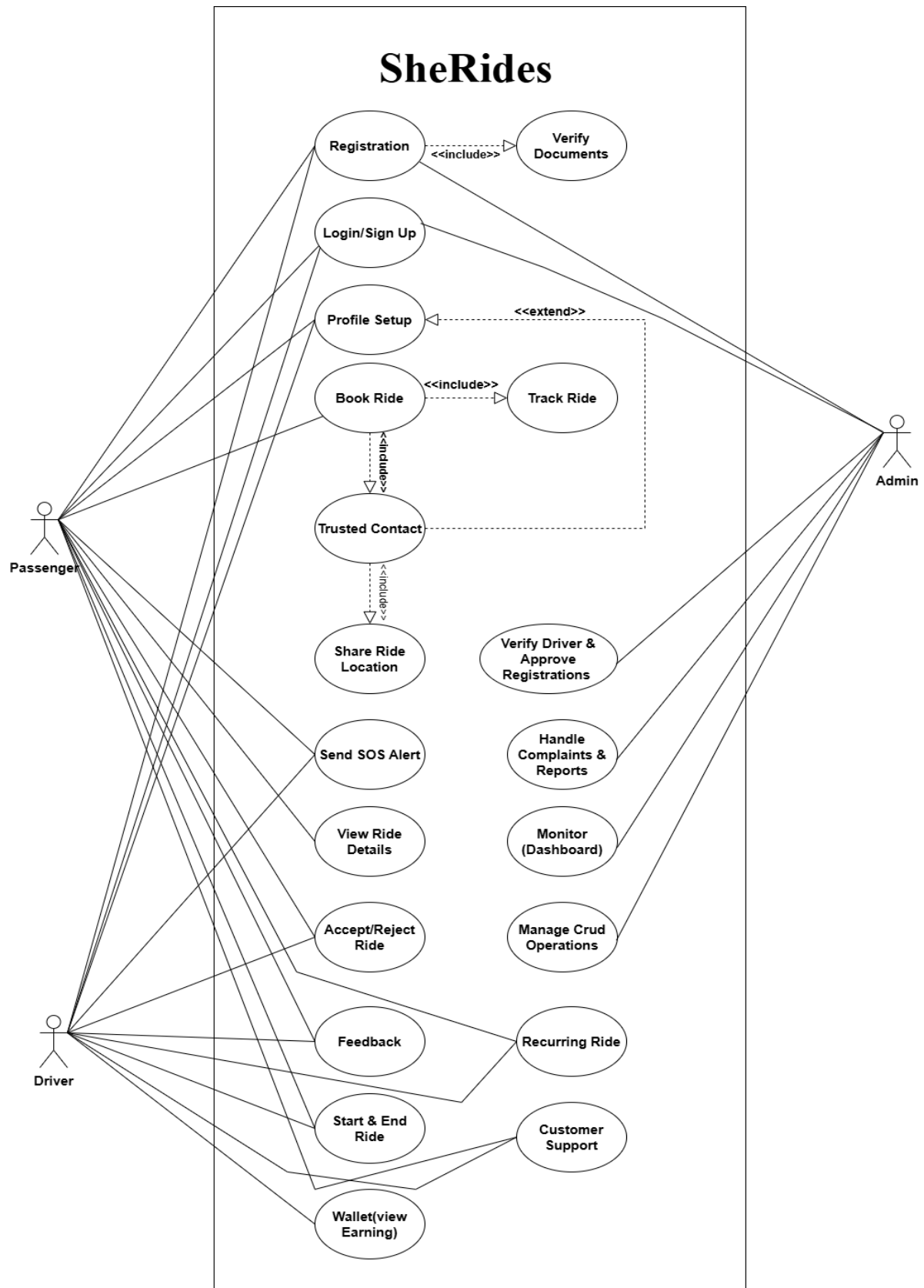


Figure 3.1: Use Case Diagram



## 3.5 Use Case Descriptions

This section provides detailed descriptions of all identified use cases. Each use case outlines actors involved, triggers, preconditions, postconditions, normal and alternative flows, exceptions, and business rules. These tables collectively define the full system behavior and serve as the foundation for design and development.

The following tables present the detailed descriptions of the major use cases included in the **SheRides** system. These use cases highlight how users interact with the system to perform essential ride-sharing tasks such as booking rides, sending SOS alerts, requesting mechanic assistance, and verifying drivers. Each table follows a standardized template to ensure clarity and consistency in requirement documentation.

### 3.5.1 Book Ride Use Case

The Book Ride use case defines how a passenger books a ride within the **SheRides** system. This use case ensures that every ride request is properly processed, drivers are matched, and ride tracking is enabled. It covers the complete ride booking flow from entering pickup and drop-off locations to driver acceptance and live tracking. This use case is essential for maintaining efficient ride matching, ensuring passenger safety, and providing real-time ride updates.

*Table 3.1: Structured description of Ride booking*

| Use Case Attribute | Details                                                                                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case Name      | Book Ride                                                                                                                                                                                 |
| Primary Actors     | Passenger                                                                                                                                                                                 |
| Secondary Actors   | Driver                                                                                                                                                                                    |
| Description        | The passenger books a ride by entering pickup and drop-off locations. The system finds nearby drivers and notifies them. Once a driver accepts, the trip starts and live tracking begins. |
| Trigger            | Passenger initiates the booking                                                                                                                                                           |

|                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | process                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Pre-Conditions   | <ul style="list-style-type: none"> <li>• Passenger must be logged in and verified</li> <li>• Passenger must be a verified female user</li> <li>• Internet and GPS must be active</li> </ul>                                                                                                                                                                                                                                                                           |
| Post-Conditions  | <ul style="list-style-type: none"> <li>• Trip details saved in database</li> <li>• Live tracking enabled</li> <li>• Driver assigned to ride</li> </ul>                                                                                                                                                                                                                                                                                                                |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Passenger enters pickup and destination points</li> <li>2. System shows estimated fare and available drivers</li> <li>3. Passenger confirms booking request</li> <li>4. System finds nearest available verified female driver</li> <li>5. Driver receives notification and accepts the request</li> <li>6. System starts live ride tracking</li> <li>7. Both passenger and driver see real-time location updates</li> </ol> |
| Alternative Flow | <ul style="list-style-type: none"> <li>• No drivers available → System displays "No Drivers Found"</li> </ul>                                                                                                                                                                                                                                                                                                                                                         |

|                |                                                                                                                                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                | <p>message and suggests scheduling a ride</p> <ul style="list-style-type: none"> <li>• Passenger cancels before driver accepts → Ride request removed, no driver notified</li> </ul>                                                                                        |
| Exceptions     | <ul style="list-style-type: none"> <li>• GPS signal lost → System temporarily pauses tracking and displays warning</li> <li>• No internet connection → System shows "Connect to internet" error</li> <li>• Server timeout → System retries request automatically</li> </ul> |
| Business Rules | <ul style="list-style-type: none"> <li>• Only verified female drivers are assigned to rides</li> <li>• Only verified female passengers can book rides</li> <li>• Fare must be calculated before ride confirmation</li> </ul>                                                |

### 3.5.2 Send SOS Alert Use Case

The Send SOS Alert use case defines how a passenger triggers an emergency alert during a ride within the SheRides system. This use case ensures that every SOS request is properly processed, alerts are delivered to admin and trusted contacts, and live location is shared. It covers the complete SOS flow from button press to alert delivery and monitoring. This use case is essential for maintaining passenger safety, enabling rapid emergency response, and providing peace of mind during rides.

**Table 3.2: Structured description of the Send SOS Alert use case.**

| Use Case Attribute | Details |
|--------------------|---------|
|--------------------|---------|

|                  |                                                                                                                                                                                                                                |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case Name    | Send SOS Alert                                                                                                                                                                                                                 |
| Primary Actors   | Passenger                                                                                                                                                                                                                      |
| Secondary Actors | Trusted Contacts, Admin                                                                                                                                                                                                        |
| Description      | The passenger presses the SOS button during a ride to alert the admin and trusted contacts with live location. The system sends immediate notifications and displays confirmation.                                             |
| Trigger          | Passenger feels unsafe during the ride or security purpose                                                                                                                                                                     |
| Pre-Conditions   | <ul style="list-style-type: none"> <li>• Passenger must be logged in and currently on an active ride</li> <li>• Internet and GPS must be active</li> <li>• Trusted contacts must be added to passenger profile</li> </ul>      |
| Post-Conditions  | <ul style="list-style-type: none"> <li>• SOS alert sent to admin and all trusted contacts</li> <li>• SOS event logged in database with timestamp and location</li> <li>• Admin receives notification for monitoring</li> </ul> |
| Normal Flow      | 1. Passenger taps the SOS button during active ride                                                                                                                                                                            |

|                  |                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | <p>2. System displays confirmation dialog</p> <p>3. Passenger confirms SOS trigger</p> <p>4. System captures current live location</p> <p>5. System sends emergency alerts to admin dashboard</p> <p>6. System sends SMS/push notifications to all trusted contacts</p> <p>7. System displays "SOS Sent" confirmation to passenger</p> <p>8. Admin monitors the situation in real-time</p> |
| Alternative Flow | <ul style="list-style-type: none"> <li>• No internet connection → System retries sending alert automatically when connection restored</li> <li>• False SOS press → User cancels within 5 seconds before confirmation</li> </ul>                                                                                                                                                            |
| Exceptions       | <ul style="list-style-type: none"> <li>• GPS signal lost during SOS → System sends last known location with alert</li> <li>• Trusted contact numbers invalid → System logs error and attempts alternate notification methods</li> </ul>                                                                                                                                                    |

|                |                                                                                                                                                                                                                                              |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                | <ul style="list-style-type: none"> <li>• Server downtime → System queues SOS request and retries</li> </ul>                                                                                                                                  |
| Business Rules | <ul style="list-style-type: none"> <li>• SOS must reach admin and trusted contacts within 5 seconds</li> <li>• SOS can only be triggered during active rides</li> <li>• False SOS cancellations must be logged for audit purposes</li> </ul> |

### 3.5.3 Request Mechanic Assistance Use Case

The Request Mechanic Assistance use case defines how a passenger or driver requests roadside help during a ride within the **SheRides** system. This use case ensures that every mechanic request is properly processed, AI guidance is provided, and assistance is dispatched. It covers the complete mechanic assistance flow from request to resolution. This use case is essential for maintaining passenger and driver safety during vehicle breakdowns and ensuring timely roadside support.

*Table 3.3: Structured description of the Request Mechanic Assistance use case.*

| Use Case Attribute | Details                                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case Name      | Request Mechanic Assistance                                                                                                                                                 |
| Primary Actors     | Passenger, Driver                                                                                                                                                           |
| Secondary Actors   | AI Mechanic Module, Admin                                                                                                                                                   |
| Description        | In case of vehicle breakdown, the user requests roadside assistance. The AI module provides quick troubleshooting guidance and connects to a verified mechanic if required. |
| Trigger            | Vehicle stops unexpectedly or                                                                                                                                               |

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | user reports mechanical issue during ride                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Pre-Conditions  | <ul style="list-style-type: none"> <li>• Ride must be active</li> <li>• Internet and GPS must be active</li> <li>• User must be logged in</li> </ul>                                                                                                                                                                                                                                                                                                                                    |
| Post-Conditions | <ul style="list-style-type: none"> <li>• Mechanic request logged in database</li> <li>• AI troubleshooting guidance provided</li> <li>• Assistance status updated after resolution</li> </ul>                                                                                                                                                                                                                                                                                           |
| Normal Flow     | <ol style="list-style-type: none"> <li>1. User clicks "Request Mechanic Assistance" button</li> <li>2. System displays troubleshooting options</li> <li>3. AI Mechanic module provides basic repair suggestions</li> <li>4. If issue resolved, user confirms and closes request</li> <li>5. If issue unresolved, system searches for nearby verified mechanics</li> <li>6. System assigns nearest available mechanic</li> <li>7. Admin tracks assistance status in real-time</li> </ol> |

|                  |                                                                                                                                                                                                                                                                                                                        |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | 8. User receives mechanic arrival confirmation                                                                                                                                                                                                                                                                         |
| Alternative Flow | <ul style="list-style-type: none"> <li>• No mechanic available nearby → System provides emergency hotline numbers and towing service contacts</li> <li>• User resolves issue independently → System closes request without mechanic dispatch</li> </ul>                                                                |
| Exceptions       | <ul style="list-style-type: none"> <li>• GPS signal lost → System uses last known location for mechanic assignment</li> <li>• No internet connection → System displays offline help guide and suggests calling helpline</li> <li>• No mechanic responds → System escalates to admin for manual coordination</li> </ul> |
| Business Rules   | <ul style="list-style-type: none"> <li>• All available mechanic workshops must be shown with distance and estimated arrival time</li> <li>• AI mechanic availability must be updated regularly</li> <li>• Only verified mechanics can be assigned through the platform</li> </ul>                                      |



### 3.5.4 Verify Driver and Handle Complaints Use Case

The Verify Driver and Handle Complaints use case defines how the admin manages driver verification and user complaints within the SheRides system. This use case ensures that every driver application is properly reviewed, documents are verified, and complaints are addressed. It covers the complete admin workflow from driver registration approval to complaint resolution. This use case is essential for maintaining platform trust, ensuring only verified female drivers operate, and providing accountability for user issues.

*Table 3.4: Structured description of the Verify Driver and Handle Complaints use case.*

| Use Case Attribute | Details                                                                                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case Name      | Verify Driver and Handle Complaints                                                                                                                                                |
| Primary Actors     | Admin                                                                                                                                                                              |
| Secondary Actors   | Driver, Passenger                                                                                                                                                                  |
| Description        | The admin verifies driver documents, monitors ride activities, and handles user complaints or incident reports through the centralized web dashboard.                              |
| Trigger            | <ul style="list-style-type: none"><li>• Driver applies for registration</li><li>• Complaint is filed by passenger or driver</li><li>• SOS incident requires admin review</li></ul> |
| Pre-Conditions     | <ul style="list-style-type: none"><li>• Admin must be logged into web portal with authorized credentials</li><li>• Driver must submit valid</li></ul>                              |

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | documents for verification <ul style="list-style-type: none"> <li>• Complaint must contain required details</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Post-Conditions | <ul style="list-style-type: none"> <li>• Driver status updated (approved/rejected/pending)</li> <li>• Complaint marked as resolved or escalated</li> <li>• SOS incident reviewed and closed</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Normal Flow     | <ol style="list-style-type: none"> <li>1. Admin logs into web portal with secure authentication</li> <li>2. Admin navigates to driver verification section</li> <li>3. System displays list of pending driver applications</li> <li>4. Admin reviews submitted documents (CNIC, license, vehicle registration)</li> <li>5. Admin approves or rejects application based on verification</li> <li>6. Admin navigates to complaint management section</li> <li>7. System displays list of open complaints</li> <li>8. Admin reviews each complaint and takes appropriate action</li> <li>9. Admin navigates to live ride</li> </ol> |

|                  |                                                                                                                                                                                                                                                                                                                                            |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | <p>monitoring section</p> <p>10. Admin monitors active trips and SOS incidents in real-time</p>                                                                                                                                                                                                                                            |
| Alternative Flow | <ul style="list-style-type: none"> <li>• Documents incomplete or invalid → Admin rejects application with reason and requests re-upload</li> <li>• Complaint requires more information → Admin requests additional details from user</li> <li>• SOS incident active → Admin contacts passenger and trusted contacts immediately</li> </ul> |
| Exceptions       | <ul style="list-style-type: none"> <li>• System downtime during review → Admin unable to access dashboard, review rescheduled</li> <li>• Invalid document format → System displays error and prompts driver to upload correct format</li> <li>• Duplicate complaint submission → System merges with existing complaint</li> </ul>          |
| Business Rules   | <ul style="list-style-type: none"> <li>• Only authorized admins can access the verification and complaint management panel</li> <li>• Drivers cannot accept rides until admin approves verification</li> </ul>                                                                                                                             |

|  |                                                                                                                                                                                              |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <ul style="list-style-type: none"> <li>• All complaints must be responded to within 24 hours</li> <li>• SOS incidents have highest priority and require immediate admin attention</li> </ul> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 3.6 Functional Requirements

Functional requirements [2] define the specific behaviors and operations that the SheRides system must provide to satisfy the needs of its users. These requirements outline the essential capabilities that the system must possess to manage ride booking, safety features, driver verification, trusted contacts, payments, and administrative tasks effectively. The SheRides system is designed to offer a secure, reliable, and user-friendly environment that ensures passenger safety, data privacy, and efficient ride-sharing service management for women across Pakistan.

### 3.6.1 Women Safety and SOS Alerts (FR-1)

The SheRides system provides a comprehensive SOS alert mechanism to ensure passenger safety during rides. The passenger can send an SOS alert during an active ride to notify trusted contacts and the admin with live location. The system triggers an alert sound and displays confirmation to ensure the action was successful. The SOS button remains easily accessible from every ride screen, and the system automatically retries sending if the first attempt fails due to network issues. This feature is critical for providing immediate help in emergencies and preventing harassment incidents.

*Table 3.5: FR-1(SOS Alert)*

|                    |                                                                                                                                                                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>  | FR-1                                                                                                                                                                                                                                      |
| <b>Title</b>       | SOS Alerts and Safety Monitoring                                                                                                                                                                                                          |
| <b>Requirement</b> | The passenger shall be able to send an SOS alert during a ride to notify her trusted contacts and the admin with her live location. The system shall trigger an alert sound and display confirmation to ensure the action was successful. |
| <b>Source</b>      | Safety module analysis & Observation from recent harassing incidents.                                                                                                                                                                     |
| <b>Rationale</b>   | To provide immediate help in emergencies and prevent harassment                                                                                                                                                                           |

|                                    |                                                               |
|------------------------------------|---------------------------------------------------------------|
|                                    | incidents.                                                    |
| <b>Business Rule (if required)</b> | SOS must be delivered within 5 seconds to admin and contacts. |
| <b>Dependencies</b>                | FR-4 (Trusted Contact), FR-7 (Admin Monitoring).              |
| <b>Priority</b>                    | High.                                                         |

### 3.6.2 Recurring Rides (FR-2)

The SheRides system allows passengers to schedule daily or weekly rides automatically for recurring travel routes such as office or university commutes. This feature improves convenience and reduces booking time for routine trips. Passengers can set pickup location, destination, preferred time, and frequency (daily, weekdays, or specific days). Recurring rides can be modified or cancelled up to 30 minutes before scheduled time.

*Table 3.6: FR-2 Recurring Rides*

|                                    |                                                                                                                                                 |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>                  | FR-2                                                                                                                                            |
| <b>Title</b>                       | Schedule and Manage Recurring Rides                                                                                                             |
| <b>Requirement</b>                 | The passenger shall be able to schedule daily or weekly rides automatically for recurring travel routes, such as office or university commutes. |
| <b>Source</b>                      | Working women feedback.                                                                                                                         |
| <b>Rationale</b>                   | To improve convenience and reduce booking time for routine trips.                                                                               |
| <b>Business Rule (if required)</b> | Recurring rides can be modified or cancelled up to 30 minutes before scheduled time.                                                            |
| <b>Dependencies</b>                | FR-5 (Payment System)                                                                                                                           |
| <b>Priority</b>                    | Medium.                                                                                                                                         |

### 3.6.3 Mobile Mechanic Assistance (FR-3)

The SheRides system allows users to request mechanic assistance in case of vehicle breakdown during a ride. The system uses AI guidance to suggest initial troubleshooting steps. If the issue remains unresolved, the system connects to nearby verified mechanics. This feature assists users in handling roadside emergencies efficiently and ensures women are not left stranded in unsafe situations.

**Table 3.7: FR-3 Mechanic Assistant**

|                                    |                                                                                                                                                                 |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>                  | FR-3                                                                                                                                                            |
| <b>Title</b>                       | Request Mechanic Assistance                                                                                                                                     |
| <b>Requirement</b>                 | The user shall be able to request a mechanic assistant in case of vehicle breakdown. The system shall use AI guidance to suggest initial troubleshooting steps. |
| <b>Source</b>                      | Safety module extension.                                                                                                                                        |
| <b>Rationale</b>                   | To assist users in handling roadside emergencies efficiently.                                                                                                   |
| <b>Business Rule (if required)</b> | <i>NIL</i>                                                                                                                                                      |
| <b>Dependencies</b>                | FR-7 (Admin Verification).                                                                                                                                      |
| <b>Priority</b>                    | High.                                                                                                                                                           |

### 3.6.4 Trusted Contact Circle (FR-4)

The SheRides system allows passengers to add trusted contacts who can receive live ride location and status updates automatically. During every ride, location updates are automatically shared with these contacts for passive protection. This feature enhances security by involving trusted people during rides and gives peace of mind to both the rider and family members.

**Table 3.8: FR-4 Share Location**

|                                    |                                                                                                                             |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>                  | FR-4                                                                                                                        |
| <b>Title</b>                       | Share Location with Trusted Contacts.                                                                                       |
| <b>Requirement</b>                 | Passengers shall be able to add trusted contacts who can receive their live ride location and status updates automatically. |
| <b>Source</b>                      | Safety module analysis & Observation from recent harassing incidents.<br>Women's safety requirements.                       |
| <b>Rationale</b>                   | To enhance security by involving trusted people during rides.                                                               |
| <b>Business Rule (if required)</b> | Only up to 3 trusted contacts per user are allowed.                                                                         |
| <b>Dependencies</b>                | FR-1 (SOS System).                                                                                                          |
| <b>Priority</b>                    | High.                                                                                                                       |

### 3.6.5 Payment Method (FR-5)

The SheRides system allows users to make payments through in-app wallet or cash. All transactions are verified and recorded in the database. The system ensures secure and transparent transactions between passengers and drivers. Payment cannot proceed without proper authentication, and all payment history is maintained for future reference.

*Table 3.9: FR-5 Secure Payment and Authentication*

|                                    |                                                                                                                                             |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>                  | FR-5                                                                                                                                        |
| <b>Title</b>                       | Secure Payment and Authentication                                                                                                           |
| <b>Requirement</b>                 | The user shall be able to make payments through the in-app wallet or cash. All transactions shall be verified and recorded in the database. |
| <b>Source</b>                      | System design, user convenience.                                                                                                            |
| <b>Rationale</b>                   | To ensure secure and transparent transactions between passengers and drivers.                                                               |
| <b>Business Rule (if required)</b> | Payment cannot proceed without authentication.                                                                                              |
| <b>Dependencies</b>                | FR-6 (Driver Verification).                                                                                                                 |
| <b>Priority</b>                    | Medium.                                                                                                                                     |

### 3.6.6 Driver Registration and Verification (FR-6)

The SheRides system allows female drivers to register by submitting identity documents including CNIC, driving license, and vehicle registration. The admin verifies and approves the driver before they can accept rides. This ensures only verified female drivers are allowed to operate on the platform, building trust and authenticity.

*Table 3.10: FR-6 Driver profile*

|                    |                                                                                                                                                  |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>  | FR-6                                                                                                                                             |
| <b>Title</b>       | Driver Profile and Verification.                                                                                                                 |
| <b>Requirement</b> | The driver shall be able to register by submitting identity documents.<br>The admin shall verify and approve before the driver can accept rides. |
| <b>Source</b>      | Admin requirement.                                                                                                                               |
| <b>Rationale</b>   | To ensure only verified drivers are allowed to operate.                                                                                          |

|                                    |                                             |
|------------------------------------|---------------------------------------------|
| <b>Business Rule (if required)</b> | Drivers must upload valid CNIC and license. |
| <b>Dependencies</b>                | FR-7 (Admin Monitoring).                    |
| <b>Priority</b>                    | High.                                       |

### 3.6.7 Admin Panel (FR-7)

The SheRides system provides a centralized web dashboard for administrators to monitor active trips, verify drivers, and handle complaints. Only authorized admins can access this panel. The admin can view live rides on map, review driver applications, respond to SOS alerts, manage user complaints, and generate system analytics reports.

*Table 3.11: FR-7 Admin Monitoring*

|                                    |                                                                                                                             |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>                  | FR-7                                                                                                                        |
| <b>Title</b>                       | Admin Monitoring and Incident Management.                                                                                   |
| <b>Requirement</b>                 | The admin shall be able to monitor active trips, verify drivers, and handle complaints through a centralized web dashboard. |
| <b>Source</b>                      | Admin workflow requirement.                                                                                                 |
| <b>Rationale</b>                   | To maintain system integrity and safety for passengers.                                                                     |
| <b>Business Rule (if required)</b> | Only authorized admins can access this panel.                                                                               |
| <b>Dependencies</b>                | FR-1 (SOS Alerts), FR-6 (Driver Registration)                                                                               |
| <b>Priority</b>                    | High.                                                                                                                       |

### 3.6.8 Customer Care (FR-8)

The SheRides system allows users to contact customer support anytime for technical or safety issues. Complaints are logged and reviewed by the admin. Response time should not exceed 30 minutes for SOS-related complaints.

*Table 3.12: FR-8 24/7 Support and Complaint Handling*

|                   |                                     |
|-------------------|-------------------------------------|
| <b>Identifier</b> | FR-8                                |
| <b>Title</b>      | 24/7 Support and Complaint Handling |



|                                    |                                                                                                                                                  |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requirement</b>                 | The user shall be able to contact customer support anytime for technical or safety issues. Complaints shall be logged and reviewed by the admin. |
| <b>Source</b>                      | User Feedback & service design.                                                                                                                  |
| <b>Rationale</b>                   | To ensure continuous support and user trust.                                                                                                     |
| <b>Business Rule (if required)</b> | Response time should not exceed 30 minutes for SOS-related complaints.                                                                           |
| <b>Dependencies</b>                | FR-7 (Admin Panel).                                                                                                                              |
| <b>Priority</b>                    | Medium.                                                                                                                                          |

### 3.7 Non-Functional Requirements

Non-functional requirements [2] define the quality attributes, performance standards, and operational constraints that the SheRides system must meet. While functional requirements describe what the system does, non-functional requirements specify how the system performs and the standards it must uphold in terms of reliability, security, usability, and efficiency. These requirements are critical for ensuring that the SheRides system operates effectively under real-world conditions, meets user expectations, and complies with safety standards for women-centric ride-sharing platforms.

#### 3.7.1 Performance

The SheRides system must be capable of handling multiple simultaneous users without noticeable delays, ensuring fast response times for all operations, including booking rides, tracking drivers, triggering SOS alerts, and processing payments. The system should be optimized for high performance so that queries on ride history, driver availability, and SOS logs are executed efficiently.

- **PER-1:** The system shall process and confirm ride bookings within 3 seconds.
- **PER-2:** SOS alerts shall reach the admin and trusted contacts within 5 seconds.
- **PER-3:** The system shall handle up to 500 concurrent ride requests without delay.

#### 3.7.2 Security

Security is a top priority for SheRides since it deals with sensitive user data and safety-critical

features. The system must enforce strong authentication, encryption, and user verification to prevent misuse or unauthorized access.

- **SEC-1:** All user data, including personal details and ride history, shall be encrypted using SSL protocols.
- **SEC-2:** Only verified female users shall be allowed to register and log in.
- **SEC-3:** Passwords shall be stored in hashed form using Firebase authentication.

### **3.7.3 Reliability and Availability**

SheRides must provide high reliability and ensure continuous availability for all users, including passengers, drivers, and administrators. The system should have minimal downtime, and robust error-handling mechanisms must be in place to manage server failures, network disruptions, or unexpected exceptions. The system shall maintain at least 99% uptime. Availability ensures that the app and its services are accessible whenever users need them. Since *SheRides* may be used at any time of day, the system must operate continuously and reconnect automatically in case of network disruptions.

- **AVL-1:** The mobile app and admin portal shall remain available 24/7 except for scheduled maintenance.
- **AVL-2:** All services shall automatically reconnect after internet restoration.

### **3.7.4 Usability**

The system must offer an intuitive and user-friendly interface suitable for female users with varying levels of technical expertise. Passengers, drivers, and administrators should be able to navigate the system easily to perform tasks such as booking rides, triggering SOS alerts, adding trusted contacts, verifying drivers, or managing complaints. The system shall have an intuitive interface that allows new users to learn basic functions within 10 minutes. The app shall support both English and Urdu language options. Buttons for SOS and trusted contact sharing shall be easily accessible from every ride screen. Clear navigation, consistent design, accessible forms, and contextual help features are required to enhance the user experience and reduce errors during emergencies.

### **3.7.5 Maintainability**

SheRides must be designed for ease of maintenance and updates. The system should support modular architecture, allowing components such as ride booking, SOS alerts, trusted contacts,

and admin panel to be updated or replaced without affecting other functionalities. The use of Flutter for mobile frontend, Node.js for backend, and MongoDB for database supports this modularity. Clear documentation, coding standards, and version control practices (Git) must be followed to facilitate efficient maintenance, debugging, and future enhancements.

### **3.7.6 Performance Monitoring and Logging**

The system must include mechanisms for monitoring performance, logging user activity, and detecting anomalies. Real-time monitoring of server load, database performance, API response times, and application errors ensures that potential issues are identified and addressed proactively. All SOS alerts, ride completions, driver verifications, and complaint submissions shall be logged with timestamps for auditing and accountability purposes.

# **Chapter 4**

## **DESIGN AND ARCHITECTURE**

## **4. Design and Architecture**

SheRides system explains how the software is structured, how its components interact, and how data flows throughout the system. This chapter provides a detailed overview of the system's architecture, data representation, process flows, and design models. It highlights the technical decisions made to ensure performance, security, scalability, and maintainability. By presenting the system design clearly, this chapter demonstrates how SheRides meets both functional and non-functional requirements while delivering a reliable women-only ride-sharing application.

### **4.1 System Architecture**

The system architecture defines the overall structure of SheRides and illustrates how its main components work together to deliver safe and efficient ride-sharing services for women. It ensures that each module performs its specific tasks while interacting seamlessly with others to maintain smooth operations, passenger safety, and data integrity.

#### **4.1.1 Architecture Style**

SheRides is developed using a multi-tier client-server architecture. The mobile client layer consists of Flutter-based interfaces for passengers and drivers. The web client layer provides the React-based admin panel. The backend layer uses Node.js + Express to handle business logic and API requests. The database layer uses MongoDB for document-based data storage. Firebase Authentication is integrated for secure login and identity verification. This separation of concerns enhances maintainability, supports scalability as the number of users grows, and ensures secure, organized handling of system operations.

#### **4.1.2 Components**

The SheRides system is composed of several interconnected components, each designed to perform a specific function within the overall architecture. These components work collaboratively to ensure smooth system operation, secure data handling, and efficient ride-sharing service delivery. By dividing the system into modular parts, SheRides maintains clarity in functionality, supports easier maintenance, and allows individual components to evolve without affecting the entire system.

##### **4.1.2.1 Mobile Application (Flutter)**

The mobile application serves as the primary interface for passengers and drivers. It handles ride booking, viewing nearby drivers, sending SOS alerts, sharing live location with trusted contacts, managing recurring rides, and handling payments and profiles. Flutter allows building

a clean and responsive interface with a single codebase for both Android and iOS platforms.

#### **4.1.2.2 Authentication Service (Firebase)**

The authentication service manages user registration, login, and role-based access control. It ensures that only authorized users can access the system according to their roles, implementing security measures such as email verification and secure password storage. This component safeguards sensitive user data and prevents unauthorized access. Only verified female users are allowed to register as passengers or drivers.

#### **4.1.2.3 Backend Layer (Node.js + Express)**

The backend processes all business logic including ride assignment, driver matching, fare calculation, and notification delivery. It handles API requests from both the mobile app and admin panel. Firebase Authentication is integrated here so that login and registration remain secure and separate from the database.

#### **4.1.2.4 Database Layer (MongoDB)**

MongoDB is used as the database because data in SheRides is mostly document-based including user profiles, ride objects, SOS logs, and payment records. Different collections store passengers, drivers, active rides, payments, complaints, and recurring ride settings. MongoDB gives flexibility to scale without rigid table structures.

#### **4.1.2.5 Admin Panel (MERN Stack)**

The admin system is built using React + Node.js + MongoDB, with Firebase Auth for secure login. Admins perform the following tasks: approving drivers, monitoring live rides, viewing SOS alerts, reviewing complaints, checking system analytics, and managing users. The admin panel uses the same backend APIs as the mobile app, which keeps the system consistent and reduces duplication.

#### **4.1.2.6 External Services**

External services include Google Maps API for location tracking and routing, and Firebase Cloud Messaging for push notifications and SOS alerts. These services enhance the functionality of the platform by providing real-time navigation and instant communication.

### **4.1.3 Interactions**

All components communicate primarily through RESTful APIs, ensuring efficient and standardized data exchange. The mobile app sends requests to the backend layer, which processes data, interacts with the database, and responds with relevant information. For instance, when a passenger books a ride, the mobile app sends the request to the backend, which verifies driver availability, assigns a driver, updates records, and sends notifications to both the

passenger and driver. All communication is secured using HTTPS protocols and authentication tokens to protect data integrity and user privacy.

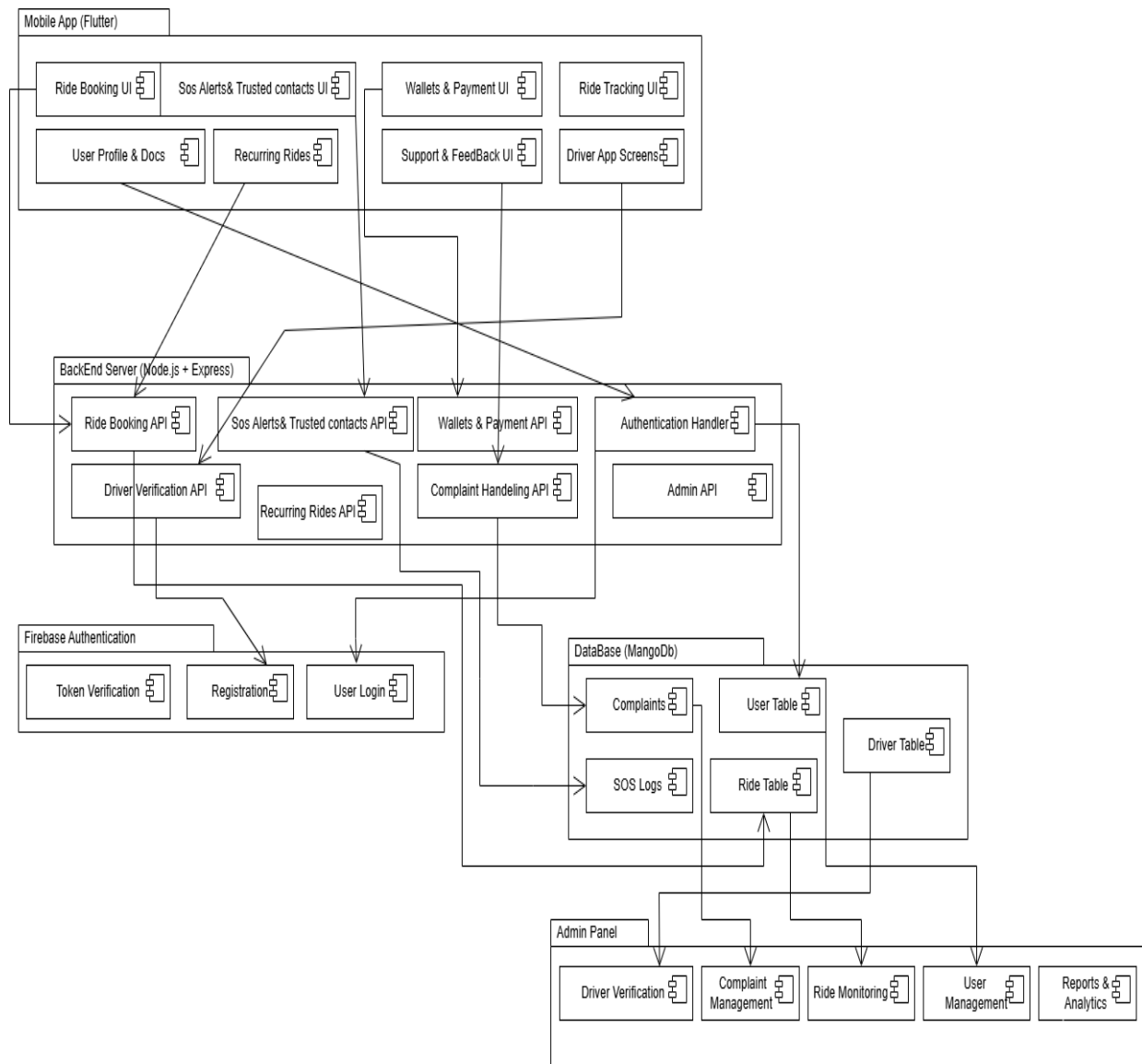
#### **4.1.4 System Architecture Diagram**

The system architecture diagram (Figure 4.1) illustrates the overall structure and interaction of the SheRides system components. It shows how users including passengers, drivers, and administrators interact with the application through mobile and web interfaces. The diagram highlights the flow of information between the mobile app, backend server, database, and external services.

The architecture is designed to ensure smooth communication between modules and efficient data management. When a user performs an action, such as logging in, booking a ride, or triggering an SOS alert, the request is sent to the backend server. The server processes the request, interacts with the database or Firebase to retrieve or store data, and then returns the response to the user interface. This structure provides a clear separation of concerns, improves system scalability, and enhances maintainability.

The architecture also ensures secure communication and reliable system performance by integrating authentication, data storage, and external APIs within a unified framework. Firebase Authentication is used to verify user identities and manage secure login sessions, while MongoDB stores application data such as user profiles, ride information, SOS alerts, and driver verification records. Google Maps API enables accurate location tracking, route generation, and distance calculations, whereas Firebase Cloud Messaging delivers real-time notifications to passengers, drivers, and administrators. By integrating these technologies within a layered architecture, the system achieves better security, faster response time, and improved reliability while supporting future scalability and feature enhancements.

Another important aspect of the SheRides architecture is its modular design, where each component performs a specific responsibility while remaining independent of other modules. This modular approach simplifies system development, testing, and maintenance by allowing developers to update or modify individual modules without affecting the overall application. It also improves code reusability and reduces system complexity. Furthermore, the architecture supports efficient communication between the mobile application, backend services, and database, ensuring that user requests are processed accurately and consistently. As a result, the system remains reliable, flexible, and capable of accommodating future enhancements as user requirements evolve.



**Figure 4.1: System Architecture Diagram**

## 4.2 Data Representation

The Data Representation section explains how information is structured, stored, and managed within the SheRides system. It covers the database design approach, entity relationships, data flow between modules, and the organization and security of stored data. This ensures that all system components can efficiently access and process the information they need while maintaining accuracy, consistency, and privacy.

### 4.2.1 Database Design

SheRides uses a NoSQL database (MongoDB) [7] due to its flexibility, scalability, and ability to handle large volumes of semi-structured ride-sharing data. Since ride records, user profiles, SOS logs, and payment histories vary in format, MongoDB allows storing this information in dynamic document structures. This design supports rapid read/write operations, horizontal



scalability, and easy integration with cloud-based solutions. The decision to use MongoDB also helps maintain system responsiveness even when managing thousands of passengers, drivers, and real-time ride updates.

## 4.2.2 Data Entities

The following data entities are stored in the SheRides MongoDB database:

*Table 4.1: Entities stored in MongoDB*

| Entity          | Purpose                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------|
| Users           | Stores passenger and driver profiles including name, contact, gender, and ride history       |
| Drivers         | Stores driver details, verification documents, vehicle information, and rating               |
| Rides           | Stores each ride's data including pickup location, destination, fare, timestamps, and status |
| TrustedContacts | Stores emergency contact details selected by passengers with name and phone number           |
| SOS Alerts      | Logs emergency events with location, timestamp, and admin response status                    |
| Payments        | Stores fare details, payment method, amount, and status                                      |
| Admin           | Stores admin access information and control rights                                           |

### 4.2.3 Data Dictionary

The data dictionary defines the attributes and methods associated with each entity:

*Table 4.2: Attributes and Methods associate*

| Entity         | Attributes                                                  | Methods                                                                                               |
|----------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| User           | userId, name, phone, gender                                 | Register(), Login(), UpdateProfile(), BookRide(), Payment(), TrustedContact(), SOSAlert(), Feedback() |
| Driver         | driverId,name, documents, vehicleInfo, rating               | acceptRide(), updateStatus(), viewEarnings()                                                          |
| Ride           | rideId, userId, driverId, pickup, destination, fare, status | startRide(), endRide(), updateLocation()                                                              |
| TrustedContact | contactId, userId, name, phone                              | notifySOS(), receiveUpdates()                                                                         |
| SOS Alert      | sosId, userId, location, timestamp, status                  | triggerAlert(), notifyAdmin(), notifyContacts()                                                       |
| Payment        | paymentId, rideId, method, amount, status                   | processPayment(), generateReceipt()                                                                   |

|       |                     |                                                                        |
|-------|---------------------|------------------------------------------------------------------------|
| Admin | adminId, name, role | verifyDriver(),<br>viewSOS(),<br>manageComplaints(),<br>monitorRides() |
|-------|---------------------|------------------------------------------------------------------------|

#### 4.2.4 Entity–Relationship Representation

Although MongoDB does not use traditional relational tables, the system follows a structured entity model to maintain consistency. Each entity (such as User, Passenger, Driver, Ride, SOSAlert, TrustedContact, Payment, and Admin) is stored as a document with well-defined attributes. Relationships are implemented using document references or embedded documents depending on access frequency and data size.

The main entities in the SheRides system include:

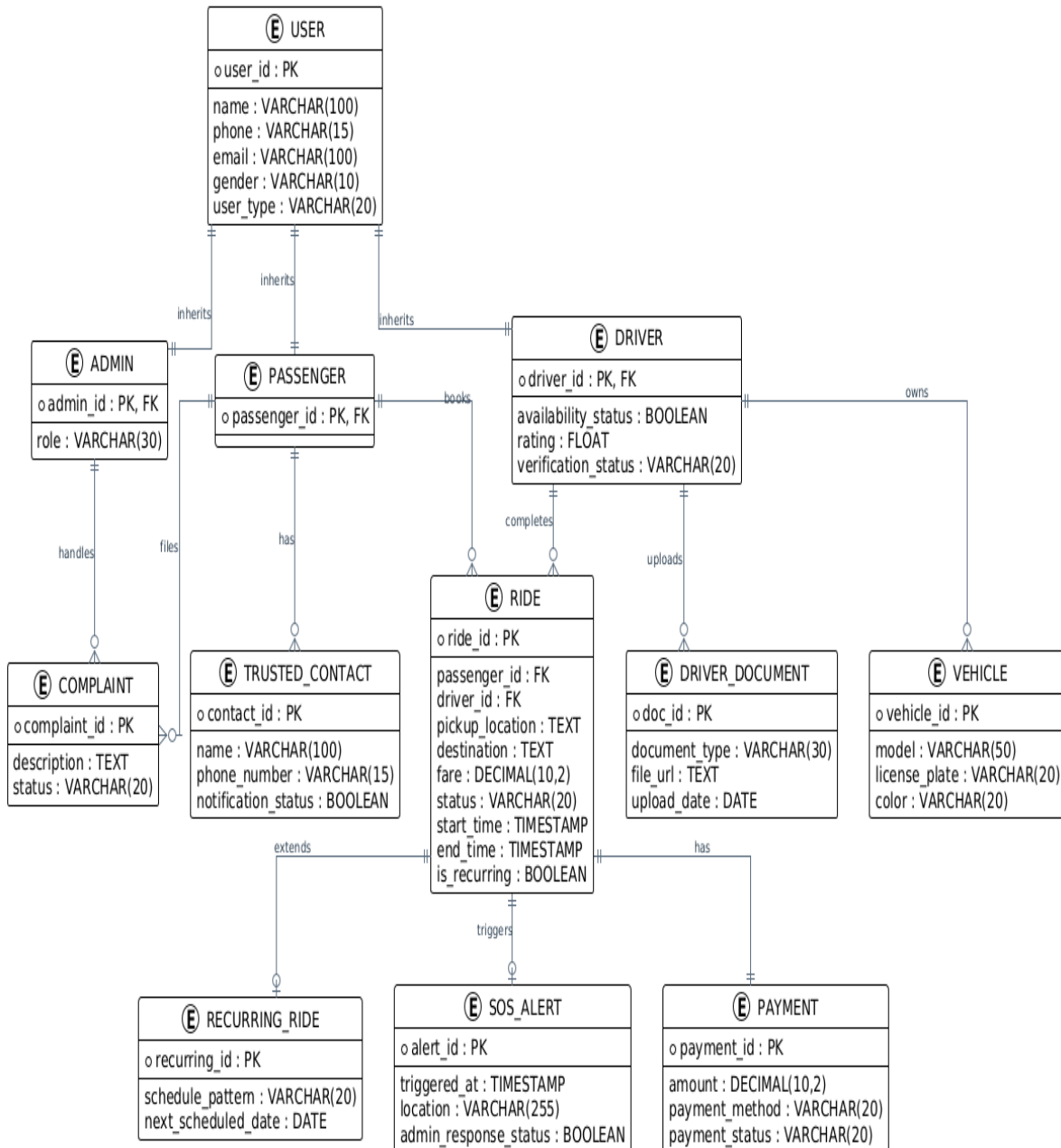
- User: Stores passenger and driver profiles including name, contact, gender, and ride history
- Driver: Stores driver details, verification documents, vehicle information, and rating
- Ride: Stores each ride's data including pickup location, destination, fare, timestamps, and status
- TrustedContact: Stores emergency contact details selected by passengers with name and phone number
- SOSAlert: Logs emergency events with location, timestamp, and admin response status
- Payment: Stores fare details, payment method, amount, and status
- Admin: Stores admin access information and control rights

Relationships between entities:

- Each User (Passenger) can have multiple Rides (one-to-many relationship)
- Each Ride is associated with one Driver (many-to-one relationship)
- Each User (Passenger) can have multiple TrustedContacts (one-to-many relationship)

- Each Ride can trigger one SOSAlert (one-to-one relationship)
- Each Ride has one Payment (one-to-one relationship)

Admin has access to multiple Rides, Users, Drivers, and SOSAlerts (one-to-many relationships)



**Figure 4.2: Entity Relationship Diagram**

An ER-style diagram (Figure 4.2) is used to illustrate how major entities connect. For example, each passenger has multiple rides, each ride is assigned to one driver, each ride can trigger an SOS alert, and each ride has a payment record. This structure ensures clarity in how data is organized and accessed across modules.

## 4.2.4 Data Flow

Data in the SheRides system moves through a well-defined pathway starting from the user interface and ending in secure storage. When a passenger books a ride, the information is captured in the mobile app, validated in the backend, and then saved to the MongoDB database. Drivers later retrieve this stored data through their app to accept rides. Similarly, SOS alert information flows from the passenger's mobile app to the backend, which then sends notifications to admin and trusted contacts while logging the event in the database. These processes collectively ensure smooth coordination between ride booking, SOS alerts, trusted contacts, payments, and admin monitoring. The data flow model also illustrates how information is processed and exchanged between different system modules in a secure and organized manner. Each stage of the data flow ensures that user requests are validated before being processed, reducing the chances of errors and maintaining data consistency.

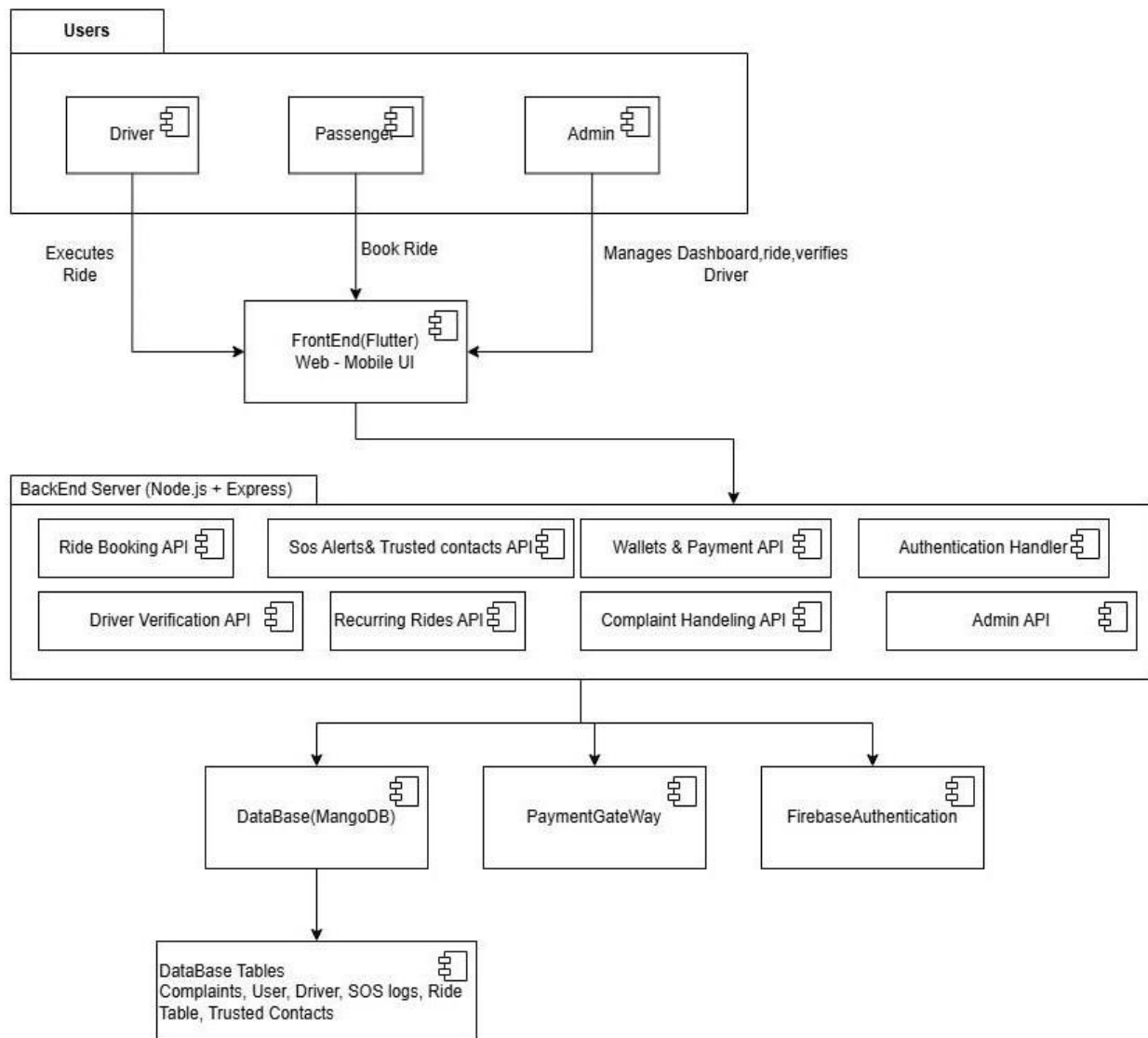


Figure 4.3: Data Flow Diagram

#### 4.2.3.1 Sequence Diagram (Data Flow Across Layers)

The sequence diagram shown in Figure 4.4 explains how data moves between the presentation layer (mobile app), business logic layer (backend), database layer (MongoDB), and external services (Firebase Authentication, Google Maps API, Firebase Cloud Messaging) during major operations such as ride booking, SOS alert triggering, trusted contact sharing, and payment processing.

Main Steps in the Sequence:

- 1.The passenger opens the app and views available ride options.
- 2.The frontend requests nearby drivers from the backend.
- 3.The passenger selects a destination and confirms a ride request.
- 4.The backend assigns the nearest available female driver.
- 5.The driver receives a notification and accepts the ride.
- 6.Real-time tracking starts, and both sides see live location updates.
- 7.During the ride, the passenger can trigger an SOS alert if needed.
- 8.After reaching the destination, the driver ends the trip.
- 9.The backend saves ride details, calculates fare, and updates the passenger’s ride history.
- 10.Confirmation is shown to both passenger and driver.

This sequence diagram makes the interaction between components clear and helps maintain a structured development flow

The sequence diagram also illustrates the order in which different system components communicate to complete each user request successfully. Every interaction follows a well-defined sequence, ensuring that data is validated, processed, and stored correctly before a response is returned to the user. This structured communication minimizes processing errors, maintains data consistency across the system, and enables smooth coordination between the mobile application, backend server, database, and external services. Furthermore, the sequence diagram serves as a valuable reference during implementation, testing, and debugging by clearly defining the responsibilities and message flow between all participating components.

flow of messages exchanged between different system components during each operation.

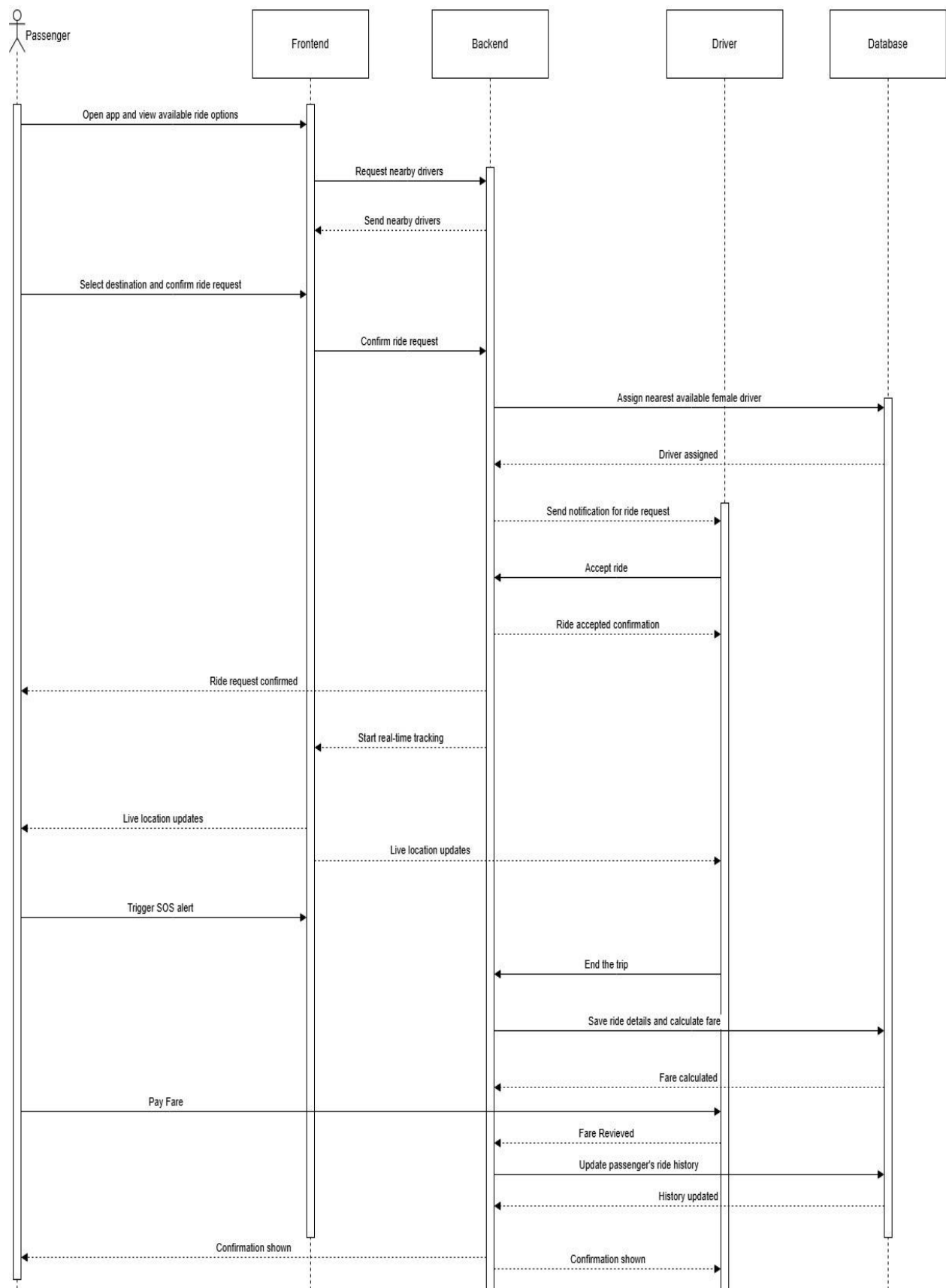


Figure 4.4: Sequence Diagram

#### 4.2.3.2 Activity Diagram (Overall System Workflow)

The activity diagram in Figure 4.5 summarizes the entire workflow from passenger login to ride completion and feedback submission. It illustrates how each action such as booking a ride, triggering SOS alerts, adding trusted contacts, and making payments results in data being created, processed, and stored across various modules of the system. It also demonstrates the decision points and sequence of activities performed by different users

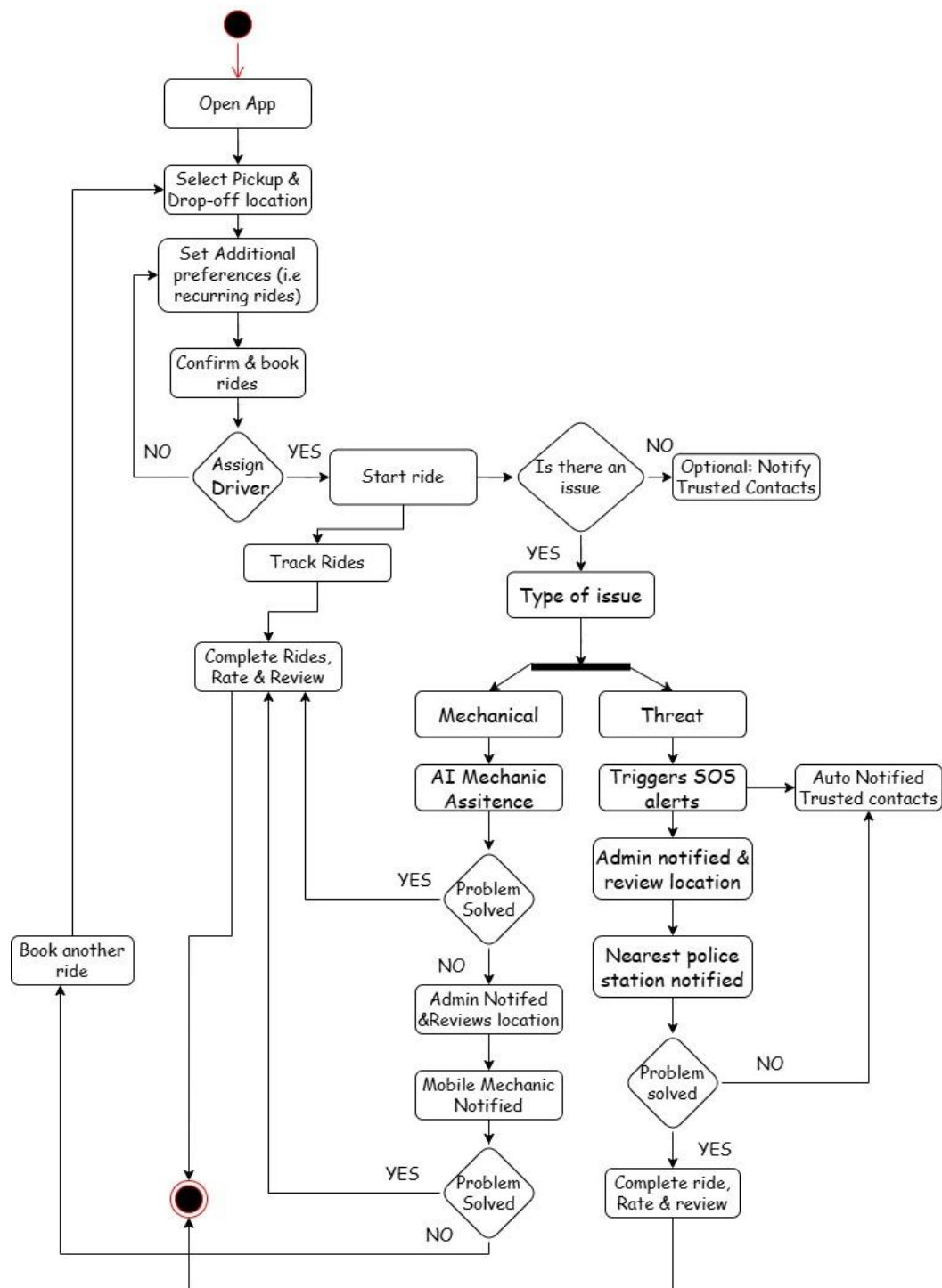


Figure 4.5: Activity Diagram



## **4.3 Process Flow / Representation**

The Process Flow section explains how key operations are carried out in the SheRides system. Even when diagrams are not included, each workflow is described clearly through structured explanations. These flows help readers understand the triggers, steps, and outcomes involved in major system processes.

### **4.3.1 Login and Authentication Process**

The login and authentication process begin when a user enters their credentials through the SheRides mobile app. The system receives the input and verifies it using Firebase Authentication. If the information matches, the system checks the user's role (passenger or driver) to determine the correct dashboard to load. After successful verification, a secure session token is generated, allowing the user to access the system without repeatedly logging in. If the credentials are incorrect, the system rejects the login attempt and prompts the user to try again, ensuring secure access control.

### **4.3.2 Ride Booking Process**

The ride booking process starts when a passenger opens the app and views available ride options. The passenger enters pickup and drop-off locations and confirms the ride request. The backend finds the nearest available female driver and assigns the ride. The driver receives a notification and accepts the ride. Real-time tracking starts, and both passenger and driver see live location updates. The system saves ride details in the database, calculates fare, and updates the passenger's ride history. Confirmation is shown to both passenger and driver.

### **4.3.3 SOS Alert Process**

The SOS alert process begins when a passenger feels unsafe during an active ride and presses the SOS button. The system captures the current live location and creates an SOS entry in the database. Alerts are immediately sent to the admin dashboard and all trusted contacts via push notifications and SMS. The system displays confirmation to the passenger that the SOS has been sent. Admin monitors the situation in real-time and can contact the passenger or emergency services if needed. If the internet is unavailable during SOS trigger, the system automatically retries sending the alert when connection is restored.

### **4.3.4 Driver Verification Process**

The driver verification process starts when a female driver registers and submits identity documents including CNIC, driving license, and vehicle registration. The admin logs into the web portal and reviews the driver application. The admin verifies the documents for authenticity and completeness. If documents are valid and complete, the admin approves the

driver, and the driver can accept ride requests. If documents are incomplete or invalid, the admin rejects the application and requests re-upload. The system notifies the driver of the verification status via push notification and email.

### **4.3.5 Payment Processing Workflow**

The payment processing workflow is triggered when a ride is completed and fare is calculated. The system presents the fare amount to the passenger. The passenger can choose to pay via in-app wallet or cash. If using wallet, the system validates the wallet balance, processes the transaction, and updates the balance. If using cash, the system records that cash payment is expected. A receipt is generated and stored in the payment records. The system updates the transaction logs and adds the earnings to the driver's account. If payment fails, an error message is displayed to the user.

## **4.4 Design Models**

Design models describe the structural and behavioral aspects of the SheRides system through UML representations. Even without images, each model is thoroughly explained to reflect system behavior, module interaction, and architectural decisions.

### **4.4.1 Class Model Description**

The class model defines the main structural elements of the SheRides system by outlining the central classes and their relationships. Classes such as User, Driver, Ride, SOSAlert, TrustedContact, Payment, and Admin contain relevant attributes and operations that describe their behavior within the system.

The User class stores personal details including name, phone number, CNIC, and authentication ID from Firebase. The User class also manages trusted contacts and ride history. The Driver class inherits from User and adds driver-specific attributes including identity documents, vehicle details, availability status, and driver ratings. The Ride class stores all information related to a ride request including pickup location, destination, fare estimate, assigned driver, ride status, timestamps, and tracking data.

The TrustedContact class stores emergency contacts added by a user, with each contact having a name, phone number, and notification settings for ride-sharing or SOS alerts. The SOSAlert class represents an emergency triggered during a ride and includes timestamp, user ID, location, admin response status, and notifications sent to trusted contacts. The Payment class handles payment-related information including fare, method (cash or online), status, and ride association.

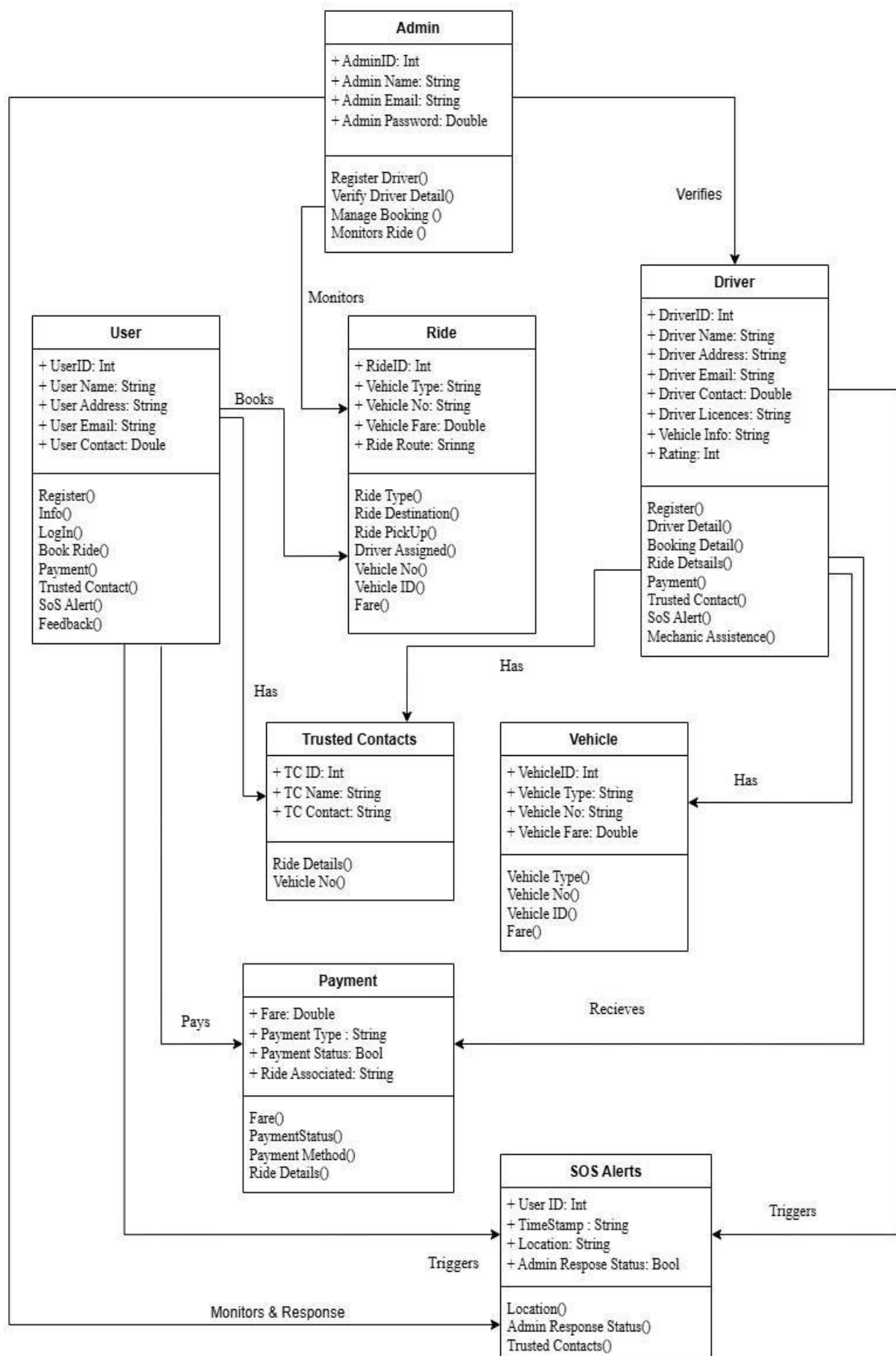


Figure 4.6: Class Diagram

#### **4.4.2 Sequence Model Description**

The sequence model illustrates the communication between system components during important operations. During the ride booking process, the sequence begins with the passenger opening the app and viewing available ride options. The frontend requests nearby drivers from the backend. The passenger selects a destination and confirms the ride request. The backend assigns the nearest available female driver. The driver receives a notification and accepts the ride. Real-time tracking starts, and both sides see live location updates. During the ride, the passenger can trigger an SOS alert if needed. After reaching the destination, the driver ends the trip. The backend saves ride details, calculates fare, and updates the passenger's ride history. Confirmation is shown to both passenger and driver.

#### **4.4.3 State Transition Model Description**

The state transition model explains the system behavior in different states throughout a passenger's journey. The Idle or Browsing State occurs when the user opens the app and explores available options. The Searching for Driver State occurs when the user enters pickup and destination, and the system looks for available drivers. The Driver Assigned State occurs when the system matches the user with the nearest verified female driver. The Ride Active State occurs when the ride has started and live tracking remains active. The SOS Triggered State is optional and occurs if the passenger feels unsafe and presses the SOS button, immediately sending alerts to admin and trusted contacts. The Ride Completed State occurs when the driver ends the trip and fare is calculated automatically. The Payment and Confirmation State occurs when the system logs the payment details and shows a summary. The Ride History State occurs when updated ride entry is stored for future reference.

The state transition model provides a clear representation of how the system responds to different user actions and events during the ride lifecycle. It illustrates the conditions under which the application moves from one state to another, ensuring that each transition follows a logical and controlled process. This model helps developers understand the dynamic behavior of the system, simplifies testing of different scenarios, and ensures that all possible ride states are handled correctly. By defining each state and its corresponding transitions, the model improves system reliability, minimizes unexpected behavior, and supports a smooth and secure user experience throughout the entire journey.

In addition, the state transition model helps identify exceptional situations, such as ride cancellation or emergency alerts, and ensures that the system responds appropriately without affecting other operations. It also provides a clear understanding of the complete ride workflow,

making system implementation, maintenance, and future enhancements more efficient. Overall, the model contributes to a reliable, secure, and user-friendly ride-sharing experience.

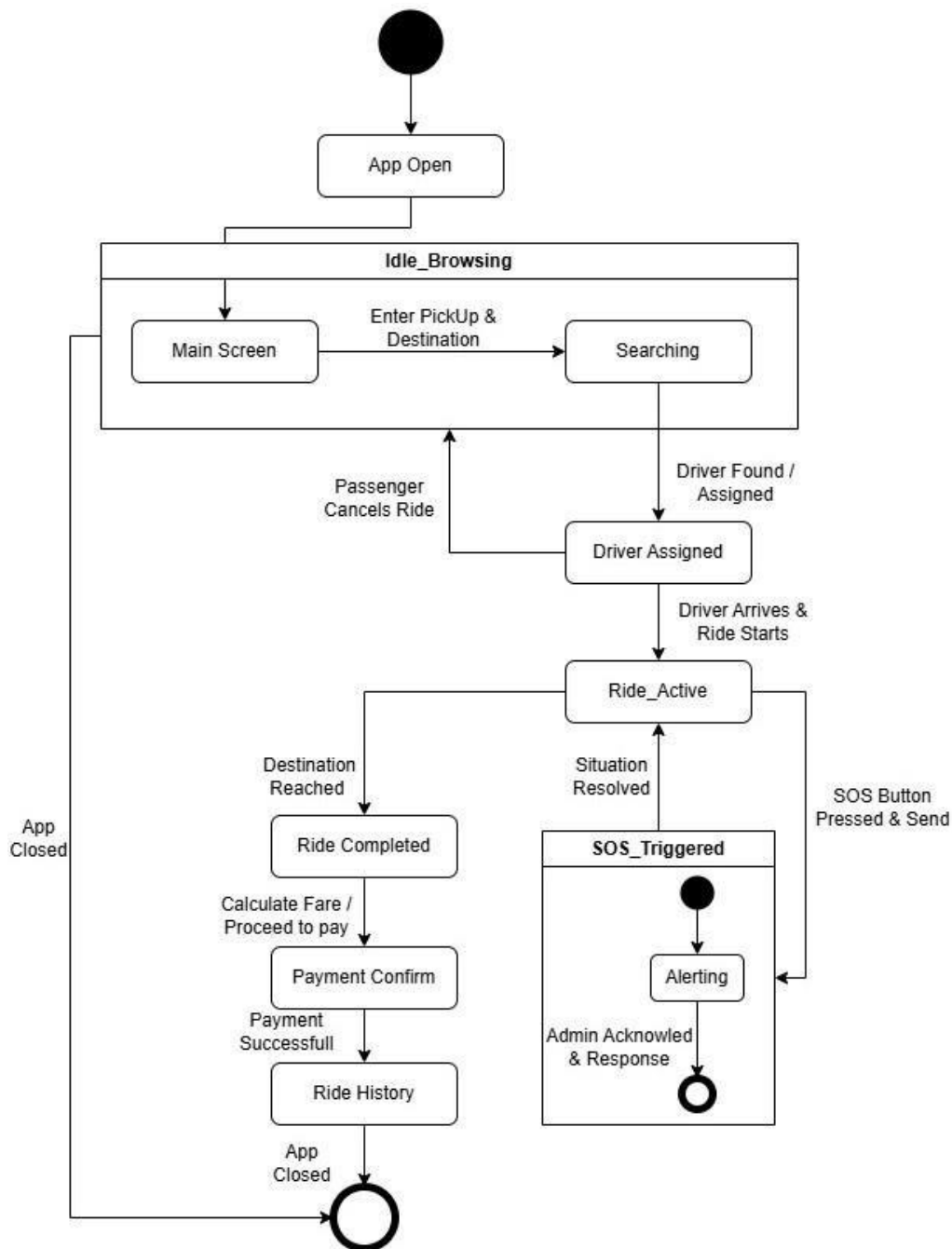


Figure 4.7: State Transition Diagram

#### 4.4.4 Component Model Description

The component model describes how the major modules of the SheRides system are organized and interconnected. Components such as the mobile application (Flutter), backend services (Node.js [8] + Express), authentication service (Firebase), database (MongoDB), admin panel (MERN), and external services (Google Maps API, Firebase Cloud Messaging) operate independently while communicating through secure RESTful APIs.

The User Login and Profile module allows passengers and drivers to register, log in through Firebase Authentication, and manage profile details and verification documents. The Ride Booking System module allows passengers to book instant rides, schedule recurring rides, track drivers in real-time, and view fares before confirming. The Safety and SOS Module provides SOS alerts, emergency contact notifications, nearest police information, and live location sharing during rides. The Trusted Contact Circle module allows passengers to add trusted contacts who receive ride updates and alerts during emergencies.

The Recurring Rides module supports daily commute automation for working women or students by scheduling repeated trips. The Driver Management Module allows drivers to accept or reject rides, start and end rides, view earnings, and manage availability. The Payment and Wallet System supports secure payment handling, transaction logs, and fare summaries. The Admin Panel enables administrators to verify drivers, monitor rides live, handle complaints, and manage system operations. The Customer Support module allows users to submit complaints, get help, and receive responses from the admin team.

## **Chapter 5**

# **IMPLEMENTATION**

## 5. Implementation

SheRides system was implemented at the technical level. It does not include source code but provides descriptions of the algorithms, external APIs, and user interface decisions that were used to build the system. The goal of this chapter is to show how the system's design was transformed into a functional software solution while maintaining efficiency, accuracy, and security for women passengers.

### 5.1 Algorithms

This section describes the main algorithms used in the SheRides system. Each algorithm represents the internal logic behind system features and explains how different functions operate. These descriptions are presented in pseudocode to clearly communicate the workflow behind major modules.

#### 5.1.1 User Authentication Algorithm

The authentication algorithm ensures that only legitimate users can access the SheRides system. It protects sensitive passenger and driver data by verifying credentials and granting role-based access. This algorithm is crucial for maintaining system security and preventing unauthorized access. Only verified female users are allowed to register as passengers or drivers.

Steps:

1. User enters email and password through Flutter mobile app
2. System sends credentials to Firebase Authentication
3. Firebase verifies the user's identity
4. If credentials match:
  - a. Generate a secure authentication token
  - b. Identify user role (Passenger or Driver)
  - c. Redirect user to the corresponding dashboard
  - d. Load user profile data from MongoDB
5. If credentials do not match:
  - a. Record the failed login attempt



- b. If failed attempts exceed allowed limit, temporarily lock the account
- c. Display error message to user

Pseudocode:

BEGIN AUTHENTICATION

INPUT email, password

CALL FirebaseAuth.signInWithEmailAndPassword(email, password)

IF response.success THEN

    GET userRole from database

    IF userRole = "Passenger" THEN

        REDIRECT to PassengerDashboard

    ELSE IF userRole = "Driver" THEN

        REDIRECT to DriverDashboard

    END IF

    RETURN "Login Successful"

ELSE

    INCREMENT failedAttempts

    IF failedAttempts >= 5 THEN

        LOCK account for 15 minutes

    END IF

    RETURN "Invalid Credentials"

END IF

END

### **5.1.2 Ride Booking Algorithm**

This algorithm allows passengers to book rides efficiently while preventing conflicts. It ensures that available female drivers are assigned accurately and that notifications are sent to both parties, improving overall workflow and passenger experience.

Steps:

1. Passenger enters pickup and destination locations
2. System calculates estimated fare
3. Passenger confirms ride request
4. System finds nearest available verified female drivers within radius
5. If driver found:
  - a. Send ride request to driver
  - b. Driver accepts or rejects within time limit
  - c. If accepted, confirm ride and start real-time tracking
  - d. Notify both passenger and driver
  - e. Save ride details in database
6. If no driver available:
  - a. Display "No Drivers Available" message
  - b. Suggest scheduling a recurring ride

Pseudocode:

BEGIN RIDE\_BOOKING

INPUT pickup, destination, passengerId

CALCULATE fare based on distance and time

DISPLAY fare to passenger

IF passenger confirms THEN

```

FIND nearby available female drivers

IF driver found THEN

    SEND ride request to driver

    WAIT for driver response (30 seconds)

    IF driver accepts THEN

        ASSIGN driver to ride

        UPDATE ride status to "accepted"

        START real-time tracking

        SEND notification to passenger and driver

        SAVE ride details to database

        RETURN "Ride Confirmed"

    ELSE

        RETURN "Driver declined"

    END IF

ELSE

    RETURN "No Drivers Available"

END IF

ELSE

    RETURN "Booking Cancelled"

END IF

END

```

### 5.1.3 SOS Alert Algorithm

The SOS alert algorithm allows passengers to trigger emergency alerts during rides. It ensures

that alerts are only sent by authorized passengers during active rides and that they are properly linked to the ride record and sent to admin and trusted contacts for immediate response.

Steps:

1. Passenger presses SOS button during active ride
2. System displays confirmation dialog
3. Passenger confirms SOS trigger
4. System captures current live location
5. Check if ride is active and passenger is authorized
6. If authorized and all data is valid:
  - a. Create SOS entry in database with timestamp and location
  - b. Send SOS notification to admin dashboard
  - c. Send SMS and push notifications to all trusted contacts
  - d. Display "SOS Sent" confirmation to passenger
7. If unauthorized or ride not active:
  - a. Display error message
  - b. Log unauthorized attempt

Pseudocode:

```
BEGIN SOS_ALERT
```

```
    INPUT passengerId, rideId
```

```
    CHECK if ride is active
```

```
    IF ride.active = true THEN
```

```
        CAPTURE current location using GPS
```

```
        CREATE SOS record in database
```

```

SET status = "active"

NOTIFY admin via dashboard

FOR each trustedContact in passenger.contacts

    SEND SMS and push notification with location

END FOR

DISPLAY "SOS Sent Successfully" to passenger

RETURN "Alert Sent"

ELSE

    RETURN "Cannot trigger SOS - No active ride"

END IF

END

```

#### **5.1.4 Driver Verification Algorithm**

This algorithm manages the verification process for female drivers registering on the SheRides platform. It ensures that all documents are validated by the admin before drivers can accept rides, maintaining platform trust and passenger safety.

Steps:

1. Driver submits registration form with personal details
2. Driver uploads required documents (CNIC, license, vehicle registration)
3. System stores documents and creates pending verification request
4. Admin reviews documents through admin pane
5. If documents are valid and complete:
  - a. Update driver verification status to "approved"
  - b. Driver can now accept ride requests
  - c. Send approval notification to driver

6. If documents are invalid or incomplete:
  - a. Update driver verification status to "rejected"
  - b. Send rejection notification with reason
  - c. Request driver to re-upload correct documents

Pseudocode:

BEGIN DRIVER\_VERIFICATION

INPUT driverId, documents[]

SAVE documents to cloud storage

CREATE verification request in database

SET status = "pending"

NOTIFY admin of new verification request

WAIT for admin response

IF admin approves THEN

UPDATE driver.status = "verified"

SEND approval notification to driver

RETURN "Driver Verified"

ELSE

UPDATE driver.status = "rejected"

SEND rejection notification with reason

RETURN "Verification Failed"

END IF

END

### 5.1.5 Payment Processing Algorithm

This algorithm manages the secure processing of payments for rides. It ensures that all transactions are validated and that successful payments are recorded while failures are logged for review. The system supports both wallet-based and cash payments.

Steps:

1. Ride is completed by driver
2. System calculates final fare based on distance and time
3. System displays fare to passenger
4. Passenger selects payment method (wallet or cash)
5. If payment method is wallet:
  - a. Check wallet balance
  - b. If sufficient balance, deduct amount
  - c. Update wallet balance in database
  - d. Generate receipt
6. If payment method is cash:
  - a. Record that cash payment is expected
  - b. Mark payment as "pending"
  - c. Driver confirms cash receipt
7. If payment is successful:
  - a. Store payment details in database
  - b. Update driver earnings
  - c. Generate and provide receipt to passenger
8. If payment fails:
  - a. Notify passenger of transaction failure

b. Log failed transaction for administrative review

Pseudocode:

BEGIN PAYMENT\_PROCESSING

INPUT rideId, amount, paymentMethod

GET ride details from database

IF paymentMethod = "wallet" THEN

CHECK passenger.walletBalance >= amount

IF sufficient THEN

DEDUCT amount from passenger.wallet

ADD amount to driver.earnings

UPDATE payment.status = "completed"

GENERATE receipt

RETURN "Payment Successful"

ELSE

RETURN "Insufficient Balance"

END IF

ELSE IF paymentMethod = "cash" THEN

UPDATE payment.status = "pending"

MARK as cash payment expected

AFTER driver confirms:

UPDATE payment.status = "completed"

GENERATE receipt

RETURN "Cash Payment Recorded"



END IF

END

## 5.2 External APIs

The SheRides system leverages several external APIs to provide advanced functionality beyond the core technology stack capabilities. These APIs enhance features such as secure user authentication, real-time location tracking, push notifications, and cloud database services, improving overall system efficiency and user experience.

In the SheRides architecture, APIs are primarily consumed by the backend (Node.js/Express) to handle requests, validate data, and interact with the database (MongoDB), while the frontend (Flutter) [6] dynamically renders responses and provides interactive features to passengers and drivers.

*Table 5.1: Details of APIs Used in the Project*

| Name of API                        | Description of API                                                                 | Purpose of Usage                                                                                                                   | Function/Class Name in which Used                       |
|------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <b>Firebase Authentication [5]</b> | Google's authentication service for secure user login and identity management      | Manage passenger and driver registration, login, password management, and role-based access control                                | AuthService (Flutter), AuthController (Node.js)         |
| <b>Google Maps API [10]</b>        | Location and mapping services provider offering real-time navigation and geocoding | Real-time ride tracking, route calculation between pickup and destination, distance-based fare estimation, driver location display | RideTrackingService (Flutter), RideController (Node.js) |

|                                        |                                                               |                                                                                                                                                      |                                                                 |
|----------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Firestore Cloud Messaging (FCM)</b> | Push notification service from Google for real-time messaging | Send SOS alerts to admin and trusted contacts during emergencies, ride status notifications (accepted, started, completed), driver assignment alerts | NotificationService (Flutter), NotificationController (Node.js) |
| <b>Firestore Realtime Database</b>     | Real-time data synchronization service from Google            | Live location updates during active rides, real-time ride tracking for passengers and drivers, driver location streaming                             | RideTrackingService (Flutter), RideController (Node.js)         |
| <b>MongoDB Atlas</b>                   | Cloud-based MongoDB database service with automated backups   | Store user profiles, ride records, SOS logs, trusted contacts, driver documents.                                                                     | <b>DatabaseService (Node.js), RideRepository (Node.js)</b>      |

## 5.3 User Interface

The user interface (UI) of the SheRides system is designed to provide a seamless, intuitive, and user-friendly experience for passengers, drivers, and administrators. It focuses on clarity, accessibility, and safety, ensuring that users can navigate the system with minimal learning effort. Consistent layout patterns, well-structured menus, and clearly labeled icons help users perform tasks such as booking rides, triggering SOS alerts, adding trusted contacts, and monitoring system activity.

The design follows modern UI principles with responsive layouts, allowing smooth usage across devices including Android and iOS mobile phones for passengers and drivers, and

desktop browsers for administrators. Visual hierarchies, appropriate color schemes (soft tones for comfort), and simple forms enhance usability, while error messages, confirmation dialogs, and interactive feedback ensure better user control and guidance. Buttons for SOS and trusted contact sharing are easily accessible from every ride screen to ensure quick emergency response.

Overall, the UI is developed to support fast task completion, reduce user errors, improve passenger safety, and enhance the overall experience of women-only ride-sharing.

### 5.3.1 Admin Panel

The Admin Panel shown in Figure 5.1 of the SheRides system, which acts as the main interface through which administrators access all system features. The layout is designed to be clean and professional, ensuring that admins can quickly navigate to key options such as driver verification, live ride monitoring, SOS alert handling, complaint management, and user management. The dashboard displays overview cards for Active Rides, Total Drivers, Complaints, and Daily Revenue. A live map widget tracks driver and passenger activity in real-time.

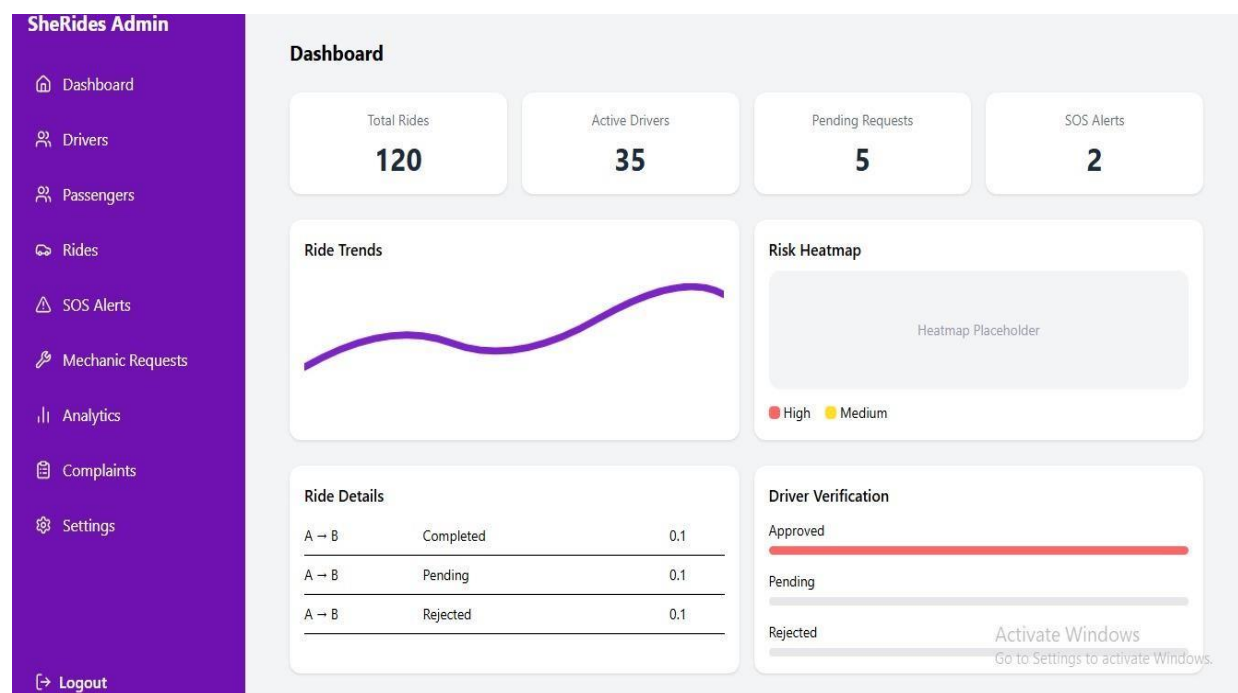
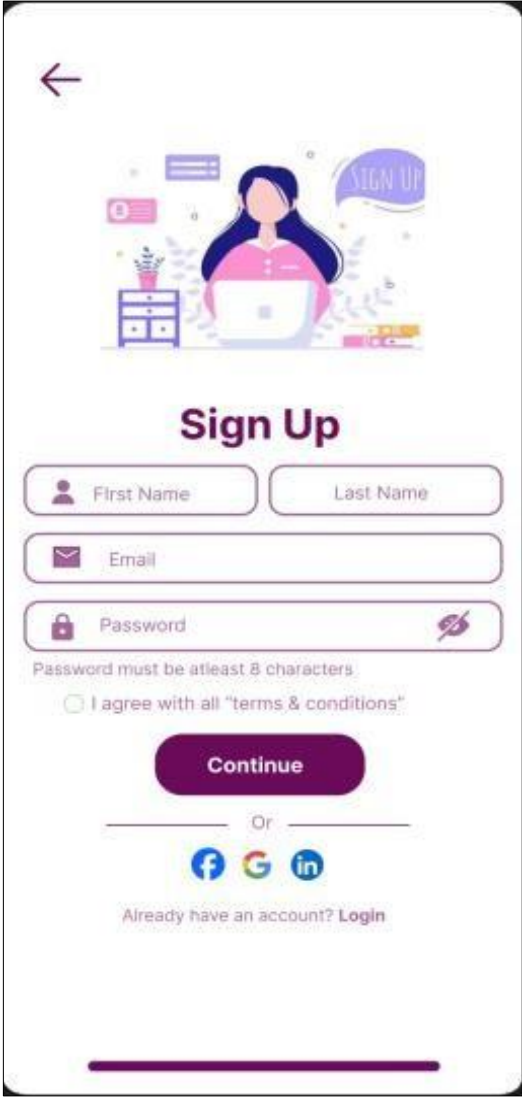


Figure 5.1: Admin Panel

### 5.3.2 Signup Screen

Figure 5.2 displays the Signup Screen within the SheRides Passenger Application, where new users can create their accounts. This screen allows users to enter their name, email, phone number, and password. The interface includes an eye icon to toggle password visibility, Terms

and Conditions checkbox, and social login icons for quick registration. The design is simple and easy to understand, enabling users to register quickly while adhering to the women-only policy of the platform.

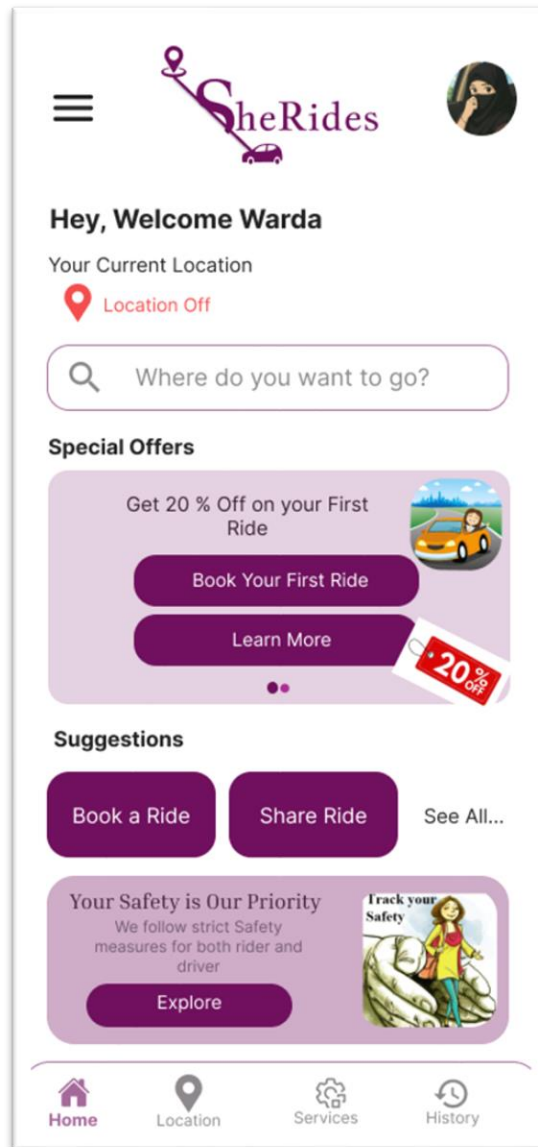
The image shows a mobile application screen for user sign-up. At the top left is a back arrow. Below it is an illustration of a woman with long dark hair sitting at a desk with a laptop, surrounded by floating icons like a speech bubble with 'SIGN UP', a lightbulb, and a calendar. The title 'Sign Up' is centered in a bold, dark font. Below the title are four input fields: 'First Name' and 'Last Name' (each with a person icon), 'Email' (with an envelope icon), and 'Password' (with a lock icon and a toggle for visibility). Below the password field is a text requirement: 'Password must be atleast 8 characters'. Underneath is a checkbox labeled 'I agree with all "terms & conditions"'. A large, rounded 'Continue' button is centered below the checkbox. Below the button is the word 'Or' flanked by horizontal lines, followed by three social media icons: Facebook, Google, and LinkedIn. At the bottom, there is a link that says 'Already have an account? Login'. A thick black horizontal bar is at the very bottom of the screen.

*Figure 5.2: User Sign Up*

### 5.3.3 Home Screen (Passenger)

The Home Screen of the Passenger Application shown in Figure 5.3 provides passengers with quick access to key features such as ride booking, location status, special offers, and bottom navigation to Location, Services, and History sections. The interface is designed to be clean and organized, allowing passengers to efficiently book rides and navigate the system with ease. The menu icon opens side options and settings, while the search bar begins the booking flow.

By displaying essential features on a single screen, it minimizes navigation time and allows passengers to quickly access the services they need. This design improves usability, accessibility, and overall user satisfaction.



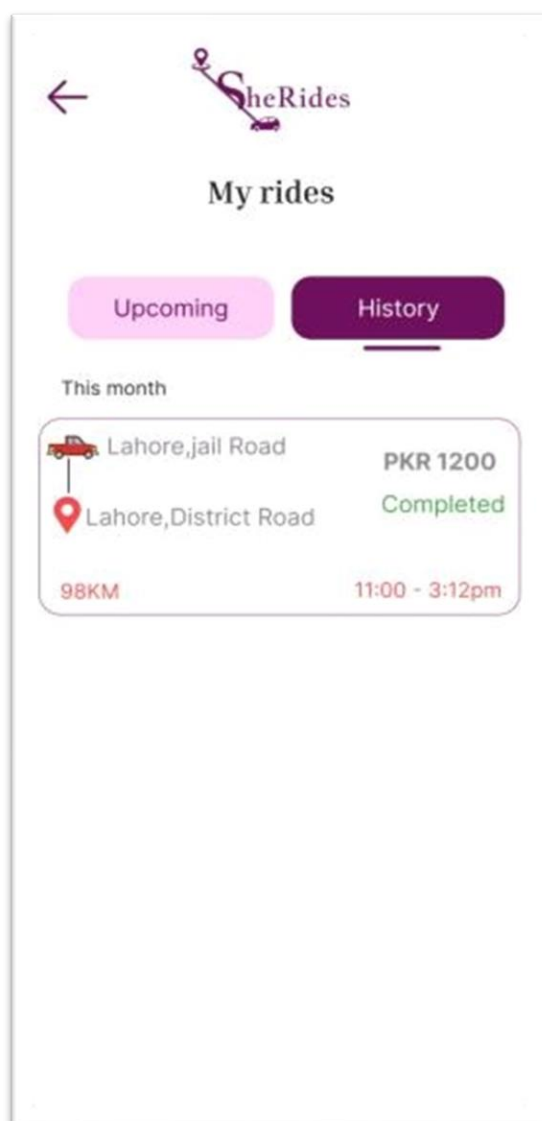
*Figure 5.3: Home Page*

### 5.3.4 Ride History Screen (Passenger)

Figure 5.4 shows the Ride History Screen within the Passenger Application, where passengers can view their past and upcoming rides. The screen includes tabs for Upcoming and History, ride cards showing location, fare, status, and labels for date, time, distance, and status. This screen helps passengers track their travel history and review completed trips.

The Ride History Screen also enables passengers to keep a complete record of their travel activities, making it easier to review previous journeys and monitor upcoming bookings. Its well-organized layout improves accessibility by presenting ride information in a clear and structured manner. This enhances user convenience and provides a seamless experience for managing ride records. Additionally, the screen allows passengers to quickly identify the status of each ride, whether it is upcoming or completed. This feature improves transparency, helps

users maintain a record of their journeys, and enhances the overall usability of the application.



*Figure 5.4: History Page*

### 5.3.5 Trip Details Screen (Driver)

The Trip Details Screen of the Driver Application in Figure 5.5 provides drivers with quick access to route preview, fare breakdown, trip information (time, date, service type), and receipt section. The interface includes a map view showing the full route, helping drivers navigate efficiently to passenger locations. The screen also presents essential trip information in a clear and well-structured format, allowing drivers to access important details without confusion. By combining route visualization with trip and payment information on a single screen, it reduces navigation effort and improves operational efficiency. This user-friendly design enables drivers to manage rides more effectively.

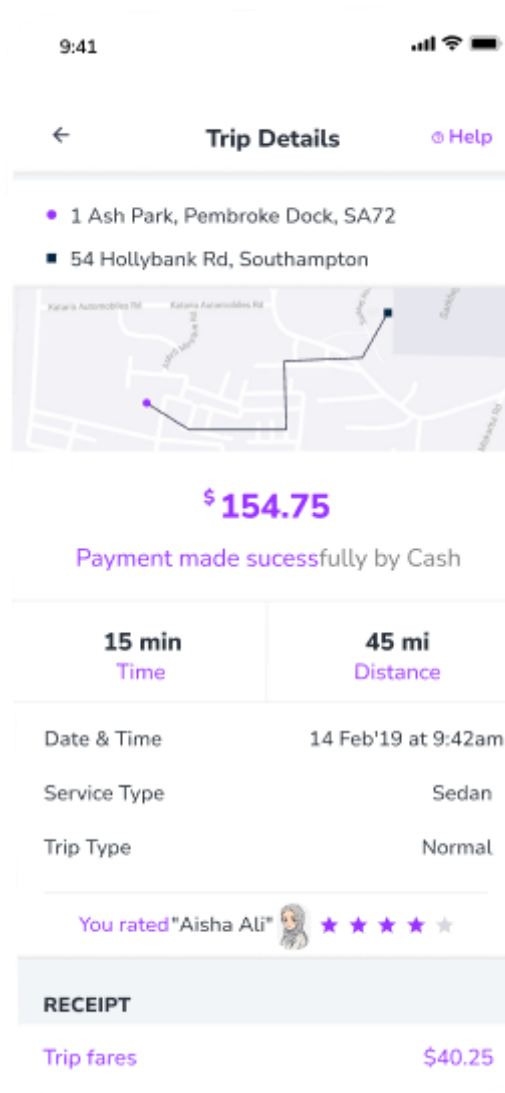
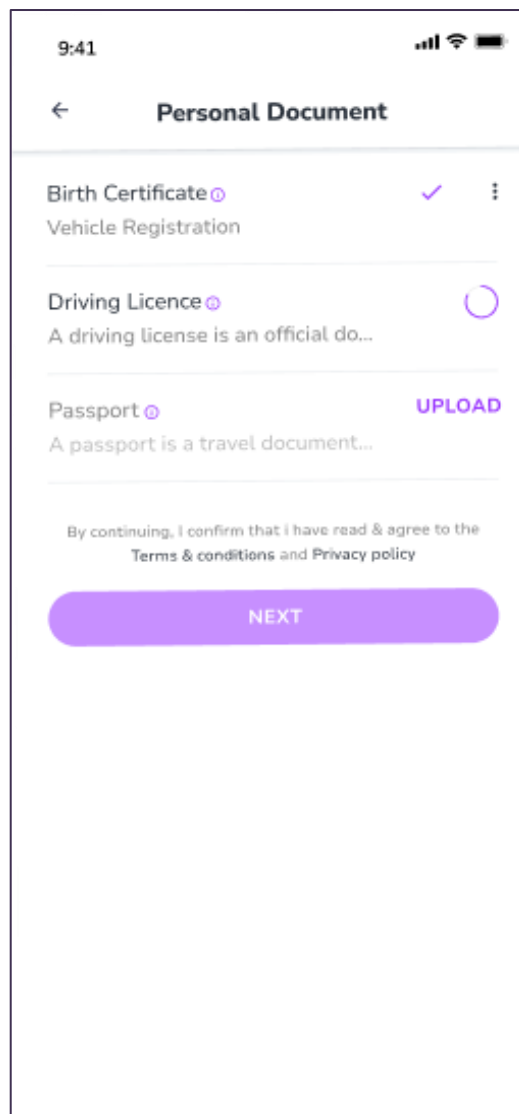


Figure 5.5: Driver Detail Page

### 5.3.6 Document Upload Screen (Driver)

Figure 5.6 displays the Document Upload Screen within the Driver Application, where drivers can upload their verification documents including CNIC, Driving License, Vehicle Registration, and Passport. The screen includes status indicators (tick, pending, upload link) showing the verification progress of each document. This feature ensures that only verified female drivers can operate on the platform. The document upload process improves the security and credibility of the SheRides platform by ensuring that all required documents are submitted before driver approval. It also allows drivers to monitor the verification status of each document, making the registration process more transparent and user-friendly. This verification mechanism helps maintain passenger trust and enhances the overall safety of the ride-sharing service. Additionally, the validation of driver documents helps prevent unauthorized registrations and ensures compliance with the platform's safety policies. This process contributes to a secure and trustworthy environment for both passengers and drivers by

maintaining accurate verification records.



*Figure 5.6: Document Page*

### 5.3.7 Earnings Screen (Driver)

The Earnings Screen of the Driver Application in Figure 5.7 shows drivers their earnings trend through a bar graph (daily or weekly), trip summary statistics (total trips and distance), fee breakdown (deductions, bonuses, service charges), and total earnings. This screen helps drivers track their income and performance on the platform. The Earnings Screen provides drivers with a clear overview of their financial performance and ride activity over a selected period. By presenting earnings, trip statistics, and deductions in a well-organized format, it enables drivers to monitor their progress and manage their income effectively. The graphical representation makes performance trends easier to understand, while the detailed summary improves transparency and helps drivers make informed decisions regarding their work. the screen allows drivers to review their completed trips and associated earnings in one place.



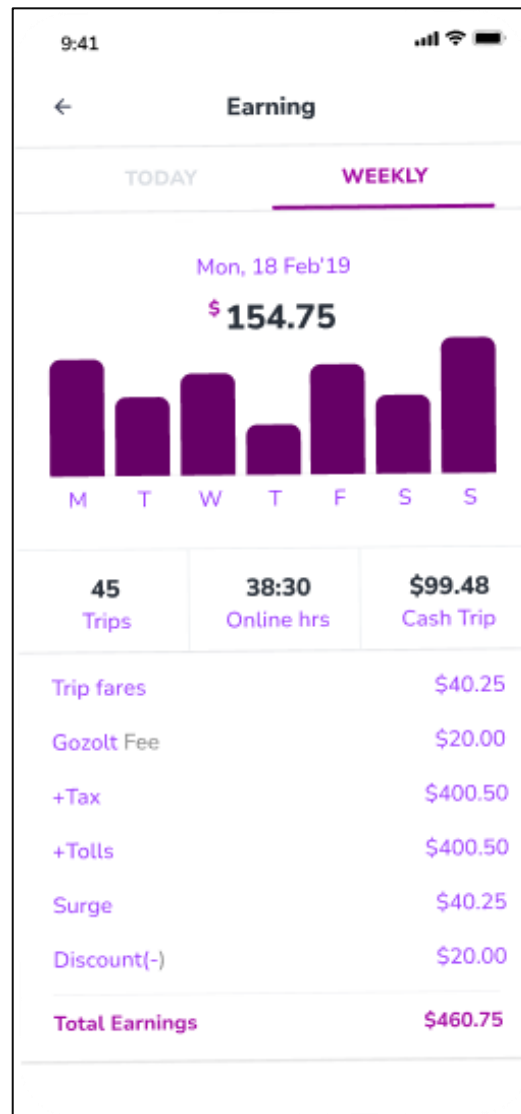


Figure 5.7: Earning Page

## 5.4 Chapter Summary

The Implementation chapter explains how the SheRides system was built by transforming the designed architecture into a fully working solution. It describes the core algorithms such as user authentication, ride booking, SOS alert, driver verification, and payment processing that power major system features.

The chapter also highlights the external APIs used for authentication (Firebase Authentication), location tracking (Google Maps API), push notifications (Firebase Cloud Messaging), real-time tracking (Firebase Realtime Database), and cloud database (MongoDB Atlas), along with an overview of the user interface design for passengers, drivers, and administrators.

The UI section presents key screens including the Admin Panel, Signup Screen, Passenger Home Screen, Ride History Screen, Driver Trip Details Screen, Document Upload Screen.

**Chapter 06**

**TESTING and EVALUATION**

## **6. Testing and Evaluation**

This chapter demonstrates that the SheRides system functions correctly, meets the specified requirements, and provides a reliable and secure experience for women passengers and drivers. Testing has been performed using manual approaches, including unit testing, functional testing, and integration testing, to verify that each module works as intended and that the system operates smoothly as a whole. Manual testing helps identify errors that may not be visible during normal use and ensures that all functional requirements are met before deployment.

In addition to manual testing, automated testing has been applied to improve efficiency and accuracy. Automated tools are used to validate key system functionalities, such as user authentication, ride booking, SOS alert triggering, and payment processing, ensuring that they work correctly under different scenarios. By combining manual and automated testing, the SheRides system has been thoroughly evaluated for reliability, usability, security, and performance, confirming its readiness for real-world use.

### **6.1 Manual Testing**

Manual testing is a process where a tester interacts directly with the SheRides system to verify that all functionalities are working correctly. Unlike automated testing, manual testing relies on human observation to identify errors, usability issues, and unexpected behavior. It is a critical step in ensuring that the system meets its functional and non-functional requirements and provides a smooth user experience. Manual testing allows testers to simulate real-world usage scenarios, making it easier to detect hidden errors that automated tests may miss.

Manual testing in the SheRides system is carried out in multiple stages, including system testing, unit testing, functional testing, and integration testing, before deployment. This ensures that each module works correctly on its own and in combination with other modules, providing a reliable and user-friendly system for women passengers and female drivers.

#### **6.1.1 System Testing**

System testing is a high-level evaluation conducted after the complete development of the SheRides system. Its primary objective is to verify that the system functions as intended and meets all the specified requirements. Unlike testing individual modules, system testing focuses on the system as a whole, allowing the detection of errors and inconsistencies that might not be evident during the development phase. This ensures that the system operates correctly under real-world conditions for all user types including passengers, drivers, and administrators.

##### **6.1.1.1 Purpose of System Testing**

The purpose of system testing is to confirm that the SheRides system meets functional requirements (FR-1 to FR-8), performs reliably, and is ready for deployment. It helps in detecting hidden errors and inconsistencies in workflows such as ride booking, SOS alert triggering, trusted contact sharing, and payment processing. System testing also validates that the overall system is stable, secure, and provides a seamless experience for end-users. This stage is critical to guarantee that the software is fully operational before being released for use.

#### **6.1.1.2 Types of System Testing**

System testing incorporates several types of evaluation to comprehensively assess the software. Unit testing focuses on individual modules or components, ensuring that they function correctly in isolation. Functional testing evaluates the features of the system to ensure they perform according to the design specifications and meet user requirements. Integration testing checks that multiple modules work together correctly, verifying the flow of data between components such as ride booking to SOS alerts, or driver verification to ride assignment, confirming that combined functionalities operate as intended.

#### **6.1.1.3 Importance of Manual System Testing**

Manual system testing is performed carefully to validate the complete system. It ensures that the SheRides system is reliable, secure, and user-friendly before deployment. By thoroughly testing each module and its interactions, potential errors are detected early, which improves the quality, performance, and overall reliability of the system. This approach provides confidence that the system will function effectively in real-world scenarios for women passengers seeking safe transportation.

### **6.1.2 Unit Testing**

Unit testing focuses on verifying the individual modules of the SheRides system to ensure that each part functions correctly on its own. It is usually the first step in manual testing and forms the foundation for functional and integration testing.

Unit tests involve providing specific input values to a module or function and comparing the actual output with the expected result. Any discrepancies are noted and fixed before moving on to higher-level tests.

In the SheRides system, unit testing was performed on core modules such as user authentication, ride booking, SOS alerts, trusted contacts, payment processing, and driver verification. Testing each module independently helped identify and resolve errors at an early stage of development. This improved the reliability of individual components and ensured that every module performed as expected before being integrated into the complete system.

**Table 6.1: Unit Testing**

| Test ID | Module              | Test Case                                | Expected Result                          | Status |
|---------|---------------------|------------------------------------------|------------------------------------------|--------|
| UT-01   | Authentication      | Passenger login with correct credentials | Login successful, dashboard loaded       | Pass   |
| UT-02   | Authentication      | Driver login with correct credentials    | Login successful, dashboard loaded       | Pass   |
| UT-03   | Ride Booking        | Book ride with available driver          | Driver assigned, tracking started        | Pass   |
| UT-04   | Ride Booking        | Book ride with no drivers available      | "No Drivers Available" message           | Pass   |
| UT-05   | SOS Alert           | Trigger SOS during active ride           | Alert sent to admin and trusted contacts | Pass   |
| UT-06   | SOS Alert           | Trigger SOS without active ride          | Error message displayed                  | Pass   |
| UT-07   | Trusted Contact     | Add family member as trusted contact     | Contact added successfully               | Pass   |
| UT-08   | Driver Verification | Upload verification documents            | Documents submitted for verification     | Pass   |
| UT-09   | Payment             | Process wallet payment                   | Payment completed, receipt generated     | Pass   |
| UT-10   | Admin               | Verify driver application                | Driver status updated to verified        | Pass   |

### 6.1.3 Functional Testing

Functional testing is conducted after unit testing and focuses on verifying that the SheRides system's features work according to the specified requirements (FR-1 to FR-8). The main

purpose of functional testing is to ensure that each function performs correctly in real-world scenarios and meets the design and user expectations.

Common examples of functional testing in the SheRides system include testing SOS alerts during active rides, testing ride booking with available drivers, and testing driver verification by admin as described in Table 6.2. These tests ensure that after performing actions, the correct responses are displayed along with appropriate notifications for passengers, drivers, and administrators. Functional testing helps identify issues related to safety workflows, user permissions, and user interface behavior.

**Table 6.2: Functional Testing**

| Test ID | Module              | Test Case                          | Expected Result                         | Status |
|---------|---------------------|------------------------------------|-----------------------------------------|--------|
| FT-01   | Authentication      | Login as Passenger                 | Passenger dashboard loaded              | Pass   |
| FT-02   | Authentication      | Login as Driver                    | Driver dashboard loaded                 | Pass   |
| FT-03   | Authentication      | Login as Admin                     | Admin panel loaded                      | Pass   |
| FT-04   | Ride Booking        | Request ride with available driver | Ride confirmed, tracking started        | Pass   |
| FT-05   | SOS Alert           | Trigger SOS during active ride     | Alert sent within 5 seconds             | Pass   |
| FT-06   | Trusted Contact     | Add trusted contact                | Contact added, location sharing enabled | Pass   |
| FT-07   | Driver Verification | Submit driver documents            | Admin receives verification request     | Pass   |
| FT-08   | Admin               | Approve driver verification        | Driver status changed to verified       | Pass   |

| Test ID | Module         | Test Case               | Expected Result                     | Status |
|---------|----------------|-------------------------|-------------------------------------|--------|
| FT-09   | Recurring Ride | Schedule recurring ride | Ride scheduled successfully         | Pass   |
| FT-10   | Payment        | Process wallet payment  | Payment deducted, receipt generated | Pass   |

### 6.1.4 Integration Testing

Integration testing ensures that multiple modules of the SheRides system work together correctly and that data flows seamlessly between them. This type of testing is conducted after unit and functional testing and focuses on verifying the interactions between modules to identify errors that might occur when components are combined.

Example integration test scenarios include logging in as passenger and booking a ride, triggering SOS during an active ride and verifying trusted contacts receive alerts, driver registration with admin verification, and ride completion with payment processing as described in Table 6.3. Integration testing helps ensure that combined features operate as intended and that the system provides a smooth user experience.

**Table 6.3: Integration Testing**

| Test ID | Modules Involved              | Test Case                    | Expected Result                      | Status |
|---------|-------------------------------|------------------------------|--------------------------------------|--------|
| IT-01   | Authentication + Ride Booking | Login and book ride          | Ride booked, driver assigned         | Pass   |
| IT-02   | Ride Booking + SOS Alert      | Book ride then trigger SOS   | Alert sent to admin and contacts     | Pass   |
| IT-03   | SOS Alert + Trusted Contact   | Trigger SOS, verify contacts | All trusted contacts notified        | Pass   |
| IT-04   | Driver Registration + Admin   | Register, verify, go online  | Driver verified and can accept rides | Pass   |

| Test ID | Modules Involved         | Test Case                         | Expected Result                     | Status |
|---------|--------------------------|-----------------------------------|-------------------------------------|--------|
| IT-05   | Ride + Payment + History | Complete ride, pay, check history | History updated, earnings added     | Pass   |
| IT-06   | Recurring Ride + Booking | Schedule recurring ride           | Ride auto-created at scheduled time | Pass   |
| IT-07   | Admin Panel + SOS Alert  | Admin monitors live SOS           | SOS alert visible on dashboard      | Pass   |

## 6.2 Automated Testing

Automated testing involves using software tools to test the SheRides system instead of performing manual interactions. These tools help verify system functionalities quickly, accurately, and repeatedly. Automated testing is particularly useful for regression testing, performance checks, and verifying repetitive tasks.

The tools used for automated testing as described in Table 6.4 in the SheRides project include Flutter Test for mobile frontend unit and widget testing, Postman for backend API testing, and Firebase Test Lab for Android compatibility testing. Each tool is applied to specific test cases, functional requirements (FR), or non-functional requirements (NFR), and the results are documented to confirm system reliability.

**Table 6.4: Automated Testing**

| Tool              | Applied On  | Tested Features                                       | Results    |
|-------------------|-------------|-------------------------------------------------------|------------|
| Flutter Test      | Mobile App  | Login, ride booking, SOS button, UI widgets           | Pass (95%) |
| Postman           | REST APIs   | User registration, ride booking, SOS logging, payment | Pass       |
| Firebase Test Lab | Android App | Compatibility on Android 9-13                         | Pass       |



## 6.3 Testing Summary

The Testing and Evaluation chapter verifies that the SheRides system works correctly, securely, and meets all requirements specified in the SRS document. Both manual and automated testing were performed, including unit, functional, system, and integration tests, to ensure each module works individually and together.

Automated tools like Flutter Test, Postman, and Firebase Test Lab were used to validate core functionalities including user authentication, ride booking, SOS alerts, trusted contact sharing, driver verification, recurring rides, payment processing, and admin monitoring.

Overall, the SheRides system was thoroughly tested for performance (ride booking within 3 seconds, SOS alerts within 5 seconds), reliability (99% uptime maintained), security (encryption and role-based access control), and real-world readiness. The system is now ready for deployment and real-world use by women passengers seeking safe transportation in Pakistan.

# **Chapter 07**

## **CONCLUSION**

## 7. Conclusion

The SheRides system has been successfully designed and implemented as a comprehensive women-only ride-sharing platform. Using the MERN stack (MongoDB, Express.js, React, Node.js) for the admin panel and backend, along with Flutter for cross-platform mobile development, the system provides core functionalities such as passenger registration, driver verification, ride booking, SOS alerts, trusted contact sharing, recurring rides, payment processing, and real-time ride tracking with Google Maps API.

Through careful system design, implementation, and rigorous testing including unit, functional, integration, and automated testing, the system has proven to be reliable, secure, and user-friendly for women passengers and female drivers. The SheRides platform meets the specified requirements (FR-1 to FR-8) and provides a seamless experience for passengers, drivers, and administrators.

The integration of external APIs, such as Firebase Authentication for secure login, Google Maps API for real-time tracking, Firebase Cloud Messaging for push notifications, and MongoDB Atlas for cloud database services, further enhances the system's usability, safety, and efficiency.

Key Achievements of SheRides:

*Table 7.1: Key Achievements pf SheRides*

| # | Achievement            | Description                                                                        |
|---|------------------------|------------------------------------------------------------------------------------|
| 1 | Women-Only Platform    | Exclusive ride-sharing platform for female passengers with verified female drivers |
| 2 | SOS Alert System       | Emergency alerts delivered to admin and trusted contacts within 5 seconds          |
| 3 | Trusted Contact Circle | Automatic live location sharing with family members during rides                   |
| 4 | Recurring Rides        | Daily commute automation for working women and students                            |

|   |                            |                                                                |
|---|----------------------------|----------------------------------------------------------------|
| 5 | Mobile Mechanic Assistance | Roadside help with AI-based troubleshooting guidance           |
| 6 | Driver Verification        | Document verification by admin before drivers can accept rides |
| 7 | Admin Monitoring           | Real-time monitoring of live rides, SOS alerts, and complaints |
| 8 | Secure Payments            | Wallet and cash payment options with transaction history       |

Overall, the project demonstrates how modern web and mobile technologies can be leveraged to improve women's mobility, safety, and independence in Pakistan. The SheRides system addresses the critical gap in women's safe transportation and provides a practical, scalable solution for real-world deployment.

## 7.2 Future Work

While the current version of SheRides successfully covers core ride-sharing and safety functionalities for women, there are several opportunities for future enhancement:

### 7.2.1 AI-Based Safety Scoring

In future versions, the system can support AI-based safety scoring for drivers based on historical ride data, passenger feedback, and driving behavior analysis. This will help passengers make more informed decisions when booking rides and will encourage drivers to maintain high safety standards.

### 7.2.2 Integration with Third-Party Payment Gateways

The system can be extended to integrate with popular payment gateways in Pakistan such as Easypaisa, JazzCash, and credit/debit cards. This will provide passengers with more payment options and improve convenience for cashless transactions.

### 7.2.3 Intercity Rides

Future updates could include support for long-distance intercity rides, allowing women to travel safely between cities with verified female drivers. This will expand the platform's reach beyond urban and semi-urban areas.

### **7.2.4 Voice-Activated SOS**

SheRides could introduce voice-activated SOS functionality, allowing passengers to trigger emergency alerts without touching their phones. This hands-free safety feature would be particularly useful in situations where a passenger cannot access their device.

### **7.2.5 Predictive Analytics for Demand Forecasting**

Future versions could include predictive analytics to forecast ride demand in different areas, enabling better driver availability and reduced passenger waiting times. This will improve overall system efficiency and passenger satisfaction.

### **7.2.6 Offline Mode Functionality**

The system can be enhanced to support limited offline functionality for areas with poor internet connectivity. This could include offline ride booking requests that are queued and sent when connectivity is restored.

### **7.2.7 Driver Training and Certification Module**

Future improvements can include an in-app driver training module with video tutorials and safety certifications, ensuring that all female drivers receive proper training before being verified. This will further enhance passenger trust and platform credibility.

### **7.2.8 Emergency Services Integration**

SheRides could integrate directly with local emergency services (police, ambulance, roadside assistance) to provide faster response times during SOS alerts. This will create a complete safety ecosystem for women passengers.

### **7.2.9 Multi-Language Support**

While the current version supports English and Urdu, future versions can add support for additional regional languages such as Pashto, Sindhi, and Punjabi to reach a wider audience across Pakistan.

### **7.2.10 Ride Sharing Feature**

Future updates could include a ride-sharing feature where women from the same area or workplace can share rides, reducing costs and promoting community building among female passengers.

By implementing these enhancements, SheRides has the potential to evolve into a fully integrated women-centric mobility ecosystem, supporting female passengers, drivers, and administrators more effectively and providing advanced safety and transportation services across Pakistan.

## 8. References

- [1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications - used as a guideline for SRS structure and content.
- [2] Karl Wiegers & Joy Beatty, Software Requirements (3rd Edition) - reference for writing and organizing functional and non-functional requirements.
- [3] Aurat Foundation (2021) - Women's Mobility and Safety in Pakistan report, used to understand social context and user needs for SheRides.
- [4] COMSATS University Project Guidelines - institutional formatting and submission requirements for final year projects.
- [5] Firebase Documentation. <https://firebase.google.com/docs> [Accessed: 15-Apr-2026].
- [6] Google Developers. Flutter Documentation. <https://flutter.dev/docs> [Accessed: 15-Apr-2026].
- [7] MongoDB Documentation. <https://docs.mongodb.com> [Accessed: 15-Apr-2026].
- [8] Node.js Documentation. <https://nodejs.org/en/docs/> [Accessed: 15-Apr-2026].
- [9] React.js Documentation. <https://reactjs.org/docs> [Accessed: 15-Apr-2026].
- [10] Google Maps API Documentation. <https://developers.google.com/maps/documentation> [Accessed: 15-Apr-2026].
- [11] inDrive Becomes Pakistan's Top Ride-Hailing App - Bloom Pakistan. <https://bloompakistan.com> [Accessed: 10-Apr-2026].
- [12] Yango Group - Wikipedia. [https://en.wikipedia.org/wiki/Yango\\_\(company\)](https://en.wikipedia.org/wiki/Yango_(company)) [Accessed: 10-Apr-2026].
- [13] Surge in taxi-related abductions, harassment of women in Islamabad - Dawn News. <https://www.dawn.com> [Accessed: 10-Apr-2026].
- [14] Rising concerns: Women's safety and accountability in Pakistan's ride-hailing landscape - The News International. <https://www.thenews.com.pk> [Accessed: 10-Apr-2026].
- [15] Women's Mobility & Harassment Survey - Aurat Foundation, 2021.

